Calibrating Engineers' Trust in LLM-Based Support in the SCAILAB Synthetic Data Pipeline

Madura Herath

For the Degree of Master of Science in Innovation and Design

Master's thesis, advanced level, 30 credits
Examiner: Peter E. Johansson
Supervisor: Adam Palmquist
The Department of Health Sciences, Innovation and Design
Mälardalen University
2026.06.02

Mälardalen University

Abstract

Robotics development depends on large, varied, and well-labelled datasets for training, testing, and evaluating perception and automation systems. However, collecting real-world data can be costly, time-consuming, difficult to scale, and sometimes unsafe or impractical, especially when rare scenarios, varied object poses, lighting conditions, cluttered environments, or safety-critical situations are required. Synthetic data pipelines address this challenge by enabling engineers to generate, configure, inspect, annotate, and evaluate artificial data in a controlled and repeatable workflow.

This thesis examines engineers' trust in Large Language Model (LLM)-based support within the SCAILAB synthetic data pipeline. The purpose of the study is to understand how engineers experience existing LLM-supported functions, identify mismatches between expectations and actual use, explore where additional support could be introduced, and develop human-centred design recommendations for more trustworthy integration. The study is based on a qualitative case study with four internal engineers who had direct experience of the pipeline and its LLM-supported features. Data were collected through semi-structured interviews, task-based think-aloud sessions, and screen-recorded workflow interactions. Interview and think-aloud data were analysed thematically, while screen recordings were examined through behavioural coding to identify verification effort, workflow friction, and possible support opportunities.

The findings show that engineers' trust is conditional, task-dependent, and closely connected to verification and control. Trust increased when outputs were easy to inspect, when the system provided clear context or explanations, and when engineers could decide how changes were applied. Trust decreased when outputs were difficult to verify, when the system misunderstood task context, when results changed unpredictably, or when errors could affect later pipeline stages. The study also identified possible future support areas, including requirement configuration, pipeline construction, parameter tuning, validation, output quality checking, dataset preparation, annotation review, evaluation, and monitoring.

The thesis concludes that trustworthy LLM support in engineering workflows should be staged, reviewable, transparent, and designed around engineer control. Rather than replacing human judgement, such support should reduce effort while preserving verification, accountability, and confidence in downstream data quality.

Keywords: LLM support, trust calibration, synthetic data, robotics, engineering workflow, human-centred AI, SCAILAB

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Dr. Adam Palmquist, for his guidance, support, and constructive feedback throughout this thesis process. His advice helped me shape the research direction, strengthen the methodological approach, and develop the study into its final form.

I am especially grateful to the CEO and co-founders of SCAILAB for giving me the opportunity to conduct this thesis within the company. I would also like to thank the engineers who supported this research. Their experiences and reflections gave this study a clearer view of the practical challenges, verification needs, and trust-related decisions involved in using LLM support in the synthetic data pipeline.

My appreciation also goes to the Department of Health Sciences, Innovation and Design at Mälardalen University for providing the academic environment and support needed to complete this work.

Finally, I would like to thank my family, friends, and everyone who supported me throughout this journey. Their encouragement, patience, and belief in me gave me strength during the challenging stages of this research. This thesis would not have been possible without their continuous support.

Personal Background

My background is shaped by business management, business analysis, and practical experience in working with digital systems and organizational processes. My academic journey began with a Bachelor of Science in Business Management from Cardiff Metropolitan University, where I developed an interest in how organizations operate, how people make decisions, and how problems are addressed in real-world settings.

After completing my undergraduate studies, I entered professional roles in business analysis. In these roles, I worked on understanding business needs, discussing challenges with stakeholders, documenting requirements, and supporting improvements in existing processes and systems. I also had the opportunity to work with stakeholders across different countries, industries, and organizational contexts. These experiences helped me understand that digital solutions are not only shaped by business goals and technical requirements, but also by how people interact with them in practice.

Over time, this experience developed my interest in Human-Computer Interaction. Working between business users, technical teams, and digital systems made me more aware of the importance of usability, user needs, communication, and the context in which technology is used. I became interested in how people understand, experience, and interact with digital products, and how better design can make systems more useful, understandable, and meaningful.

This interest led me to pursue a Master's degree in Innovation and Design at Mälardalen University. The programme provided an opportunity to build on my previous experience while developing a broader understanding of design, innovation, and human-centered problem-solving. It also helped me look beyond requirements and processes, and consider how ideas can be explored, tested, refined, and developed with greater attention to users and their experiences.

Therefore, my position in this thesis is shaped by a combination of business understanding, analytical experience, stakeholder communication, and an interest in Human-Computer Interaction. I approach problems as someone who has worked between business needs, users, and technical teams, and who is interested in how digital solutions can be designed to better support people in practical contexts.

Table of Contents

Abstract 2

Acknowledgement 3

Personal Background 4

Table of Contents 5

Figures 8

Tables 9

Abbreviations 10

Chapter 1 - Introduction 11

1.1 Background 11

1.1.1 Synthetic Data in Robotics Development 11

1.1.2 Importance of Synthetic Data in Robotics 11

1.1.3 Synthetic Data as Part of Robotics Development Workflows 12

1.2 Problem contextualization 13

1.3 Purpose and research questions 14

1.4 Scope 14

Chapter 2 – Literature Review 16

2.1 Introduction 16

2.2 Trust and Appropriate Reliance in AI-Supported Work 16

2.3 LLM-Supported Professional and Engineering Work 17

2.4 Mismatch, Verification Burden, and Trust Breakdown 18

2.5 Trust Calibration in LLM-Supported Engineering Work 19

2.5.1 System Performance and Dependability 20

2.5.2 Output transparency and verifiability 21

2.5.3 User Control and Verification Effort 21

2.5.4 Context and Workflow fit 22

2.5.5 Temporal Trust Formation 22

2.6 Synthetic Data Pipelines as a Workflow Context for Trust 23

2.7 Research Gap 23

2.8 Chapter Summary 25

Chapter 3 – Research Design and Methods 26

3.1 Research Design 26

3.2 Qualitative Case Study Design 26

3.3 Data Collection 27

3.3.1 Semi-Structured Interviews 28

3.3.2 Think-Aloud Sessions 33

3.3.3 Sampling Strategy 35

3.4 Data Analysis 36

3.4.1 Thematic Analysis of Semi-Structured Interviews 38
3.4.2 Thematic Analysis of Task-Based Think-Aloud Data 41
3.4.3 Integration of Interview and Think-Aloud Themes 44
3.4.4 Behavioural Coding of Screen Recordings 47
3.4.5 From Analysis to Design Implications 50
3.5 Ethical Considerations 50
3.6 Research Quality and Trustworthiness 51
3.7 Methodological Reflections 52
Chapter 4 – Empirical Findings 53
4.1 Introduction 53
4.2 Integrated Thematic Findings 53
4.2.1 Trust Depends on Task Risk and Engineer Confidence 53
4.2.2 Trust Requires Easy Verification 54
4.2.3 Trust Depends on Clear Context and Explanation 55
4.2.4 Trust Increases When Engineers Have Control 56
4.2.5 Trust Decreases When LLM Outputs Change Unexpectedly 56
4.2.6 Trust Is Limited by Downstream Risk 57
4.3 Behavioural Findings from Screen-Recorded Task Interaction 58
4.3.1 Behaviour during the Requirement and Project configuration 58
4.3.2 Behaviour during Pipeline Construction and Parameter Tuning 58
4.3.3 Behaviour during the Validation, Preview, and Fixing 59
4.3.4 Behaviour during Generation and Output Quality Checking 60
4.3.5 Behaviour during Dataset, Annotation, Evaluation, and Monitoring 60
4.4 Convergence Between Thematic and Behavioural Findings 60
4.5 Chapter Summary 62
Chapter 5 – Discussion 63
5.1 Introduction 63
5.2 RQ1: Current LLM Support and Engineers' Trust 63
5.2.1 Current LLM Support Is Useful but Conditional 63
5.2.2 Trust Depends on Task Risk and Engineer Confidence 64
5.2.3 Trust Requires Verification Before Use 64
5.3 RQ2: Where Additional LLM Support Could Be Added 65
5.3.1 Early Requirement and Project Configuration Support 65
5.3.2 Pipeline Construction and Parameter Support 65
5.3.3 Pipeline Validation, Preview, and Output Quality Support 65
5.3.4 Dataset, Annotation, and Evaluation Support 66
5.4 RQ3: Mismatches That Reduce Trust 68
5.4.1 Context and Prompt Mismatch 68
5.4.2 Verification Mismatch 69
5.4.3 Control Mismatch 69
5.4.4 Predictability and Stability Mismatch 69
5.4.5 Downstream Risk Mismatch 70
5.5 RQ4: Design Recommendations for Trustworthy LLM Support 70
5.5.1 Make LLM Support Staged and Reviewable 71

5.5.2 Support Verification Before Reliance 71
5.5.3 Make Context, Assumptions, and Reasoning Visible 71
5.5.4 Keep Engineers in Control of Changes 71
5.5.5 Add Downstream Risk Safeguards 72
5.6 Chapter Summary 73
Chapter 6 – Conclusion 74
6.1 Summary of Conclusions 74
6.2 Theoretical Contributions 75
6.3 Practical Implications 76
6.4 Limitations 77
6.5 Future Research 78
6.6 Usage of Tools 79
References 80
Annex 1: SCAILAB Synthetic Data Pipeline full workflow 86
Annex 2: Interview analysis overview 89
Annex 3: Think-aloud analysis overview 91
Annex 4: Behavioural coding overview 93
Annex 5: Analytical Traceability Across Thematic Analyses 95
Table A: Interview thematic analysis traceability 95
Table B: Think-aloud thematic analysis traceability 98

Figures

Figure 1: Analytical framework of conditions shaping trust calibration in LLM-supported engineering work....20
Figure 2: Methodological framework 27
Figure 3: Data analysis roadmap.....37

Tables

Table 1: Data collection summary....28
Table 2: Summary of the interview questions....32
Table 3: Tasks followed in the think-aloud session....34
Table 4: Sampling summary....36
Table 5 : Analysis Stage....38
Table 6: Theme integration summary....47
Table 7: Convergence of behavioural and thematic data....61
Table 8: Pipeline stages and possible LLM support opportunities....68

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
HCI	Human–Computer Interaction
LLM	Large Language Model
ML	Machine Learning
POC	Proof of Concept
R&D	Research and Development
URL	Uniform Resource Locator

Chapter 1 - Introduction

1.1 Background

1.1.1 Synthetic Data in Robotics Development

Robotics has become an increasingly important part of industrial and technological development. The International Federation of Robotics reported that 4,281,585 industrial robots were in operation worldwide in 2023, according to the World Robotics 2024 report. The World Robotics 2025 report later showed that 542,000 industrial robots were installed globally in 2024 (International Federation of Robotics, 2024; International Federation of Robotics, 2025).

Data plays a central role in developing, training, and deploying robots. To train the robots, organisations require different forms of data, such as visual and spatial data, as well as interaction and control data. These data are used to support robots in object recognition, position and pose estimation, scene understanding, action planning, and performance improvement across different environments (Singh et al., 2024).

Collecting such data in real-world settings is often difficult. Training robots requires large volumes of data, and collecting this data often involves substantial time investment, repeated environment setup, and, in some cases, manual annotation. This process can therefore become expensive and difficult to sustain in terms of cost, labour, and scalability. For example, training robots often requires data spanning different object poses, lighting conditions, backgrounds, clutter levels, and task settings. In addition, some scenarios are particularly difficult to capture in physical environments, especially those involving unstable interactions or safety-related conditions.

Instead of relying solely on data collected in real environments, robotic engineers use simulation and rendering tools to generate images, scenes, trajectories, and interactions that represent such scenarios. Robotic engineers can decide which kinds of cases to include, how much variation to create, and how often certain difficult conditions should occur (Tobin et al., 2017; James et al., 2019). These simulated data are known as synthetic data, and, because of their ease of generation, they are becoming increasingly popular and useful in robotics development. In the robotics development process, synthetic data is recognised as a practical resource because they help developers generate data that are difficult to obtain directly and makes it easier to repeat experiments and vary conditions during system development (Singh et al., 2024).

1.1.2 Importance of Synthetic Data in Robotics

In robotic environments, organisations use synthetic data in several different ways. One important use of synthetic data is robot perception. Before a robot can grasp, sort, inspect, or navigate, it usually needs to detect objects and interpret its surroundings. To identify those surroundings, robots need labelled datasets. Producing such datasets manually can be difficult when custom objects or complex environments are involved. For instance, annotating images

of specialised industrial components in a densely cluttered warehouse scene can take considerable time. As a result, synthetic data are used to generate labelled images and scenes in simulation, thereby reducing the need for manual data collection and annotation (Tobin et al., 2017; Singh et al., 2024).

Another example is robotic grasping. In robotic grasping, robots must handle unfamiliar objects, varying arrangements, uncertain viewpoints, and cluttered environments. In these tasks, synthetic data help engineers generate grasping scenes and object variations for robotic training (James et al., 2019; Xie et al., 2023).

Beyond these areas, synthetic data are also used for pose estimation, bin-picking, and robot control. In these areas, robotic engineers require data that reflects object orientation, motion, physical interaction, and varying environmental conditions. Simulation enables the generation of such data before real-world testing, which is particularly useful when physical trials are slow, expensive, or risky. In this way, synthetic data support not only visual training tasks but also broader aspects of robotics development, where interaction with real hardware is difficult to scale in the early stages of development (Chen et al., 2022; Tan et al., 2018; Yu et al., 2019).

1.1.3 Synthetic Data as Part of Robotics Development Workflows

Robotics organisations use software-based environments to generate and prepare synthetic data. These environments are commonly referred to as synthetic data pipelines. Within these pipelines, engineers can define scenarios, configure generation settings, produce synthetic outputs, inspect results, add or refine annotations, and assess whether the generated data is suitable for subsequent use. However, synthetic data generation should not be considered a single-step process, as the pipeline brings together several connected activities through which synthetic data are created, reviewed, and prepared for downstream development tasks (Long et al., 2024).

This emphasises the significance of the pipeline not only as a technical tool but also as a framework that shapes how engineers engage with data during development. From this perspective, the pipeline becomes part of the decision-making process, as engineers must determine whether the generated data reflects the intended conditions, supports model training and testing, and is appropriate for later stages of development. Therefore, its value depends not only on the volume of data generated but also on the extent to which it supports evaluation, quality control, and integration into practical workflows (Singh et al., 2024; Long et al., 2024).

Synthetic data generation involves a combination of manual and automated processes. These tasks involve engineers' interpretation and practical judgement. As a result, robotics development depends not only on data generation but also on how those data can be understood, assessed, and applied within the broader development process (Parasuraman et al., 2000; Baxter and Sommerville, 2011).

1.2 Problem contextualization

SCAILAB is a company focused on generating synthetic data for robotics, perception, and automation. To generate synthetic data, SCAILAB has developed its own synthetic data pipeline platform. The platform comprises all the standard functions required to generate and prepare synthetic data, including data generation, annotation, and dataset training. Using the pipeline, SCAILAB produces synthetic data for a wide range of clients across application domains, including logistics, bin picking, and quality inspection (SCAILAB, 2026).

Within the SCAILAB synthetic data pipeline, engineers' initial observations indicate that pipeline tasks require substantial mental effort and sustained attention. For an example, an incorrect parameter configuration can have downstream effects on other activities in the development workflow and across subsequent pipeline stages. As a result, the effort involved in synthetic data development lies not only in the generation process itself but also in the repeated review, adjustment, and evaluation required to make the generated data usable in practice. (Long et al., 2024; James et al., 2019; Singh et al., 2024).

As a result, SCAILAB introduced LLM-based support in selected parts of the pipeline to assist engineers. The intention behind this support is to reduce manual effort and assist engineers with demanding tasks. However, based on engineers' initial expectations and early impressions, the support does not appear to provide the level of assistance expected. This concern becomes important because support in this setting is expected to contribute to work that is already effortful and time-consuming. If the support does not significantly reduce effort, engineers may still need to spend time interpreting outputs, verifying their appropriateness, and making corrections before the results can be used (Barke et al., 2023; Mozannar et al., 2024; Tankelevitch et al., 2024).

In this thesis, trust is understood as engineers' situated confidence that LLM-based support can be used under conditions of uncertainty and potential risk. This understanding builds on trust-in-automation literature, where trust is connected to vulnerability, uncertainty, and appropriate reliance rather than blind acceptance (Mayer et al., 1995; Lee and See, 2004). It is also informed by Ivarsson and Lindwall's (2023) discussion of trust and suspicion, which helps frame trust as a condition that enables activity to continue without repeated questioning or suspicion. In the SCAILAB context, trust refers to whether engineers can carry LLM-supported outputs forward in the synthetic data pipeline while still preserving verification, correction, accountability, and confidence in downstream data quality. Therefore, trust does not mean accepting LLM outputs as final, but judging them as sufficiently understandable, inspectable, controllable, and aligned with the current pipeline task to be used responsibly.

Therefore, the research problem addressed in this thesis concerns the limited understanding of how engineers experience LLM-based support within a synthetic data generation pipeline, how early experiences of limited or uncertain support shape trust and reliance, and what these experiences imply for the design of more appropriate workflow-based support. By examining this issue in a real-world engineering environment, the study aims to provide a more practical

understanding of how to integrate LLM-based support into multistage development pipelines (Baxter and Sommerville, 2011; Wang et al., 2024).

1.3 Purpose and research questions

The purpose of this research is to investigate how engineers experience trust when using LLM-based support within the SCAILAB synthetic data pipeline, identify mismatches between expectations and experience, examine how those experiences can be improved, and inform human-centred design recommendations.

The following questions guide the research:

1. Which parts of the synthetic data pipeline currently use LLM support, and how does this shape engineers' trust?
2. Where could LLM support be added in the synthetic data pipeline to further increase engineers' trust?
3. What types of mismatches between engineers and LLM support lead to reduced trust in the synthetic data pipeline?
4. What design recommendations can improve the trustworthiness of LLM support in the synthetic data pipeline?

1.4 Scope

The primary target group is SCAILAB's internal engineers who interact directly with the synthetic data pipeline. With its LLM-based support features, the study seeks to examine their practical experience of trust, verification effort, and workflow fit. In addition to this primary group, several stakeholder groups are relevant to the study. SCAILAB, as an organisation, is a direct stakeholder, as the study's design recommendations are intended to inform the development and integration of LLM-based support into its platform. SCAILAB's client organisations working in domains such as logistics, bin picking, and quality inspection are indirect stakeholders, since the trustworthiness and usability of the pipeline affects the quality and reliability of the synthetic data they depend on. The broader HCI and human-centred AI research community represents an academic stakeholder group, as the study aims to contribute empirical and design knowledge about LLM-based support in professional engineering workflows that extends beyond the immediate organisational context.

Within this scope, the research examines existing points of integration of LLMs into the synthetic data pipeline. It also investigates trust and reliance in context by analysing how engineers decide when to accept, verify, modify, or reject LLM outputs, and how these decisions vary across tasks and workflow stages. On this basis, the study seeks to develop human-centred design recommendations aimed at improving practice (Shneiderman, 2020; Zhang et al., 2020; Kocielnik et al., 2019; Cai et al., 2019).

The study does not focus on developing new machine learning models or enhancing model performance. Technical benchmarking and evaluation were also outside the scope since they were not primary goals. Instead, the assessment of system performance centres on trust, verification effort, usability, and downstream impact within the pipeline. These boundaries

keep the analysis grounded in the real-world conditions under which LLM-based support may prove useful, reliable, and trustworthy in complex engineering settings.

Chapter 2 – Literature Review

2.1 Introduction

This chapter reviews the key literature and theories related to trust in LLM-based support within the SCAILAB synthetic data pipeline. It focuses on four key themes: trust and appropriate reliance in automation and AI-assisted work; LLM-supported professional and engineering activities; issues such as mismatch, verification workload, and trust breakdown; and design factors that influence trust calibration. Collectively, these themes form the theoretical and empirical foundation of the study (Mayer et al., 1995; Lee and See, 2004; Hoff and Bashir, 2015).

This chapter reviews these areas of the literature in sequence. It first reviews trust and appropriate reliance in AI-supported work, then examines what recent studies of LLM-supported professional tasks reveal about usefulness, verification burden, and misalignment. It then synthesises this literature into a framework of design conditions relevant to trust calibration. The chapter concludes by identifying the research gap addressed by this research.

2.2 Trust and Appropriate Reliance in AI-Supported Work

In the trust literature, trust is considered meaningful when there is genuine uncertainty and real vulnerability. The influential organisational model proposed by Mayer et al. (1995) defines trust as a willingness to accept vulnerability based on positive expectations of another party. When this idea is applied to automation, the core concept remains the same: trust becomes important when the user cannot fully predict, inspect, or control what the system will do, yet must still decide whether to depend on it. Lee and See (2004) therefore argue that the design goal in automation is not maximum trust but appropriately calibrated trust, meaning trust that rises or falls in relation to the system's actual capabilities and limits.

Ivarsson and Lindwall (2023) offer a complementary perspective by connecting trust to suspicion in human–AI interaction. They suggest that trust can be understood as a background condition that allows shared activity to continue, while suspicion interrupts this continuity and makes the other party's actions questionable. This perspective is useful for the present study because engineers using LLM-based support in the SCAILAB pipeline are not only assessing whether an output is correct, but also whether it can be carried forward without creating additional uncertainty, verification effort, or downstream risk.

A key distinction in this literature is the difference between trust and reliance. Trust is an internal judgement, whereas reliance is the behavioural outcome that may follow from that judgement (Lee and See, 2004; Hoff and Bashir, 2015). A user may believe that a system is generally competent and still avoid using it for a particular task because the consequences of error are too high or the verification cost is too great. Some AI-supported studies consider use, acceptance, or compliance as direct evidence of trust. Those studies explain how people may rely on a system they do not really trust because of workload, time pressure, or organisational expectation. In addition, they may reject a system they broadly trust because the current task demands stronger guarantees. Research on algorithm aversion shows that people may reject

algorithmic advice after observing errors, while allowing users some control over algorithmic output can reduce this aversion (Dietvorst et al., 2015; Dietvorst et al., 2018).

This distinction is particularly crucial in AI-assisted professional tasks. Hoff and Bashir (2015) show that trust in automation is shaped by a mixture of dispositional, situational, and learned factors. Users bring prior beliefs and habits to the encounter and then update those beliefs through repeated experience in specific contexts. The literature also makes clear that users do not calibrate trust perfectly. Yin et al. (2019) show that people's trust in machine learning models is shaped by both stated accuracy and experienced accuracy, and that these do not always align neatly in users' judgements. This finding is important because engineers using LLM-based tend to evaluate the tool not just on published capability claims. Instead, they are more likely to build trust through repeated interactions that demonstrate useful results, visible failures, and ambiguous near-misses.

Recent studies also indicate that miscalibration involves more than undertrust. Klingbeil et al. (2024) show that users over-rely on AI advice even when it conflicts with available contextual information and their own interests. Mehrotra et al. (2024) similarly show, in a systematic review of appropriate trust in human–AI interaction, that the human-AI interaction field still lacks a single stable definition of appropriate trust and relies on diverse proxies, measures, and intervention strategies. As a result, in this context, trust should not be treated as a simple attitude but must be examined as a situated judgement about when, where, and under what conditions LLM support should be used within a real workflow. Most classical trust models remain conceptually valuable but were largely developed for automation. However, this literature establishes a foundation for the challenges of LLM-supported work.

2.3 LLM-Supported Professional and Engineering Work

Recent studies indicate that LLMs can support a range of professional and engineering activities. These include coding, debugging, documentation, requirements analysis, reviews, and exploratory problem-solving (Vaithilingam et al., 2022; Barke et al., 2023; Hou et al., 2024). This perspective on LLM support for professional and engineering activities suggests that generative systems can reduce the effort required to start or build something from scratch. The literature suggests that LLM support can accelerate initial creation and enable users to explore more options than would be possible manually. From this perspective, LLM support can be understood as a tool that boosts productivity by accelerating specific tasks of the work process (Sarkar et al., 2022; Liang et al., 2024).

Furthermore, existing studies show that the benefits of LLM support are often conditional and vary across different tasks. As an example, in LLM-supported work, speed does not always lead to overall efficiency improvements. The time saved from using LLMs can be offset by the time spent verifying, correcting errors, and adapting LLM outputs to match user requirements. For instance, while users may spend less time creating a draft from scratch, they might need more time evaluating whether the generated content is reliable, relevant, and fit for purpose. In professional settings, this is important because minor errors or incorrect assumptions can lead

to hidden downstream costs that are not immediately apparent in the early stages (Mozannar et al., 2024; Tankelevitch et al., 2024; Wang et al., 2024).

The existing literature also has methodological constraints relevant to this study. Many empirical findings come from limited tasks, short interaction times, or controlled environments similar to benchmarks (Vaithilingam et al., 2022; Barke et al., 2023; Hou et al., 2024). While useful for understanding what LLMs can achieve in specific task moments, these studies are less effective at explaining how trust develops when outputs need to be integrated into a professional engineering context. In such settings, a suggestion's value depends not only on its plausibility or initial time savings but also on whether it can be understood, verified, adapted, and implemented seamlessly, without hidden costs or future problems (Liang et al., 2024; Wang et al., 2024).

This literature is significant because it emphasises key patterns relevant to professional settings in which generative AI is incorporated into daily routines, rather than providing a direct blueprint for the SCAILAB pipeline. Most research indicates that LLM support is most effective when outputs are straightforward to inspect and recover, and when they pose low risk. In contrast, trust tends to decrease when errors are hard to detect, when context becomes difficult to interpret, and when users are held responsible for results (Lee and See, 2004; Hoff and Bashir, 2015; Mehrotra et al., 2024). This highlights the significance of verification, workflow compatibility, and contextual consistency. However, the specific conditions that influence these factors remain unclear, particularly in a multi-stage synthetic data environment. These studies serve as supplementary evidence rather than direct sources for the current research, and their findings do not apply directly to a pipeline setting. (Wang et al., 2024; Long et al., 2024).

2.4 Mismatch, Verification Burden, and Trust Breakdown

Research on AI-assisted programming Mozannar et al. (2024) indicates that a significant share of user effort is shifted toward reading, validating, and fixing AI suggestions. This verification process can add managerial overhead because users must determine when to inspect outputs closely, intervene, or rely on the output (Vaithilingam et al., 2022). Productivity gains may also be constrained by the time needed to understand unfamiliar generated content and fix errors afterwards.

Mismatch is therefore not only a quality problem but also a coordination problem. Zamfirescu-Pereira et al. (2023) demonstrate that users often struggle to craft prompts that yield reliable, context-aware responses. Meanwhile, Tankelevitch et al. (2024) suggest that generative AI systems impose significant metacognitive challenges on users. When the LLM's output is unpredictable or requires repeated interactions, user trust can decline even if the system is occasionally helpful.

These studies are relevant to multi-stage synthetic data engineering workflows because they consider the impact of mismatch on downstream processes. Often, an early misaligned suggestion does not reveal its costs immediately. Instead, it may become visible later through

debugging, validation issues, reconfiguration, or rework. This can be explained by the fact that trust depends not only on immediate interactions but also on how well users can trace consequences throughout the workflow. However, the current literature examines mismatch at the point of interaction, whether a suggestion is accepted, corrected, or rejected.

2.5 Trust Calibration in LLM-Supported Engineering Work

This section synthesises the reviewed literature into a set of trust-related conditions that are relevant to the present study. These conditions are not presented as a definitive or universally agreed-upon theory of trust in LLM-supported engineering. Instead, they serve as an analytical framework developed for this study and grounded in common themes from trust-in-automation studies, human–AI interaction research, and empirical studies of LLM-supported professional tasks (Lee and See, 2004; Hoff and Bashir, 2015; Liao et al., 2020; Hou et al., 2024). Their purpose is both conceptual and methodological: conceptually, they help organise the literature around the main conditions that appear to shape appropriate reliance; methodologically, they provide a structured basis for examining how engineers in the SCAILAB pipeline experience, judge, and respond to LLM-based support in practice.

In an LLM-supported work environment, users assess not just the accuracy of the outputs but also the system's predictable behaviour. They consider whether suggestions are easy to verify, whether users can intervene and recover when needed, and whether the assistance matches the actual task and workflow requirements (Mayer et al., 1995; Lee and See, 2004; Parasuraman et al., 2000; Mehrotra et al., 2024). These clusters are analytically different but not fully separable in practice, as engineers' trust judgments are influenced by how these conditions interact during specific tasks and workflow stages. The trust literature suggests that trust can increase or decrease depending on multiple interacting conditions. It provides the basis for the following framework.

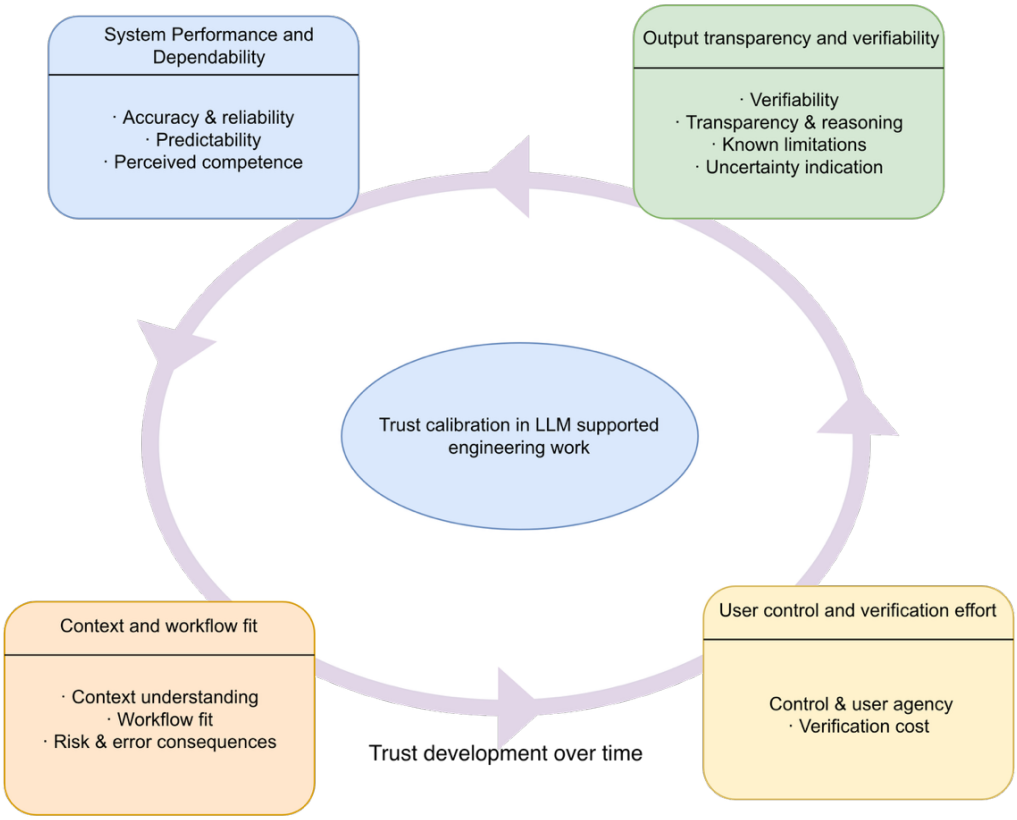


Figure 1: Analytical framework of conditions shaping trust calibration in LLM-supported engineering work

2.5.1 System Performance and Dependability

Research on trust in automation has consistently treated dependable performance as a basic condition for reliance. Engineers are unlikely to rely on a system that is only intermittently useful. System accuracy refers to whether an output is correct, appropriate, or sufficiently aligned with the task requirements. Reliability, on the other hand, concerns the system's ability to perform consistently across multiple instances. Mayer et al. (1995) frame ability as a central foundation of trust, and Lee and See (2004) similarly argue that predictable performance is essential for users to appropriately calibrate their reliance. In LLM-supported work, these concerns remain central because a system that is occasionally impressive yet erratic in practice is difficult to integrate into a workflow.

Beyond reliability, predictability plays an important role in shaping reliance. A system may be reliable overall, but it can still be difficult to trust in use if users cannot form clear expectations about when it will succeed or fail. Nourani et al. (2020) demonstrate that early interactions and first impressions influence subsequent trust, while Kelly et al. (2023) emphasise the importance of accurate mental models of AI's capabilities and limitations for establishing suitable trust and effective teamwork. These studies redirect focus from mere performance to the user’s ability to predict AI behaviour.

Perceived competence is often considered alongside accuracy, reliability, and predictability. Users assess whether the system appears capable of handling important tasks, though these perceptions may not always align with its actual capabilities. Some research indicates that fluent outputs can lead users to judge competence too quickly, while other studies show that visible signs of weakness or repeated corrections can reduce perceptions of competence, even if the system performs well technically. The literature does not define a specific threshold for when competence is considered sufficient; instead, it emphasizes that competence should be clear, reliable, and relevant to the task to support trust calibration.

2.5.2 Output transparency and verifiability

A second cluster emphasises the forms of support that help users accurately assess the system. This is important because, even if a system appears capable, trust can decline when users cannot verify whether an output is correct, complete, and appropriate for the task. Lee and See (2004) link proper reliance to effective monitoring. Additionally, human–AI research indicates that judgment remains vital in LLM-supported work. Here, verifiability refers to the practical ability to determine if an output meets standards, while transparency and explanation are valuable only if they help improve that assessment.

Transparency and explanation are often presented as solutions to trust issues. Liao et al. (2020) demonstrate that users ask different questions depending on their goals. Explanations are helpful only when they target the relevant evaluative problem. However, Bansal et al. (2021) and others warn that explanations may boost acceptance without improving judgment. Mehrotra et al. (2024) similarly conclude that interventions such as explanations, confidence cues, and communication of uncertainty do not necessarily lead to appropriate trust. Consequently, the effectiveness of these interventions varies with task type, user objectives, and the methods used to measure trust. While explanations can help calibrate trust, they can also lead to unwarranted confidence.

Known limitations, expectation-setting, and uncertainty indication are grouped because they influence how users develop realistic expectations. Kocielnik et al. (2019) demonstrate that users often work more effectively with imperfect AI when system limitations are clearly communicated, whereas Zamfirescu-Pereira et al. (2023) highlight the challenge users face in identifying these limitations independently. Tankelevitch et al. (2024) add that generative AI imposes a significant metacognitive burden on users, who must decide when to trust or intervene. Consequently, trust calibration relies less on abstract explanations and more on whether the system helps users form realistic, actionable expectations about its capabilities, limitations, and proper use.

2.5.3 User Control and Verification Effort

Trust is influenced by users' capacity to redirect, adapt to, and recover from system behaviour. Shneiderman (2020) argues that trustworthy AI should maintain meaningful human oversight rather than replace user judgment for the sake of convenience. Dietvorst et al. (2018) show that people are more willing to use imperfect algorithms when they retain some ability to intervene. In LLM-supported work, this explains the importance of accepting or rejecting a suggestion,

as well as of steering, editing, refining, and reversing support when it begins to drift away from local requirements. This suggests that LLM support can still be valuable despite its imperfections, and that a system that is difficult to redirect might be distrusted more rapidly. The verification cost should be assessed alongside the control, as the practical usefulness of support depends not only on the theoretical ability to verify outputs but also on the actual cost of verification.

2.5.4 Context and Workflow fit

A fourth cluster examines context and workflow fit. The literature suggests that trust depends not only on whether outputs appear plausible but also on whether they align with local requirements, constraints, and the stage-specific demands of the work. Understanding the context is important because even technically coherent suggestions may be poor candidates for reliance if they do not match the actual conditions under which engineers must use them.

This problem extends to workflow compatibility and socio-technical integration. Baxter and Sommerville (2011) contend that the value of a system is rooted in the interaction between its technical and social elements. Meanwhile, Parasuraman et al. (2000) demonstrate that varying levels of automation impose different requirements on human operators. Recent research on generative AI supports this perspective by showing that tools are evaluated not only by their outputs but also by how they influence the distribution of effort across the broader workflow. A suggestion might be technically accurate yet a poor fit if it is poorly timed, lacks proper context, or is difficult to validate.

Risk and the consequences of errors are grouped because trust is influenced not just by immediate output quality, but also by potential future impacts if a weak suggestion is used. In multi-stage workflows, errors might only become apparent later, so reliance should be assessed based on visibility, recoverability, and downstream effects, rather than just immediate plausibility. This grouping is key because trust in LLM-supported engineering depends on more than isolated performance; it also relies on how support integrates into the workflow in a usable, timely, and accountable manner that aligns with actual work processes.

2.5.5 Temporal Trust Formation

Trust develops over time through repeated interactions with useful outputs. For instance, it helps users recognise visible errors, recoverable failures, and recurring system behaviour patterns. Yin et al. (2019) demonstrate that perceived accuracy significantly influences trust, whereas Nourani et al. (2020) indicate that initial experiences with intelligent systems can shape expectations for subsequent interactions. This suggests that trust is not a fixed judgment made once, but an ongoing process that is continually updated as users compare their previous beliefs with observed system performance in specific tasks.

The temporal aspect is also important because early positive experiences can encourage productive experimentation. Conversely, repeated weak alignment may result in defensive behaviours such as frequent checking, limited use, or avoidance of the tool in high-risk tasks. Therefore, experience and trust development are not background factors; they directly

influence user behaviour. Users with consistently positive prior experiences tolerate the same level of uncertainty or verification effort differently than those with no prior experience.

2.6 Synthetic Data Pipelines as a Workflow Context for Trust

A key limitation identified in the literature so far is that most empirical studies originate primarily from programming, prompting, or general human–AI decision support, rather than specifically from synthetic data pipelines. Yet recent synthetic-data research makes it clear that this workflow has features highly relevant to trust. Long et al. (2024), in their survey of LLM-driven synthetic data generation, curation, and evaluation, frame the field around a generic workflow rather than a single-generation event. Their review highlights that effective synthetic data work involves not only producing candidate outputs but also filtering, curating, evaluating, and iteratively refining them. This approach aligns more closely with the context explored in this thesis than with standard code-completion studies.

Recent work on LLM-based synthetic data generation, especially for text and code, also highlights prompt-based generation, retrieval support, iterative self-refinement, and post-generation filtering or weighting as important quality-control mechanisms (Nadas et al., 2025). They also observe that quality assurance remains a challenge, as LLM-generated text and code can appear fluent or useful while still containing factual inaccuracies, limited realism, or amplified bias (Nadas et al., 2025).

This literature strengthens the chapter's argument in two ways. First, the literature shows that verification burden, workflow fit, and downstream consequences are not abstract trust concepts imposed from outside; they are already built into the complex engineering workflow itself. Second, it highlights a remaining gap. Although synthetic-data literature is increasingly attentive to generation, curation, and evaluation, it says much less about how engineers develop trust in LLM-based support while carrying out those activities in practice.

2.7 Research Gap

The literature provides a strong foundation for understanding trust in AI-supported work, but it also reveals a clear gap. Classical trust-in-automation research has established that trust is shaped by uncertainty, perceived competence, controllability, and the user’s ability to monitor and intervene in system behaviour (Mayer et al., 1995; Lee and See, 2004; Hoff and Bashir, 2015). More recent human–AI interaction research extends this perspective by showing that trust in AI systems is influenced not only by performance, but also by explanation, expectation-setting, transparency, and the way systems are embedded in work practices (Cai et al., 2019; Liao et al., 2020; Shneiderman, 2020; Mehrotra et al., 2024). At the same time, empirical studies of LLM-supported work show that these systems can be experienced as both useful and burdensome, particularly when verification effort, contextual mismatch, and hidden checking costs become part of the interaction (Vaithilingam et al., 2022; Barke et al., 2023; Mozannar et al., 2024; Tankelevitch et al., 2024; Hou et al., 2024). Altogether, these studies show that trust is not a simple matter of adoption or preference; instead, it is a situated judgement tied to whether reliance on the system feels warranted in practice.

However, much of the recent empirical literature still focuses on relatively bounded settings, especially software development tasks such as code completion, programming assistance, and other discrete interactions, where task boundaries are clearer and downstream consequences are more limited (Vaithilingam et al., 2022; Barke et al., 2023; Hou et al., 2024). These studies are highly relevant, particularly in showing that generative AI can support speed and exploration while also introducing new forms of monitoring and repair. Yet they do not fully explain how trust is formed, disrupted, and recalibrated when LLM-based support is embedded in a multi-stage industrial workflow in which outputs move across stages, local constraints are strong, and the consequences of weak suggestions may not become visible until later in the process. The literature is therefore stronger on trust in AI tools in general than on trust in stage-dependent LLM support within complex engineering systems.

A second gap relates to workflow integration. Existing studies show that factors such as inspectability, verification effort, task compatibility, and user control are important for building trust and ensuring proper reliance (Lee and See, 2004; Hoff and Bashir, 2015; Liao et al., 2020; Mehrotra et al., 2024). Even so, fewer studies have explored how these factors vary across the internal stages of a real engineering pipeline. This is important in environments such as synthetic data generation, where support cannot be judged only by immediate usefulness at the point of generation. It should also be judged by whether outputs can be interpreted, verified, and integrated into established work practices without creating hidden rework, additional correction costs, or workflow disruption (Parasuraman et al., 2000; Wang et al., 2024).

A third gap concerns the kinds of users and settings that have been studied. Much of the broader trust literature has developed through general models of automation trust or through studies conducted in relatively controlled environments (Mayer et al., 1995; Lee and See, 2004; Hoff and Bashir, 2015). There is still comparatively limited empirical research on professional engineers' use of LLM-based support within real organisational workflows. In these settings, trust is not only an individual judgment about whether a tool appears helpful. It also relates to professional responsibility, team expectations, and the requirement to justify AI-assisted work in accordance with established engineering procedures (Baxter and Sommerville, 2011; Glikson and Woolley, 2020). These aspects remain underexplored in the current literature.

A final gap relates to design knowledge. While literature offers numerous general recommendations on transparency, uncertainty communication, explanation, and human control, it provides less guidance tailored to industrial settings where support tools need to match domain-specific tasks, local standards, and verification routines (Cai et al., 2019; Kocielnik et al., 2019; Shneiderman, 2020; Zhang et al., 2020).

2.8 Chapter Summary

This chapter has explored that trust in LLM-supported engineering work should be explored as calibration. Traditional automation literature highlights that trust is meaningful in the face of uncertainty, emphasising that the goal is appropriate reliance rather than maximum use. Recent human–AI studies add complexity by showing that explanation, transparency, and cues about uncertainty do not automatically enhance judgment; instead, they assist only when they enable users to access system accurately. Empirical research on LLM-assisted tasks supports this view, indicating that perceived productivity improvements often depend on hidden effort, contextual fit, and the user's capacity to handle verification responsibilities.

The chapter also demonstrates that the context of the thesis requires a more specific bridge than programming-assistant studies alone can provide. The synthetic-data literature increasingly treats the process as a workflow comprising generation, curation, and evaluation, highlighting issues such as verification, workload, context alignment, and downstream effects. Based on this, the chapter developed a grouped analytical framework comprising 13 trust-related components. This framework helped identify a key research gap: the limited understanding of how trust in LLM-based support is built, maintained, and disrupted within a multi-stage industrial synthetic data pipeline. Consequently, the next chapter focuses on the research design and methods used to empirically explore this gap.

Chapter 3 – Research Design and Methods

3.1 Research Design

The thesis adopts a qualitative approach to examine engineers' experiences of LLM-based support within the synthetic data pipeline. It focuses on how they interpret and assess the system's outputs, and how these experiences shape trust in practical use. The research questions explore engineers' situated experiences, professional judgement, and workflow practices in relation to the synthetic data pipeline. Accordingly, the study employs methods that enable participants to describe how they make trust-related decisions, respond to system-generated outputs, and understand LLM support in real-world work contexts (Creswell and Creswell, 2018; Merriam and Tisdell, 2016).

The design combines reflective interview data, task-based think-aloud verbal data, and behavioural coding of screen-recorded task interaction. This allowed trust to be examined not only through participants' own descriptions, but also through how it emerges during their workflow activities.

3.2 Qualitative Case Study Design

Case study research is suitable when the aim is to investigate a phenomenon within its real-world context, especially when the phenomenon cannot be meaningfully separated from the environment in which it occurs (Yin, 2018). In this research, the phenomenon is engineers' trust in LLM-based support, while the context is the SCAILAB synthetic data pipeline. Since trust in this setting is shaped by workflow conditions, verification demands, downstream dependencies, and local task requirements, it cannot be properly understood outside the organisational and technical environment in which the support is used.

The case is bounded by one organisation, one synthetic data pipeline, and one primary user group, internal engineers who interact with the pipeline and with its LLM-based support. Establishing the boundary is important because the study's purpose is not to produce statistical generalisations across organisations. The case study design, therefore, supports the thesis by enabling close attention to the interactions among the tool, user, and workflow in a real socio-technical setting (Baxter and Sommerville, 2011; Yin, 2018).

The case study design is also suitable for the four research questions. To answer the first question, the study must identify where LLM support is currently embedded in the pipeline and how engineers experience those points of interaction. To answer the second, the study explored the broader workflow to identify where additional support may be useful, meaningful, and trustworthy. The third question requires examining trust-reducing mismatches in actual work situations. The fourth requires recommendations that are grounded in the specific conditions of the case. For these reasons, the study adopts a qualitative case study design, which allows it to remain closely connected to the real conditions of engineering work in the SCAILAB pipeline (Yin, 2018).

The empirical study was organised into four connected phases. The first phase focused on exploring engineers' trust in existing LLM-based support through semi-structured interviews. The second phase focused on task-based think-aloud sessions supported by screen recording to examine how engineers interacted with the support during relevant pipeline tasks. The third phase involved analysing these materials in relation to the research questions and the trust-related framework developed from the literature. The fourth phase translated the final themes and behavioural findings into design implications and requirements. Those design requirements informed the development of design recommendations for trustworthy LLM support in the SCAILAB synthetic data pipeline (Yin, 2018; Braun and Clarke, 2006).

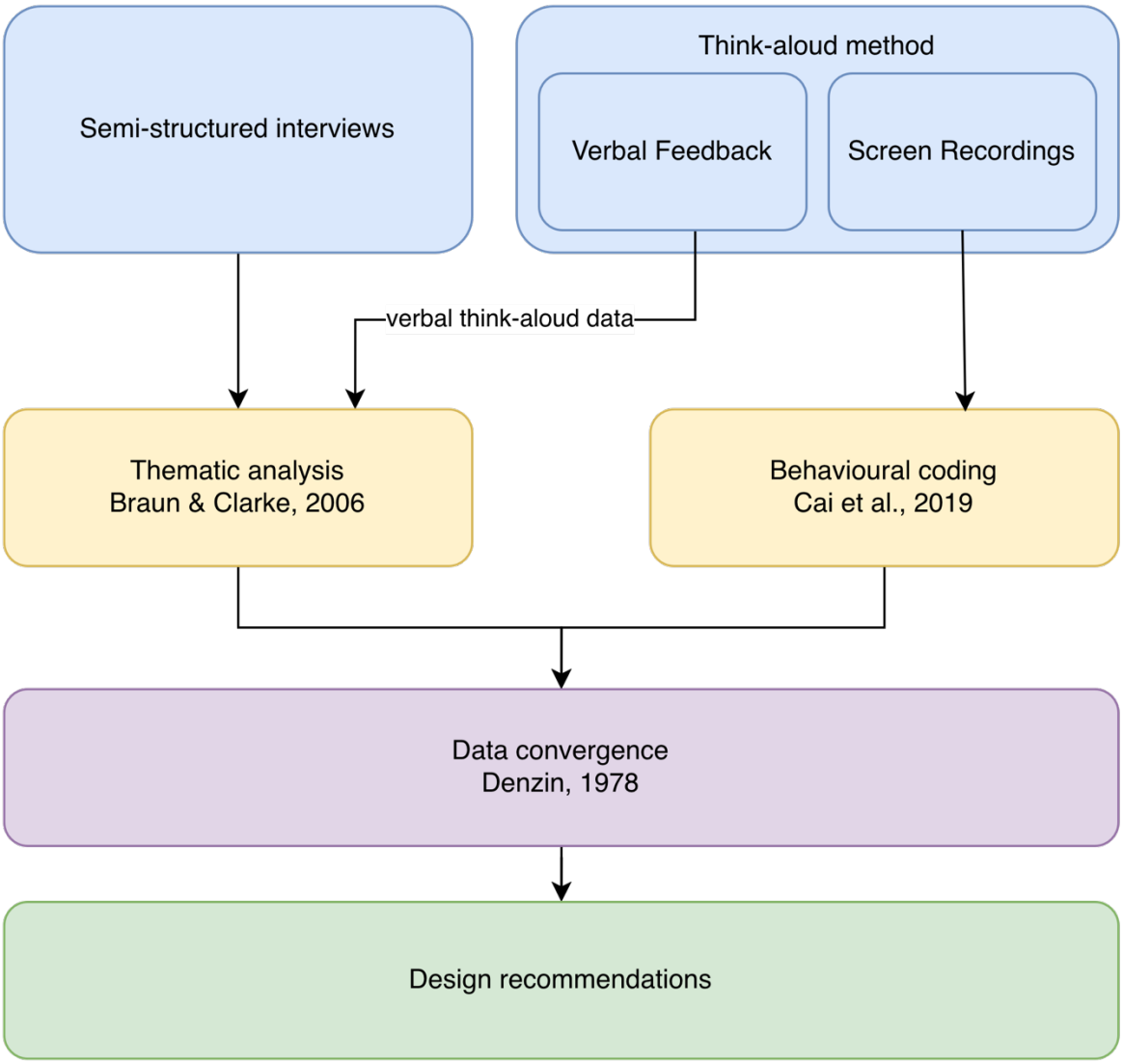


Figure 2: Methodological framework

3.3 Data Collection

Data collection in this study consisted of semi-structured interviews and task-based think-aloud sessions supported by screen recording. These methods provided different aspects of engineers' experiences with LLM-based support in the SCAILAB synthetic data pipeline.

The semi-structured interviews provided access to participants' expectations, trust-related concerns, and reflections on system use. The task-based think-aloud sessions enabled examination of how engineers use LLM support during relevant pipeline tasks. Together, these methods strengthened the study through triangulation and provided a richer basis for understanding how trust was formed, maintained, or reduced in use (Denzin, 1978; Merriam and Tisdell, 2016; Ericsson and Simon, 1993).

Data source	What it captured	Purpose in the study
Semi-structured interviews	Participants' expectations, reflections, and experiences of current LLM-based support	To understand how engineers described and interpreted LLM support, trust, verification effort, workflow fit, and expectations
Think-aloud verbal data	Participants' spoken reasoning during the two existing LLM-supported tasks	To understand how engineers reasoned during actual use of LLM support
Screen recordings	Observable workflow actions across all 15 task areas	To identify observable workflow friction points, such as configuration, checking, correction, validation, and review activities, that may indicate potential opportunities for future LLM support. These opportunities were treated as researcher-identified possibilities rather than direct participant requests.

Table 1: Data collection summary

3.3.1 Semi-Structured Interviews

Semi-structured interviews were used as one of the main data collection methods in this study. This method enabled an exploration of how engineers experienced LLM-based support in the SCAILAB synthetic data pipeline. It also provided rich insights into how they made sense of the system's outputs and how these experiences influenced trust in practice. Since the research questions focus on trust formation, trust reduction, workflow fit, and engineer–LLM mismatches, it was important to collect detailed data of participants' own experiences. Semi-structured interviews, therefore, made it possible to examine not only whether engineers used the support but also how they understood it, when they found it useful, and under what conditions they considered it trustworthy or untrustworthy (Kallio et al., 2016; Lee and See, 2004; Hoff and Bashir, 2015).

A semi-structured format was chosen for its balance between consistency and flexibility. The interviews needed enough structure to ensure that all participants were asked about the same core trust-related issues. Still, they also needed to remain open enough to allow participants to explain their experiences in their own words and raise issues that were not fully anticipated in advance. This was important because the study aimed not only to document engineers' experiences with LLM-based support, but also to examine how those experiences related to

their expectations of the system. In this sense, the interviews enabled exploration of where the support matched participants' expectations, where it failed to do so, and how such expectation–experience gaps influenced trust, verification behaviour, and perceived usefulness (Cai et al., 2019; Kallio et al., 2016).

To conduct the interviews, an interview guide was developed for this study (refer to Table 2). The guide was designed to elicit engineers' subjective experiences of trust in the LLM-based components of the SCAILAB synthetic data pipeline. It was structured around open-ended anchor questions, which allowed the researcher to guide the discussion while still allowing follow-up questions and further exploration when needed (Lee and See, 2004; Hoff and Bashir, 2015; Cai et al., 2019; Liao et al., 2020; Shneiderman, 2020).

The interview guide was organised around thirteen trust components identified from the literature review. These included accuracy and reliability, predictability, verifiability, transparency and reasoning, control and user agency, perceived competence, known limitations, context understanding, workflow fit, uncertainty indication, verification cost, risk and consequences of errors, and experience and trust evolution. Instead of asking whether participants trust or distrust the system, the guide was designed to examine the dimensions that shape trust in AI-supported work. As an example, participants were asked whether outputs were accurate and consistent, whether the system behaved predictably, how easy outputs were to verify, whether the system seemed to understand the context of the task, how much effort verification required, and how trust had changed over time through repeated use (Lee and See, 2004; Hoff and Bashir, 2015; Cai et al., 2019; Liao et al., 2020; Shneiderman, 2020).

The structure of the guide also supported the progression of the interview. It began with background and warm-up questions about the participant's role in the pipeline, the tasks they worked on most, the parts of the pipeline where LLM support currently exists, and their general impression of the support. After that, the interview moved into more focused areas related to trust. During the interviews, the participants began by answering within their own work contexts. Then they moved toward more specific reflections on trust, system behaviour, verification, workflow fit, and the consequences of error. This helped maintain the logical structure of the discussion while getting rich context from the participants.

A total of four participants were interviewed in this study. All participants were engineers working on the SCAILAB synthetic data pipeline and were selected for their direct involvement in tasks that used LLM-based support or were considered relevant to it. During the interviews, background information was collected on each participant's role in the pipeline, the tasks they most frequently worked on, and their overall experience with the existing LLM-based support. The interviews were conducted in person in March 2026, and each interview lasted between 45 and 60 minutes. This timeframe provided sufficient time to discuss both participants' backgrounds and the trust-related themes addressed in the interview guide.

In addition to exploring engineers' general experiences, the interviews helped identify what participants expected from the LLM-based support before and during use, which provided an

important basis for comparing those expectations with the behaviour captured in the task-based think-aloud sessions and screen recordings.

Category	Trust component	Interview questions	Key literature
Background and warm-up	Participant background	1. How much experience do you have in the field?	—
	Pipeline role and LLM exposure	1. Can you briefly describe your role in the development of the SCAILAB pipeline and the tasks you work on most regularly? 2. Which parts of the pipeline currently have LLM-based support, as far as you are aware? And how often do you interact with those components in a typical week? 3. Before we go deeper, when you think about the LLM support in the pipeline right now, what is your general gut feeling about it? Would you say you trust it, distrust it, or something more nuanced?	Lee & See (2004); Hoff & Bashir (2015)
System performance and dependability	Accuracy and reliability	1. Looking back, when you integrated the LLMs into the pipeline, were your initial expectations of the LLM's accuracy close to what you found in practice?	Yin et al. (2019); Mayer et al. (1995)
	Predictability	1. Do you feel like the LLM gives pretty consistent results no matter who is using it, or does it vary between different engineers? 2. Imagine you are under time pressure to complete a pipeline stage. Would you use the LLM support, knowing the output might need correction? Why or why not?	Vaithilingam et al. (2022); Zamfirescu-Pereira et al. (2023); Hoff & Bashir (2015)
	Perceived competence	1. Do you recognise a pattern where early errors shaped long-term trust in the LLM support?	Mayer et al. (1995); Nourani et al. (2020)
Output transparency and verifiability	Verifiability	1. When the LLM produces an output, how easy is it for you to check whether that output is actually correct? 2. Are there pipeline stages or task types where you cannot tell whether the LLM's output is right or wrong without additional steps? 3. Sometimes engineers use LLM tools to speed up tasks, and sometimes to explore possible solutions. Do you recognise these two modes in how you use the system?	Barke et al. (2023); Tankelevitch et al. (2024); Liang et al. (2024)

Category	Trust component	Interview questions	Key literature
	Transparency and reasoning	1. Which type of explanation would be most valuable to you in your pipeline work — global explanations or local explanations?	Liao et al. (2020); Liang et al. (2024)
	Known limitations	1. Are there situations or input types where you now know the LLM is unreliable?	Cai et al. (2019); Zamfirescu-Pereira et al. (2023); Kocielnik et al. (2019)
User control and verification effort	Control and user agency	1. When the LLM generates an output you are not fully satisfied with, how easy is it to modify, adjust, or override it? 2. Do you feel that the current LLM support in the pipeline achieves a good balance between automation and human control?	Dietvorst et al. (2018); Liang et al. (2024); Shneiderman (2020); Vaithilingam et al. (2022)
Context and workflow fit	Context understanding	1. Have you experienced situations where the LLM produced technically correct outputs that did not fit the specific context of your pipeline task?	Liang et al. (2024); Barke et al. (2023)
	Workflow fit	1. If you could redesign when and how LLM support appears in your workflow, what would you change?	Liang et al. (2024); Cai et al. (2019); Mozannar et al. (2024); Barke et al. (2023)
	Risk and consequences of errors	1. Has there been an LLM error that had significant downstream consequences, and how did it affect your trust?	Wang et al. (2024); Mozannar et al. (2024); Lee & See (2004)
Temporal trust formation	Past experience and trust evolution	1. If a new colleague joined your team and asked for your honest advice on how to use and how much to trust the LLM support in the pipeline, what would you tell them?	Nourani et al. (2020); Lee & See (2004); Hoff & Bashir (2015)

Table 2: Summary of the interview questions

3.3.2 Think-Aloud Sessions

Task-based think-aloud sessions were conducted to capture engineers' reasoning as they worked through structured tasks in the SCAILAB synthetic data pipeline. Although LLM support had already been introduced for two functions, the method was not limited to observing these existing features. Instead, it examined how engineers navigated the wider pipeline and how trust-related judgments emerged during their work.

For this reason, the full 15-task workflow was included, even though only two tasks directly involved existing LLM-supported functions. Although the think-aloud sessions covered the full 15-task workflow, the thematic analysis of verbal data focused only on the two tasks that involved existing LLM-supported functions. Verbal comments from non-LLM tasks were retained as contextual notes, while the screen recordings from all 15 tasks were used for behavioural coding to identify where additional LLM support could be explored. During the session, participants were not explicitly asked whether they wanted LLM-based support for each specific task. Instead, the researcher used the observed activities to identify potential workflow points where future LLM support could be explored. These points should therefore be understood as researcher-identified opportunities rather than direct participant requests.

The task-based think-aloud sessions were conducted in May 2026 with the same four participants who took part in the interviews. The session lasted approximately 60 minutes. During the session, participants were asked to work through the same pipeline tasks while verbalising what they were noticing, expecting, checking, accepting, rejecting, correcting, or questioning. Screen recording and audio recording were collected during the session. The audio recording captured participants' spoken reasoning, while the screen recording captured their interaction with the pipeline.

To ensure consistency across participants, all participants were given the same starting point. At the beginning of each session, the SCAILAB pipeline was already open on the computer, and participants started from the same initial project state. Only the SCAILAB pipeline and screen recording software opened during the session. This helped maintain a consistent task environment and ensured that the recorded interaction focused on the participant's engagement with the pipeline itself.

The workflow (refer to Annex 1) was organised into approximately fifteen task areas that represented the main stages of the synthetic data pipeline. These tasks were not treated as isolated tasks. Instead, they formed a connected workflow that progressed from project setup and configuration to generation, verification, dataset preparation, evaluation, and, later, reporting or monitoring. This structure allowed the study to observe where participants configured the pipeline, checked results, corrected issues, and where additional support might be useful.

The main task areas used in the think-aloud sessions were:

Task ID	Main task area	What participants were expected to do
T01	Open the pipeline	Access the pipeline through URL
T02	Requirement entry	Enter project requirements and configure basic project information
T03	Data import and asset/source selection	Create asset through the Procedural Editor
T04	Labeling setup and class/schema definition	Define or review label classes, schema settings, or annotation structure
T05	Pipeline node configuration/construction	Configure or construct pipeline nodes required for the workflow/ Create pipeline node with the support of LLM
T06	Pipeline parameter tuning	Adjust parameters and review settings that influence generation or processing
T07	Pipeline linking and configuration	Connect pipeline components and configure relationships between workflow elements
T08	Pipeline validation and preview/fix loop	Preview the pipeline, identify issues, validate outputs, or make corrections
T09	Synthetic data generation	Run the synthetic data generation process
T10	Output quality assessment and verification	Inspect generated outputs and judge whether they are suitable
T11	Training dataset curation and split	Prepare or review training dataset structure and data splits
T12	Annotation verification and dataset preparation	Review annotations and prepare dataset material for later use
T13	Model selection and training configuration	Review or configure model-related training settings
T14	Evaluation and downstream testing/reporting	Examine evaluation or testing outputs and consider downstream results
T15	Documentation, deployment, and monitoring	Review documentation, deployment-related steps, or monitoring needs

Table 3: Tasks followed in the think-aloud session

In this task structure, T03 and T05 included existing LLM-supported functions, while the remaining tasks represented surrounding workflow stages. These stages were included to understand how LLM support fits into the broader synthetic data pipeline and to identify where future support may be needed.

Rather than treating the Procedural Editor and Generate New Module function as isolated AI features, this study examined them within the broader pipeline workflow. This reflected how participants encountered LLM support. In this context, outputs from the Procedural Editor were assessed based on how well they fit the current task, how easily they can be verified, and how

they might affect later stages of the pipeline. Generated modules were evaluated similarly, particularly regarding whether participants understood their assumptions and felt confident carrying them forward.

The think-aloud sessions produced two forms of data. The first was verbal think-aloud data, which included participants' spoken reasoning during the task. This was later analysed thematically. The second was screen-recorded interaction data, which captured observable actions such as configuring requirements, selecting assets, adjusting parameters, inspecting outputs, validating results, correcting issues, and continuing through the workflow. This was later analysed through behavioural coding. These spoken comments helped explain participants' reasoning, while screen actions helped identify where workflow friction, verification effort, or decision-making burden occurred. These were then interpreted as possible support opportunities (Ericsson and Simon, 1993; Cai et al., 2019; Barke et al., 2023; Tankelevitch et al., 2024).

3.3.3 Sampling Strategy

Participants in this study were selected using purposive sampling. The study required participants with direct experience with the synthetic data pipeline and LLM-based support in that environment. This was necessary because trust could only be examined through the accounts of those who had used the system in practice. The purpose of the sampling strategy was not to achieve statistical representation, but to collect rich, relevant accounts from participants who could describe how trust developed in practice and how support fit or failed to fit their workflow (Patton, 2015; Merriam and Tisdell, 2016).

The study centred on internal engineers engaged with the SCAILAB synthetic data pipeline. Participants were selected for their active role in pipeline tasks and experience with LLM-based support tools. These criteria guaranteed that participants could offer well-founded insights into their interactions with the system, assess its outputs, and describe how their trust grew through repeated use.

The company context affected the sampling strategy. Since SCAILAB is a startup with a small number of employees, the available pool of participants was limited. The value of the sample, therefore, lies in the relevance of participants' experiences rather than in the number of participants. This is consistent with qualitative case study research, in which depth, contextual knowledge, and direct experience are more important than large sample sizes (Yin, 2018; Patton, 2015).

Participant	Experience	LLM-support experience	Relevant workflow involvement	Participation in study
P01	Long-term engineering/design experience	Moderate / occasional use	Concept definition, POC development, interface design, customer-specific task adaptation	Interview + think-aloud + screen recording
P02	Robotics-industry/research experience	Extensive use	Platform integration, ML pipeline testing, asset/component generation, compliance support	Interview + think-aloud + screen recording
P03	Engineering and machine-learning experience	Moderate / occasional use	R&D, tool and method exploration, POC development	Interview + think-aloud + screen recording
P04	Engineering and machine-learning experience	Extensive use	Platform integration, ML pipeline testing, asset/component generation,	Interview + think-aloud + screen recording

Table 4: Sampling summary

3.4 Data Analysis

The data analysis was conducted in several connected stages. The purpose of the analysis was to understand how engineers experienced LLM-based support, how they made trust-related judgements, which mismatches reduced trust, and where additional support could be useful within the SCAILAB synthetic data pipeline.

The analysis was organised into two main layers. The first layer focused on verbal data from semi-structured interviews and LLM-supported parts of the think-aloud sessions. These data were analysed thematically because they captured participants' expectations, reflections, and reasoning during actual use of LLM support.

The second layer focused on screen-recorded task behaviour across the full 15-task workflow. These recordings were analysed through behavioural coding to identify where participants configured, checked, corrected, validated, and reviewed work in the pipeline. This layer was used to locate workflow points where additional LLM support might be explored. These points were based on researcher interpretation of observable workflow activity, not on direct participant confirmation that LLM support was desired at every stage. The Non-LLM verbal comments from the think-aloud sessions were not used to develop the main themes of LLM trust. They were retained only as contextual notes when they helped interpret observed workflow difficulty.

The semi-structured interviews captured reflective experience related to trust. The think-aloud verbal data captured participants' reasoning during the two LLM-supported tasks. The screen recordings captured observable workflow behaviour. Thematic analysis was used to identify recurring patterns of meaning across participants' accounts. Braun and Clarke (2006) describe thematic analysis as a flexible method for identifying, analysing, and reporting patterns within qualitative data. This approach aligned with the study's aim, which was not to quantify mentions of trust but to investigate how trust emerged through recurring concerns about verification effort, contextual mismatch, control, predictability, and downstream risk.

The analysis also aligned with recommendations for strengthening trustworthiness in qualitative thematic analysis. Nowell et al. (2017) emphasise that transparency is important because it shows how the researcher moves from data extracts to codes, themes, and interpretation. In this study, coding spreadsheets supported the process by providing an audit trail that links participant evidence to codes, codes to potential themes, and these themes to the final integrated findings outlined in Chapter 4.

Behavioural coding was used because the screen recordings provided evidence that was different from the interview and think-aloud data. The aim was not only to understand what participants said, but also to examine how trust-related concerns appeared in their actions during task performance. Screen recordings have been used in interaction and workflow research to capture task sequences, user actions, and real work practices (Tang et al., 2006; Zhao et al., 2023). In this study, they helped identify moments where participants configured requirements, adjusted parameters, validated outputs, checked quality, or continued through the pipeline. This made it possible to locate workflow points where additional LLM support could be explored.

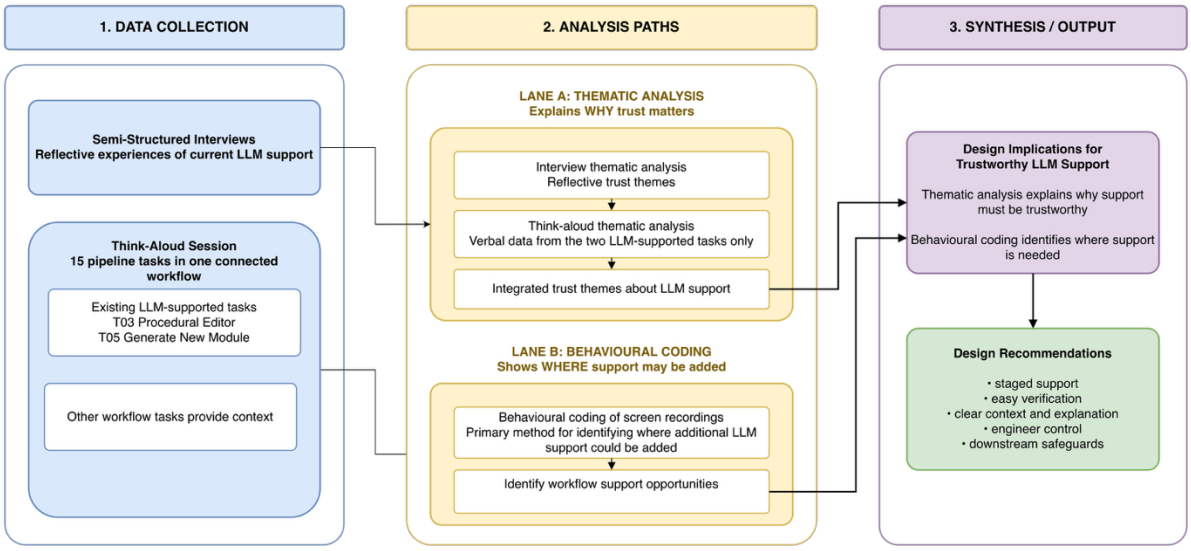


Figure 3: Data analysis roadmap

Figure 3 summarises the analytical roadmap used in the study. It shows how the data collection methods were separated into two analysis paths: thematic analysis of interview and LLM-supported think-aloud verbal data, and behavioural coding of screen recordings from the full 15-task workflow. These two paths were then synthesised to develop design implications and recommendations for trustworthy LLM support.

Overall, the analysis followed four stages.

Analysis stage	Data source	Main purpose
Stage 1	Semi-structured interview transcripts	Identify reflective trust-related themes
Stage 2	Think-aloud verbal data from LLM-supported tasks only	Identify interaction-based themes about actual LLM use
Stage 3	Interview themes and think-aloud themes	Integrate overlapping themes into one thematic structure
Stage 4	Screen recordings	Code observable workflow behaviour and identify support opportunities

Table 5 : Analysis Stage

3.4.1 Thematic Analysis of Semi-Structured Interviews

The semi-structured interviews provided the main reflective insight into participants' trust in LLM-based support in the pipeline. The analysis followed a combined deductive and inductive thematic approach (refer to Annex 2). This was used because the study was informed by existing trust literature, but it also needed to remain open to issues that emerged from participants' own experiences with the SCAILAB pipeline.

The analysis began with familiarisation. The interview transcripts were read several times to understand each participant's expressions. During this stage, attention was given not only to direct statements about trust but also to comments in which trust was expressed indirectly through caution, checking, uncertainty, confidence, or frustration. This was important because participants did not always use the word "trust" directly. In many cases, trust was established through practical descriptions of what they needed to do before using LLM output.

After familiarisation, meaningful evidence segments were extracted from the interview transcripts. For example, when one participant explained that there was *"It depends. Theres always verification effort invloved."* (P03, interview), this was treated as a trust-related statement because it showed that LLM output was not accepted without further checking. In another case, participants said that users needed to *"Try as much as possible and double check everything."* (P02, interview). These comments were coded as verification-related concerns because they indicated that trust depended on the engineer's ability to inspect and validate the result before using it.

The first part of the coding was deductive. This meant that some codes were informed by existing literature on trust. These included verification, predictability, transparency, user control, context understanding, workflow fit, and appropriate reliance (Lee and See, 2004; Hoff and Bashir, 2015). These concepts helped guide the analysis, but they were not used as fixed categories. Instead, they provided a starting point for identifying trust-related meanings in the interview data.

At the same time, the analysis also used inductive coding. This meant that new codes were created from participants' own expressions when the data revealed issues more specific to the SCAILAB pipeline. These inductive codes included hidden changes, counter-prompt loops, prompt dependency, early framing value, and downstream concern. For example, when a participant described being *"But you're then saying, you're basically going in circles around the things that you're not happy with."* (P02, interview), this was coded as a counter-prompt loop and linked to iterative instability. The comment suggested that repeated interaction with the LLM could sometimes increase uncertainty rather than resolve the issue. Similarly, when a participant described the system changing *"It does this what it tells specifically, but then suddenly changes something on a completely different and which messes up something else."* (P02, interview), this was coded as a hidden change because it showed that the LLM could affect areas outside the intended scope.

Other examples showed how participants calibrated trust depending on the task. For instance, the statement *"For me, depends on the complexity of the task. If it's a small fix in specific, very specific code files or so on, I'm going to do it myself by hand."* (P02, interview) was coded as task-complexity calibration. This showed that participants did not trust the LLM equally across all tasks. A simple or familiar task could make the support feel more acceptable, whereas a complex or risky task would require more caution. This was also evident in comments such as *"I mean, cybersecurity concerns, saying that before we launch things, we always have to check that there are now API keys somewhere exposed, and other kind of vulnerabilities which are easy to fix." (P02, interview)* where participants connected trust to the task's potential risks and sensitivity.

Some codes also focused on prompt quality, explanation, and context. For example, the comment *"Different people define the problem [in a] different way […] if you don't have a structure[d] way of defining […] the context." (P03, interview)* was coded as a prompt-structure concern because it showed that participants believed output quality depended on how clearly the task was described. Similarly, the comment *"Okay, explain me why line by line or like what are you doing […] because I would like […] to understand […] what is the core and why the this system works like that"* (P01, interview) was coded as an explanation-related concern because it showed that participants wanted to understand the reasoning behind an LLM suggestion before accepting or adapting it.

Other comments showed the importance of control and review. For example, when a participant said *"In the development, and our own development, uh, we are using different modes. […] I have to review them myself before I've merged them into our platform code"*

(P02, interview), this was coded as user control because it showed that participants wanted to remain responsible for approving outputs before they were integrated into the platform. The statement *"But right now there is a lot of a lot of feedback needed to, like, human interactions, until the for a final fix"* (P02, interview) was also coded as a control-related concern because it showed that participants saw current LLM support as requiring human steering rather than full automation.

After coding, related codes were grouped together. Codes related to checking, inspection, testing, and validation were grouped because they all describe the effort required to determine whether an output can be used. Codes related to missing information, unclear prompts, explanations, and weak task understanding were grouped together because they all emphasised the importance of context and structure. Codes related to review, approval, editing, manual correction, and unintended changes were grouped because they described the need for engineer control.

These grouped codes were then developed into candidate interview themes. At this stage, the analysis moved from describing individual comments toward interpreting broader patterns. Repeated comments such as *"The output should be verified before go further"* (P03, interview) and *"If you just want to trust it, do not. Do not assume it is right. It is encouraged to use LLMs but be aware that they might be wrong. Use them as long as you can still vouch for the result yourself. Use common sense when you use LLMs."* (P04, interview) supported a candidate theme around verification and testability. Comments such as *"I would say, like, the more I know about the problems, the less, is the chance that I won't use, and also if I would use it, I would, like to use it a assistance, not just taking the output, and the inputting it, without checking it, at all"* (P03, interview). Comments such as *"I don't see that behaviour here, but that's probably also because then I'm very specific with the things with the LLM I want to add a lot […] But always, when we fix something, a broken other thing, and then at some point we're back to the beginning"* (P02, interview) and *"right now you try to change something or maybe something that change you lost track of something else. […] It would be great […] to put some constraints […] So like focus on […] specific prompt or query."* (P01, interview) supported a theme around iterative instability and hidden changes. Comments such as *" I mean, from my side, I would say that somehow I still prefer to keep an eye on what is going on"* (P01, interview) and *"it gives you intermediate steps where you can verify what is going on. So you can make changes."* (P02, interview) supported a theme around assistive and steerable support rather than autonomous action.

The analysis also identified a theme around workflow value and downstream risk. For example, comments about *"I would integrate it into better templates or more templates for the pipeline, because the setup is difficult to understand. But for things related to actual design or databases, we would not want to trust an LLM"* (P04, interview) indicated that participants valued using LLMs for early problem framing and task structuring. However, comments such as *"I mean, it's basically, you, you produce some trash, which you would need to remove. […] it's basically user experience, which gets lost, and maybe some angry users"* (P02, interview) and *"If an LLM error affects datasets, labels, or training datasets, then the data becomes wrong and this*

compromises everything later in the pipeline, including the training part" (P04, interview) showed that weak outputs could create additional work or affect later stages of the pipeline. This meant that trust was not only linked to whether the immediate output looked useful, but also to whether it could create problems later in the workflow.

The themes were then reviewed against the coded interview evidence. This review assessed whether each theme was supported by participant evidence, internally coherent, and clearly distinct from other themes. Some codes were moved during this stage because they fitted better under another theme. Some theme labels were also refined to better represent the data's meaning. This helped ensure that the themes were not based on isolated comments, but on recurring patterns across participants' accounts.

This interview analysis produced a reflective understanding of trust in the pipeline. It showed how participants described LLM support after using the system, including where they found it useful, where they became cautious, and what kinds of outputs required additional checking. The combined deductive and inductive approach enabled the analysis to connect to existing trust literature while also capturing pipeline-specific concerns emerging from participants' own accounts. These interview themes were later compared with the think-aloud verbal themes.

3.4.2 Thematic Analysis of Task-Based Think-Aloud Data

The thematic analysis of think-aloud verbal data focused on the verbal segments produced during the two existing LLM-supported tasks. This was done because these segments captured participants' reasoning as they directly interacted with LLM support. Although participants verbalised comments across the wider 15-task workflow, comments from non-LLM tasks were not used to develop the main LLM trust themes. Instead, they were retained as contextual notes and used only when they helped interpret behavioural evidence from the screen recordings.

This separation helped avoid treating general workflow frustration as evidence of trust or distrust in LLM support. It also allowed the thematic analysis to focus specifically on actual LLM use, while behavioural coding of the screen recordings examined the broader pipeline and identified opportunities for additional support.

The think-aloud verbal data were analysed using a hybrid thematic analysis approach, which involved both deductive and inductive coding (refer to Annex 3). Deductive coding was used first to connect the analysis to the trust-related framework developed in the literature review. As a result, codes were based on concepts already identified in Chapter 2, such as verifiability, predictability, transparency, context understanding, workflow fit, control and user agency, verification cost, and risk of error. These deductive codes helped identify parts of the think-aloud data that were clearly related to established trust conditions in AI-supported work.

At the same time, the analysis also used inductive coding. This meant that new codes were created from participants' own verbal comments when the data revealed issues specific to the SCAILAB pipeline. These inductive codes included Parameter verification before trust, Useful

starting point requiring inspection, Staged and reviewable generation, Explanation before acceptance, Task-dependent usefulness of LLM support, Prompt-context dependency, Prompt-to-prompt instability, Workflow ambiguity, Unexpected scope change, Downstream pipeline risk, and Early verification of downstream risk. Although the study was guided by existing literature on trust, the analysis needed to remain sensitive to concerns that emerged directly from participants' work with the pipeline task.

The analysis began by reviewing the think-aloud verbal data together with the task context. Each comment was interpreted in relation to what the participant was doing at that moment. For example, when a participant said, *"I need a checklist of what to double check here"* (P02, think-aloud session), the comment was understood in relation to parameter checking in the pipeline. Deductively, this comment was linked to verifiability and verification cost because it showed that trust depended on whether the output could be checked. Inductively, it was also coded as Parameter verification before trust because the participant's concern was specifically about checking parameter values before using the generated result.

The think-aloud verbal data were then divided into meaningful interaction segments. These segments were not based only on time. They were based on moments where participants expressed a clear judgement, concern, uncertainty, or decision during the task. For example, the comment *"It gives a useful starting point, but I don't treat it as final"* (P04, think-aloud session) was treated as one meaningful segment because it showed both usefulness and caution. Deductively, this was coded under verification and appropriate reliance. Inductively, it was coded as Useful starting point requiring inspection because the participant saw the output as helpful but not ready to use without review.

After segmentation, each verbal segment was coded using both deductive and inductive codes. For example, when a participant said, *"I want intermediate steps where I can verify before it continues"* (P02, think-aloud session), this was deductively coded under control and user agency because it showed that the participant wanted to guide the system. It was also inductively coded as Staged and reviewable generation because the comment specifically requested that the LLM reveal its plan before generating a full module. This showed that trust was connected to staged interaction and review before accepting a larger system action.

Some comments showed concern about unexpected system changes. For example, when a participant said, *"It changed something I did not ask for, so now I have to inspect the surrounding setup"* (P02, think-aloud session), this was deductively coded under the categories of predictability and user control. It was also inductively coded as unexpected scope change because the participant was reminded that the LLM might change parts of the setup outside the intended scope. This showed that trust could decrease even when the system was trying to solve a problem, because the participant still needed confidence that the change would stay within the expected scope.

Other comments showed the importance of explanation and transparency. For example, one participant stated, *"A high-level explanation is helpful before I decide how to use the module"*

(P03, think-aloud session). Deductively, this was coded under the categories of transparency and reasoning. Inductively, it was coded as Explanation before acceptance. The comment indicated that the participant did not accept the output solely because the system suggested it. They wanted enough explanation to judge whether the connection made sense in the workflow.

The coding also captured task complexity and perceived competence. For example, the statement "*For asset generation, it helps, but only when the task is not too complex"* (P02, think-aloud session) was deductively coded under the perceived competence and task risk categories. Inductively, it was coded as Task-dependent usefulness of LLM support because the participant recognised the value of LLM support, but only within certain task limits. Similarly, the comment *"The global idea is useful, but implementation still needs review."* (P02, think-aloud session) showed that LLM support could help with early framing, while still requiring engineer judgement before use.

The analysis also included comments about context and workflow fit. For example, when a participant said, *"Without a structured way of defining the module, different prompts give different results"* (P03, think-aloud session), this was deductively coded under context understanding and workflow fit. Inductively, it was coded as Prompt-context dependency because the participant linked output reliability to how clearly the task context was given. This showed that participants did not evaluate LLM outputs only by whether they looked plausible. They evaluated whether the output fitted the actual pipeline task.

Downstream risk was also coded both deductively and inductively. For example, the comment *"When the generated module changes after each prompt, confidence goes down"* (P04, think-aloud session) was deductively coded under risk and consequences of error. Inductively, it was coded as Downstream pipeline risk because the participant was specifically concerned that a generated module could affect later pipeline connections. Another comment*, "Small errors here can move into later stages, so I would rather verify early." (P04, think-aloud session)*, was coded as Early verification of downstream risk. These comments were important because the SCAILAB pipeline is a multi-stage workflow, in which weak outputs can lead to later checking, correction, or rework.

Once the verbal segments had been coded, related deductive and inductive codes were grouped into candidate think-aloud themes. Codes related to checking parameters, inspecting outputs, testing results, and verifying before continuing were grouped because they all indicated that trust depended on the ability to verify outputs. Codes about unclear assumptions, open prompts, missing context, and workflow ambiguity were grouped because they showed that trust depended on context and explanation. Codes about staged generation, review points, editable suggestions, and approval before continuation were grouped because they showed that engineers wanted control over how LLM support entered the workflow.

Entries where no verbal feedback was available were not coded thematically. This was important because the task structure needed to remain consistent across participants, but themes should only be developed from actual verbal evidence. Therefore, rows without verbal

feedback were retained to ensure participant-task coverage, while only spoken participant comments were used for thematic interpretation.

3.4.3 Integration of Interview and Think-Aloud Themes

After the semi-structured interview data and the think-aloud verbal data had been analysed separately, the two sets of themes were compared and integrated. This step was important because the two methods produced different forms of verbal evidence. The interviews captured participants' reflective experiences of LLM-based support, while the think-aloud sessions captured participants' reasoning during task performance. Analysing them separately first helped preserve this difference. Integrating them afterwards enabled the identification of trust-related issues that appeared across both forms of evidence.

The candidate think-aloud themes were reviewed against the coded verbal segments from the two LLM-supported tasks. This review examined whether each theme was supported by direct evidence of interaction and whether it captured a meaningful pattern in how participants used, checked, steered, or questioned LLM support. The think-aloud themes did not duplicate the interview themes. Instead, they showed how similar trust concerns appeared as interaction patterns during actual LLM use.

The interview analysis produced themes reflecting participants' broader experiences with LLM-based support. The initial interview themes were:

1. Engineers' trust in LLM support depends on task familiarity and risk
2. Engineers trust LLM outputs when they can check and verify them
3. Clear prompts and context help engineers produce more trusted outputs
4. Engineers prefer to guide and control the LLM rather than let it work independently
5. Engineers lose trust when LLM outputs change unexpectedly
6. LLMs are useful for early ideas, but weak outputs can create downstream risks

The think-aloud verbal analysis produced a second set of themes. These themes were more closely connected to task performance because they emerged from participants' comments while directly using the two LLM-supported functions. In this integration, the think-aloud themes refer only to verbal data from the two existing LLM-supported tasks. The identified think-aloud themes were:

1. Engineers judged LLM trust based on the task situation
2. LLM outputs were useful starting points rather than finished results
3. Engineers needed to check outputs before relying on them
4. Clear prompts helped engineers control the quality of LLM support
5. Step-by-step interaction made LLM support feel safer
6. Unexpected LLM behaviour created extra work and reduced trust

The purpose of integration was not to force both datasets into the same structure. It was to examine whether the same trust-related issues appeared across the two methods. The

integration was therefore based on conceptual overlap rather than theme labels alone. A theme from the interview analysis was compared with a theme from the think-aloud analysis to determine whether both described the same trust mechanism.

The trust-related components identified from the literature were used as an analytical guide during the analysis, rather than as fixed categories for the final findings. They helped guide attention to issues such as verification, predictability, control, context understanding, workflow fit, and downstream risk. However, the final themes were shaped by how these issues appeared in the empirical material. Some components appeared together in participants' accounts and were therefore combined into broader themes. For example, verifiability and verification cost were treated together because participants often discussed trust in relation to the effort required to check outputs. Similarly, transparency, reasoning, context understanding, and workflow fit were closely connected in the data because participants linked explanation to whether an output made sense in the current task. Other components, such as experience and trust evolution, informed the interpretation across themes rather than becoming separate findings.

The thirteen trust-related components from the analytical framework in Chapter 2 were used as sensitising concepts during the early stages of analysis. They guided attention to trust-related issues such as accuracy, verifiability, transparency, control, context understanding, workflow fit, and downstream risk. However, they were not treated as fixed final categories. During coding, some components were merged, refined, or repositioned based on empirical patterns identified in the interview, think-aloud, and behavioural data. A full mapping of how the analytical framework informed the integrated empirical themes is provided in Annex 5.

For example, the interview theme "Engineers trust LLM outputs when they can check and verify them" was compared with the think-aloud theme "Engineers needed to check outputs before relying on them." These were integrated because both showed that participants did not rely on LLM outputs simply because they were generated. They needed to inspect, test, or judge the output before accepting it

The purpose was to compare reflective interview accounts with participants' reasoning during actual LLM use. Non-LLM verbal comments were not integrated into the final LLM trust themes; they were treated as contextual notes for interpreting behavioural findings.

Final integrated theme	Interview contribution	Think-aloud contribution from LLM-supported tasks	Integration logic
Trust depends on task risk and engineer confidence	Interviews showed that engineers trusted LLM support more when the task was familiar, low-risk, or within their own area of expertise.	Engineers used their own judgement to decide when AI support was useful.	Both datasets show that trust was conditional. Engineers did not trust the LLM in the same way for every task. They judged whether the LLM output was suitable based on task difficulty, risk, and their own ability to evaluate the result.
Trust requires easy verification	Interviews showed that engineers expected to check, test, or double-check LLM outputs before relying on them.	Engineers checked LLM outputs before using them.	Both datasets show that verification was the practical condition for trust. LLM outputs were not accepted simply because they were generated. Engineers needed to inspect or verify them before carrying them forward in the pipeline.
Trust depends on clear context and explanation	Interviews showed that prompt quality, task context, and explanation shaped whether LLM outputs were seen as useful or trustworthy.	Prompts helped engineers give context and check assumptions.	Both datasets show that trust depended on whether the LLM understood the task situation. Interviews showed context as an expectation, while think-aloud data showed how engineers actively used prompts to clarify context and guide the output.
Trust increases when engineers have control	Interviews showed that engineers preferred to guide, review, edit, and approve LLM support rather than let it work independently.	Engineers preferred step-by-step and editable LLM support.	Both datasets show that control supported safer use. Engineers were more willing to use LLM support when they could review intermediate steps, edit the result, and decide whether the output should continue in the workflow.

Trust decreases when LLM outputs change unexpectedly	Interviews showed that hidden changes, unstable outputs, and circular prompting reduced confidence in the LLM.	Unstable LLM outputs created extra work and raised concern about later effects.	Both datasets show that unpredictable LLM behaviour reduced trust because it increased checking, correction, and repair work. Engineers became cautious when they could not easily see what had changed or why.
Trust is limited by downstream risk	Interviews showed that engineers were concerned that weak LLM outputs could create later problems in the pipeline.	LLM outputs were treated as starting points, not final results.	Both datasets show that trust was shaped by what could happen after the immediate LLM interaction. LLM support was useful for producing an initial structure, but engineers still needed to review it because errors could affect later pipeline stages.

Table 6: Theme integration summary

3.4.4 Behavioural Coding of Screen Recordings

In addition to the thematic analysis of verbal data, the screen recordings from the task-based think-aloud sessions were analysed through behavioural coding. The recordings added a different type of evidence to the analysis by showing participants' workflow actions rather than only their spoken interpretations. The interviews and think-aloud verbal data helped explain how participants understood and judged LLM-based support, while the screen recordings showed what participants did as they worked through the SCAILAB pipeline. Behavioural coding was treated as the primary method for identifying where to add additional LLM support in the pipeline.

Since the study examined how engineers moved through a software-based pipeline, screen recordings provided a useful way to capture the sequence of interaction in context. They made it possible to return to the sessions after data collection and to review how participants moved between tasks, configured settings, checked outputs, and responded to workflow issues. This was useful because participant recall alone may not capture the details of interaction, especially in a multi-stage pipeline where small actions such as changing a parameter, validating a step, or returning to a previous configuration can be analytically relevant (Tang et al., 2006; Zhao et al., 2023).

The behavioural coding followed the same task structure used in the think-aloud sessions. Participant actions were organised across the fifteen workflow tasks, from project setup and requirements configuration through to data import, pipeline construction and generation, output verification, dataset preparation, evaluation, and monitoring. This gave the behavioural

analysis a consistent structure. It also meant that each action could be interpreted in relation to the stage of the pipeline where it occurred, rather than being treated as an isolated screen action.

The first stage of coding involved reviewing the recordings and identifying observable participant actions. These actions were coded only when they showed something the participant did within the pipeline. For example, opening the project, moving through the dashboard, or entering the project area was coded as NAVIGATES_PROJECT. Reviewing or entering project requirements was coded as CONFIGURES_REQUIREMENTS. Changing parameter values or adjusting node settings was coded as ADJUSTS_PARAMETERS. This made it possible to describe participant behaviour in a structured and comparable way.

The second stage involved linking each coded action to the relevant task area. For example, when a participant adjusted Random Number nodes and changed values in the pipeline workbench, the action was coded under T06 Pipeline parameter tuning with the behaviour code ADJUSTS_PARAMETERS. This action was not treated as a simple interface movement. It was interpreted as part of the participant's configuration work, where the engineer needed to decide whether the value was suitable and whether the pipeline could be trusted to continue from that point.

A similar approach was used for pipeline connection behaviour. When a participant linked pipeline components or configured relationships between nodes, the action was coded under T07 Pipeline linking and configuration using the behaviour code CONNECTS_OR_CONFIGURES_PIPELINE. In one case, the recording showed a connection issue where a participant connected a subject to itself. This was coded as pipeline-linking behaviour because the main action involved configuring relationships among pipeline elements. It also indicated a possible support opportunity around connection validation, clearer system feedback, and guidance when node relationships are incorrectly configured.

Validation and checking actions were coded separately because they were especially relevant to trust. When participants previewed the pipeline, checked whether an output looked correct, identified an error, or returned to fix a problem, the behaviour was coded as PREVIEWS_VALIDATES_AND_FIXES. For example, if a participant became uncertain during validation and returned to an earlier step to understand why the expected output did not appear, this was treated as evidence of a preview-and-fix loop. This type of behaviour is important in LLM-supported work because apparent automation benefits can shift part of the effort from producing an output to checking, validating, and correcting it (Mozannar et al., 2024; Tankelevitch et al., 2024).

Generation and output review were also kept separate in the coding. Starting or monitoring synthetic data generation was coded as RUNS_GENERATION. Inspecting generated data or reviewing whether the output was suitable was coded as CHECKS_OUTPUT_QUALITY. This distinction mattered because generating data and trusting generated data are not the same activity. A participant may complete the generation step, but still need to inspect the result before deciding whether it can be used. This aligns with the trust literature, in which appropriate

reliance depends on the user's ability to monitor and judge system output before relying on it (Lee and See, 2004; Hoff and Bashir, 2015).

The behavioural coding also included dataset and annotation-related actions. Preparing a training dataset, selecting images, or reviewing dataset splits was coded as CURATES_TRAINING_DATASET. Reviewing labels or annotations was coded as REVIEWS_ANNOTATIONS. For example, when participants considered whether an object should be labelled in a particular way, this was treated as annotation verification behaviour. This was relevant because synthetic data pipelines involve more than generation alone. They also include reviewing, curating, filtering, and preparing data for later use (Long et al., 2024).

Only participant actions were included in the behavioural coding. Teaching, coaching, host setup, facilitator instruction, and comments about another participant's action were excluded. This kept the analysis focused on participants' own workflow behaviour. For example, if a facilitator explained where to click, that moment was not coded as participant behaviour. If the participant independently opened a node, changed a parameter, reviewed an output, or corrected an issue, it was included as behavioural evidence. Making these inclusion and exclusion decisions explicit also supported the transparency and dependability of the analysis (Lincoln and Guba, 1985; Nowell et al., 2017).

The coding distinguished between detailed behavioural evidence and task-level completion. Some rows were based on direct screen recordings of actions, with clear evidence of what the participant did. These were treated as stronger behavioural evidence. Other rows were recorded as task-level completion when the task area was covered, but the recording did not provide a separate detailed non-teaching action segment. This distinction helped preserve coverage across the workflow without overstating the strength of every observation. Later-stage tasks such as evaluation, reporting, or monitoring were therefore interpreted more cautiously when they were supported mainly by task-level completion rather than by detailed evidence of action.

The final stage of behavioural coding involved identifying potential opportunities for workflow support. Repeated parameter adjustments suggested a need for clearer parameter explanations or support for recommended values. Connection errors suggested a possible need for node compatibility checks or clearer guidance on connections. Preview and fix behaviour suggested a need for error explanation, repair suggestions, or validation support. Output quality checks suggested the need for verification checklists, anomaly detection, or clearer comparisons between generated outputs.

Behavioural coding was not derived from the trust-related framework components and was not used to code trust directly. It was not used to create the final integrated verbal themes. Instead, it was used as a separate workflow-oriented layer to document observable actions and identify workflow friction or decision-making effort in the pipeline. This separation was important because the verbal themes explained how engineers understood trust, while Behavioural coding showed where workflow friction and decision-making effort appeared in practice. Together,

these two layers helped connect participants' trust concerns to specific parts of the SCAILAB pipeline and supported the later development of design implications.

These behaviours were treated as indicators of workflow friction or decision-making effort. They were not treated as direct evidence that participants wanted LLM support at those points. Instead, they provided a basis for identifying researcher-identified opportunities for future LLM support.

3.4.5 From Analysis to Design Implications

The final stage of the analysis connected the empirical findings to design implications. The integrated verbal themes identified the key trust-related conditions shaping engineers' experience of LLM-based support. These included verification, context, explanation, control, predictability, task risk, and downstream consequences. The behavioural coding then helped locate where these issues appeared in the pipeline workflow.

For example, when the verbal analysis showed that verification was important, the behavioural coding helped identify where verification appeared in practice. This included preview-and-fix loops, output-quality checks, annotation reviews, and evaluation-related tasks. Similarly, when the verbal analysis showed that engineers wanted more control, the behavioural coding helped identify workflow moments where control was already important, such as parameter adjustment, pipeline configuration, node linking, and correction work.

This connection between what participants said and did informed the development of design implications. The analysis suggested that trustworthy LLM support should not only generate outputs. It should also help engineers understand outputs, check them efficiently, control how changes are applied, and recognise when weak outputs may affect later stages of the pipeline.

These design implications informed the later design recommendations. In this way, the analysis moved from participant evidence to themes, from themes to workflow interpretation, and from workflow interpretation to design direction. This helped ensure that the design recommendations were grounded in empirical evidence rather than developed in isolation from it.

3.5 Ethical Considerations

This study considered ethical considerations a key priority, as it involved an organisational setting and addressed work practices involving commercially sensitive tools, processes, and internal knowledge. Accordingly, careful attention is given to both participants' rights and the secure management of company-related information.

Before data collection began, participants were informed about the purpose of the study, the nature of their participation, and how the collected material would be used. Participation was voluntary, and participants were informed that they could withdraw their responses at any time until the thesis was published. This was especially important in an organisational context,

where employees may otherwise feel implicit pressure to participate or to present the system in a particular way.

Confidentiality and anonymity were maintained throughout the study. Participants were not identified by their real names in transcripts or analyses; instead, they were referred to by neutral identifiers.

The study also considered organisational sensitivity. Since the thesis focuses on an internal synthetic data pipeline and on engineers' experiences with company-specific tools, the analysis avoids disclosing unnecessary proprietary details.

3.6 Research Quality and Trustworthiness

The quality of the study is addressed through the qualitative criteria of credibility, transferability, dependability, and confirmability. These criteria are used because the thesis uses qualitative methods and seeks to produce contextually grounded understanding rather than statistical generalisation (Lincoln and Guba, 1985).

In this study, credibility is strengthened by using multiple methods. Semi-structured interviews provide reflective accounts of how engineers experience LLM-based support. In contrast, task-based think-aloud sessions and screen recordings provide access to real-time trust-related reasoning and observable interaction behaviour during task performance. Together, these methods allow different forms of evidence to be compared and interpreted in relation to one another, which strengthens the credibility of the study (Denzin, 1978; Lincoln and Guba, 1985).

Transferability concerns the extent to which readers can assess the relevance of the findings for other contexts. This study does not seek statistical generalisation. Instead, it aims to provide a sufficiently detailed description of the empirical setting, the workflow context, the user group, and the role of LLM-based support so that readers can assess whether the findings are relevant to similar engineering or AI-supported workflow environments (Lincoln and Guba, 1985).

Dependability concerns the transparency and consistency of the research process. In this thesis, dependability is supported by a clear description of the research design, case boundaries, data collection methods, interview guide logic, sampling strategy, and stages of analysis. The use of an interview guide also contributes to consistency across interviews by ensuring that all participants were asked about the same trust-related dimensions while still allowing flexibility in discussion (Lincoln and Guba, 1985; Nowell et al., 2017).

Confirmability concerns whether the findings are grounded in the empirical material instead of the researcher's personal preferences or assumptions. This is supported by keeping the analysis closely tied to participants' accounts, think-aloud material, and screen-recorded interaction data, and by combining deductive and inductive coding. The theoretical framework influenced the analysis, but it did not determine the findings in advance. As a result, the study remained open to patterns emerging from the empirical material while remaining analytically connected to the literature on trust, calibration, and workflow fit (Lincoln and Guba, 1985).

3.7 Methodological Reflections

A central methodological decision in this study was to combine semi-structured interviews, task-based think-aloud sessions, and behavioural coding of screen recordings rather than relying on a single data source. This combination was used because each method captured a different aspect of engineers' trust in LLM-based support. The interviews offered reflective insights into how participants gradually understood, assessed, and described the support provided by LLMs over time. However, interviews alone could not show how engineers interacted with the support during task performance. The think-aloud sessions captured participants' reasoning while they worked through pipeline tasks, but these data were limited to what participants chose or were able to verbalise during the session. Behavioural coding of screen recordings complemented these methods by capturing observable workflow actions, such as navigation, parameter adjustment, validation behaviour, and correction loops. At the same time, behavioural data could not explain why a participant took a particular action. Combining the three methods therefore helped build a more complete understanding of trust, verification effort, and workflow fit than any single method could provide.

A further methodological decision concerned the structure of the think-aloud sessions. The sessions included both LLM-supported and non-LLM pipeline tasks. This created a possible risk of confusing direct evidence about trust in existing LLM support with broader workflow difficulty. To address this, the thematic analysis of think-aloud verbal data focused only on the two tasks that involved existing LLM-supported functions. Verbal comments from non-LLM tasks were retained as contextual notes rather than treated as primary evidence of trust in LLM support. In contrast, the screen recordings from the full workflow were analysed separately through behavioural coding to identify workflow areas where additional LLM support could be explored.

It is also important to reflect on the interpretive role of behavioural coding. Identifying observable actions in a screen recording is not the same as fully understanding what those actions mean. For example, a participant returning to an earlier configuration step may indicate uncertainty, verification effort, task unfamiliarity, or a need to correct a previous decision. For this reason, the behavioural findings in Chapter 4 are framed as researcher-identified patterns rather than direct participant-reported needs. The behavioural coding was used to identify points of workflow friction and decision-making effort, not to claim that participants explicitly requested LLM support at those points.

Chapter 4 – Empirical Findings

4.1 Introduction

This chapter presents the empirical findings from the semi-structured interviews, task-based think-aloud verbal data, and behavioural coding of the screen-recorded task sessions.

The findings are presented in two main parts. The first part presents the integrated thematic findings from the interviews and the verbal data collected during the two LLM-supported think-aloud tasks. These findings explain how engineers experienced LLM-based support and how they judged whether the support could be trusted. The second part presents behavioural findings from the screen recordings across the full 15-task workflow. These findings show where participants configured, checked, corrected, validated, and reviewed work inside the SCAILAB pipeline, and were used to by the researcher to identify where additional LLM support could be explored.

The final integrated themes are:

1. Trust Depends on Task Risk and Engineer Confidence
2. Trust Requires Easy Verification
3. Trust Depends on Clear Context and Explanation
4. Trust Increases When Engineers Have Control
5. Trust Decreases When LLM Outputs Change Unexpectedly
6. Trust Is Limited by Downstream Risk

Together, these themes show that engineers' trust in LLM-based support was conditional. Participants did not describe the LLM as simply trustworthy or untrustworthy. Their trust changed depending on the task, the ease of checking the output, the clarity of context, the level of control, the stability of system behaviour, and the possible consequences of carrying weak outputs forward in the pipeline.

4.2 Integrated Thematic Findings

4.2.1 Trust Depends on Task Risk and Engineer Confidence

The first theme shows that engineers did not trust LLM support equally across all tasks. Trust varied with task complexity, the potential consequences of error, and the engineer's confidence in judging the output.

This pattern suggests that trust was partly prospective rather than only reactive. Engineers were not simply judging whether a particular LLM output appeared correct. They were also assessing whether they could identify and manage potential errors if the output turned out to be incorrect. In this sense, trust was shaped not only by the perceived behaviour of the LLM, but also by the engineer's confidence in their own ability to evaluate the output.

In the interview data, this appeared in comments such as *"For me, depends on the complexity of the task. If it's a small fix in specific, very specific code files or so on, I'm going to do it myself by hand"* (P02, interview). This showed that participants did not evaluate LLM support

in a general way. A simple or familiar task made the support easier to accept. A complex task required more caution. Participants also connected trust to sensitive or high-risk work, for example when one participant referred to *"I mean, cybersecurity concerns, saying that before we launch things, we always have to check that there are now API keys somewhere exposed, and other kind of vulnerabilities which are easy to fix." (P02, interview)*. This showed that the same LLM behaviour could feel acceptable in one context but less acceptable in another.

There was also variation across participants. Engineers with more extensive experience with LLMs appeared to describe trust in a more differentiated way. For example, they were more explicit about when they would use the support, when they would avoid it, and which kinds of tasks required stronger verification. This suggests that repeated exposure did not simply increase trust in general. Instead, it seemed to support more calibrated trust, in which engineers developed a clearer sense of the types of tasks for which the support was useful and those where caution was needed.

During the LLM-supported think-aloud tasks, participants exhibited the same pattern when interacting directly with the LLM features. One participant stated, *"For asset generation, it helps, but only when the task is not too complex*" (P02, think-aloud session). The comment shows that the participant saw value in the LLM support, but only within certain boundaries. The support was not rejected, but it was also not treated as suitable for every type of task.

Another participant said*, "The global idea is useful, but implementation still needs review"* (P02, think-aloud session). This statement clearly distinguishes between early support and practical use. The LLM could help with a general idea or first structure, but the engineer still needed to judge whether the output would work in the actual pipeline.

This theme therefore shows that trust was not primarily a general judgement about the LLM as a system. It was a situated judgement about whether the engineer, in a specific task, retained enough understanding and control to use the system's output safely.

4.2.2 Trust Requires Easy Verification

The second theme was among the strongest findings in the data. Participants repeatedly described the need to check, inspect, test, or validate LLM outputs before using them. This pattern shows that verification was not an optional activity added after LLM use. Rather, it functioned as a condition for using the support in the first place. Participants did not treat checking as evidence that the LLM had failed. Instead, they treated verification as a normal and necessary part of the pipeline for working with LLM-generated output.

In the interviews, participants used direct language around verification. One participant explained that there was *"It depends.there's always verification effort involved"* (P03, interview). Another stated that users needed to *"Try as much as possible and double check everything"* (P02, interview). These comments show slightly different orientations toward verification. P03's account suggests an acceptance that LLM support always involves some degree of checking, while P02's account reflects a more active form of vigilance. What

connects both accounts is that trust and verification were not separate processes. Participants could only rely on the support when they were able to inspect and confirm its output.

The think-aloud verbal data showed verification as part of task performance. For example, one participant said, *"It is close, but I need to check these parameters before use it"* (P01, think-aloud session). The participant considered the output close enough to be useful but not ready for use without inspection. Trust depended on whether the engineer could check the parameter values before carrying the output forward.

Another participant said, *"I would use this first to get the structure, but I still need to inspect what it created"* (P01, think-aloud session). This statement clearly captures the central finding. The LLM helped with structure, but trust depended on inspection. The output had value as a first step, not as a final result.

4.2.3 Trust Depends on Clear Context and Explanation

The third theme shows that engineers trusted LLM support more when the task context was clear and when the system provided sufficient explanation of its outputs. Participants were less confident when prompts were overly open, assumptions were unclear, or outputs did not clearly fit the pipeline.

In the think-aloud data, participants connected output quality to the clarity of the task definition. One participant said, *"Without a structured way of defining the module, different prompts give different results"* (P03, think-aloud session), referring to the difficulty of obtaining reliable outputs when the task is not clearly described. The comment indicated that participants did not view LLM output quality as independent of how the task was framed.

Participants also wanted the system to explain its reasoning. One participant said, *"Okay, explain me why line line by line or like what are you doing […] because I would like […] to understand […] what is the core and why the this system works like that" (P01, interview).* This showed that explanation was part of how engineers judged whether an output made sense. They wanted to understand why the system made a suggestion before accepting or adapting it.

The think-aloud verbal data showed the same issue during task performance. One participant stated, *"Explain why this connection is needed; otherwise, I am just accepting it blindly"* (P01, think-aloud session). The participant wanted to understand the reason behind a suggested connection before accepting it. The concern was not only whether the connection existed. The participant needed to know why it was appropriate.

Another participant said, *"It depends on the context; if the prompt is too open, the output is not reliable"* (P03, think-aloud session). This showed that participants directly linked trust to the quality of the context provided to the LLM. Open prompts created uncertainty because the output could take directions that did not align with the pipeline task.

This theme shows that engineers were not satisfied with outputs that only appeared plausible. They wanted outputs that were clearly connected to the task, the pipeline state, and the available requirements.

4.2.4 Trust Increases When Engineers Have Control

The fourth theme shows that engineers trusted LLM support more when they could guide, review, edit, approve, or reject outputs. Participants preferred assistance to autonomous action.

In the interviews, this appeared in comments such as *"In the development, and our own development, uh, we are using different modes. […] I have to review them myself before I've merged them into our platform code"* (P02, interview). This showed that participants wanted to remain responsible for approving outputs before they entered the platform or workflow. Another participant said that "it gives you intermediate steps where you can verify what is going on. So you can make changes" (P02, interview). This suggested that current LLM support still required human steering and correction.

The think-aloud verbal data showed a similar preference for staged interaction. One participant said, *"Step by step is better; I want to see what it plans before it generates the whole module"* (P01, think-aloud session). This comment shows a clear preference for review before a larger system action. The participant did not reject module generation. Instead, they wanted the system to expose its plan before producing a full output.

Control also appeared in concerns about the scope of changes. One participant stated, "I don't want it to change other parts of the setup just to make this one work" (P01, think-aloud session). This showed that participants wanted the LLM to remain within the intended action space. A useful suggestion could still undermine trust if it unexpectedly affected other parts of the setup.

The behavioural data supported this theme through actions such as parameter adjustments, pipeline configuration, node linking, corrections, and reviews. Participants were not passive users of the system. They actively shaped the workflow, checked outputs, and made decisions before continuing.

This theme shows that trust increased when engineers remained in control of the interaction. The LLM was more acceptable when it acted as a support tool that could be steered, reviewed, and corrected.

4.2.5 Trust Decreases When LLM Outputs Change Unexpectedly

The fifth theme shows that unpredictable or unstable LLM behaviour reduced trust. Participants became less confident when outputs changed unexpectedly, when repeated prompts created circular fixes, or when the system affected unintended parts of the output.

In the interview data, one participant described the interaction as *"But you're then saying, you're basically going in circles around the things that you're not happy with."* (P02, interview). This was linked to repeated attempts to correct or improve the output without

reaching a stable result. The issue was not only that the output was wrong. The repeated interaction itself created uncertainty.

Another participant described the system changing unexpectedly as part of the output. *"It does this what it tells specifically, but then suddenly changes something on a completely different and which messes up something else"* (P02, interview). This showed concern about hidden or unintended changes. When the system shifted focus to areas outside the user's attention, participants had to check more carefully. This increased effort and reduced confidence.

The think-aloud verbal data also showed concern about unexpected changes. The comment *"It changed something I did not ask for, so now I have to inspect the surrounding setup" (P02, think-aloud session)* showed that participants wanted the system to keep changes within a predictable scope. The participant's concern was about control as well as stability. If the system solved one issue by changing another part of the setup, the engineer could not easily know whether the workflow remained safe.

This theme shows that trust was affected by stability across interaction. Participants needed to understand not only what the LLM produced but also how its outputs changed across iterations. Unexpected changes made the system harder to monitor and increased the amount of checking needed before use.

4.2.6 Trust Is Limited by Downstream Risk

The sixth theme shows that engineers valued LLM support for early framing and ideation, but became cautious when outputs could affect later pipeline stages.

In the interview data, participants described value in using LLMs for early work. For example, comments about *"I would say first of all, its great to have the user breaking down the problem. […] You describe the kind of data you want and LLM help you to divide your problem in sequential steps. Breaking down the problem is the first thing that I would suggest."* (P01, interview) showed that participants saw the LLM as useful for structuring or starting a task. However, this value was limited when weak outputs could create later work. One participant referred to outputs that could *"I mean, it's basically, you produce some trash, which you would need to remove. […] It's not just an unnecessary amount of data or too little data, it's basically user experience, which gets lost, and also maybe some angry users. […] This is their costs for the users as well"* (P02, interview). Another raised concern that poor outputs could mean *"If an LLM error affects datasets, labels, or training datasets, then the data becomes wrong and this compromises everything later in the pipeline, including the training part"* (P04, interview) howed that weak outputs could create additional work or affect later stages of the pipeline.

The think-aloud verbal data showed the same concern during task performance. One participant said, *"If this wrong module goes forward, it could affect the other connections"* (P01, think-aloud session). Another stated, *"Small errors here can move into later stages, so I would rather verify early"* (P04, think-aloud session). These comments show that participants were thinking

beyond the immediate output. They were considering what could happen if the output was carried forward.

The behavioural data also supported this theme. Participants moved through tasks that connected early setup and configuration to later stages such as dataset preparation, annotation review, and evaluation. This showed that the pipeline was experienced as a connected workflow. A weak decision in one place could create work later.

This theme shows why trust in the SCAILAB pipeline cannot be judged only at the moment of generation. Engineers evaluated LLM support in relation to later consequences. The LLM could be useful early on, but reliance became limited when its outputs could affect downstream work.

4.3 Behavioural Findings from Screen-Recorded Task Interaction

This section presents behavioural findings from the screen-recorded task interaction. The analysis focused on observable workflow activities, including requirement configuration, parameter adjustment, output inspection, validation, correction, and review before continuing (refer to Annex 4). These activities were used to identify workflow points where participants incurred configuration, verification, or decision-making effort. Based on this behavioural evidence, the researcher identified potential opportunities to explore future LLM support. These opportunities should not be consider as direct requests from participants for LLM support at each stage.

4.3.1 Behaviour during the Requirement and Project configuration

The first workflow phase involved project setup, dashboard orientation, and requirements configuration. Participants began with the same initial project state and moved into the pipeline environment before proceeding to later tasks.

Behavioural evidence in this phase included navigation, reviewing the project state, and configuring or considering requirements. These actions showed that early project information shaped the later workflow. Requirement configuration was not only a setup step. It influenced how later outputs would be generated, checked, and judged.

The findings from this phase showed a need for support around early clarification. If requirements are unclear or incomplete, later outputs may become harder to verify. This phase therefore appeared as an early point at which trust-related uncertainty could arise.

4.3.2 Behaviour during Pipeline Construction and Parameter Tuning

The second workflow phase involved configuring nodes, adjusting parameters, and connecting pipeline components. This phase provided some of the clearest behavioural evidence because participants had to make decisions that affected the pipeline's structure and behaviour.

Parameter tuning appeared as an important action. When participants adjusted values or modified settings, they were not only interacting with the interface. They were making decisions about how the pipeline should behave. These actions were directly connected to trust because participants needed confidence that the selected values were appropriate for the pipeline creation.

For example, the behavioural coding captured actions where participants adjusted node settings and modified parameter values during the pipeline workbench stage. These actions were coded as parameter tuning because the participant was actively shaping the pipeline's behaviour. The action was therefore more than navigation; it involved judgement about the suitability of the configuration.

Pipeline linking also appeared as an important behavioural area. In one case, a connection issue occurred when a participant connected a subject to itself. This showed that pipeline construction can create errors that may not be obvious without validation or clearer feedback. It also revealed a need for support around connection validation, connection rules, and clearer system feedback during pipeline construction.

Workflow friction and decision-making effort appeared where engineers configured the pipeline, adjusted values, or connected components. These were points where small mistakes could affect later output.

4.3.3 Behaviour during the Validation, Preview, and Fixing

The third workflow phase involved previewing, validating, identifying issues, and returning to fix problems. This phase was strongly connected to the theme that trust requires easy verification.

Participants used validation and preview behaviour to check whether the pipeline was working as expected. When something did not appear correctly, they returned to earlier steps or tried to understand what had gone wrong. These behaviours showed that validation was not only a final check. It was part of how participants moved through the workflow.

Preview and fix loops also showed a need for support around error understanding and repair. The behavioural evidence indicated that participants spent effort identifying what had gone wrong and deciding how to continue. This made validation a clear trust-related point in the pipeline. For example, when a participant became uncertain during validation and returned to earlier configuration steps, the action showed that trust depended on making sense of the pipeline state before moving forward.

This behavioural phase showed that workflow friction and decision-making effort were concentrated around checking, explanation, and repair, not only around generation.

4.3.4 Behaviour during Generation and Output Quality Checking

The fourth workflow phase involved running or preparing synthetic data generation and then reviewing generated outputs. Behavioural coding separated generation from output checking because completing generation did not mean trusting the result.

Participants inspected generated outputs before continuing. This showed that output quality assessment was a key trust-related action. Even when generation was successful, participants still needed to decide whether the result was usable.

This phase supported the finding that verification is central to trust. Generated outputs needed to be inspected, compared, and judged. Participants did not treat generation as the end of the task. They treated it as a point requiring further checking.

The behavioural evidence in this phase indicated a need for support in output comparison, anomaly recognition, and quality review. These were visible in participants' checking behaviour rather than as abstract design ideas.

4.3.5 Behaviour during Dataset, Annotation, Evaluation, and Monitoring

The later workflow phase involved dataset preparation, annotation review, model-related configuration, evaluation, reporting, and monitoring. The behavioural evidence for some of these later tasks was less detailed than the evidence for configuration, validation, and output checking. These later-stage findings should therefore be interpreted more cautiously.

Even so, these stages remained relevant because they showed how early pipeline outputs could move toward downstream use. Dataset curation, annotation verification, and evaluation-related actions connected directly to the theme of downstream risk. Participants needed to ensure that outputs were not only generated, but also suitable for later use.

Annotation-related behaviour showed that reviewing labels and prepared data could become a trust-relevant activity. If labels or dataset structure are weak, later model training or evaluation may be affected. The later workflow stages therefore reinforced the finding that trust in LLM support depends on what happens after generation, not only on the immediate output.

4.4 Convergence Between Thematic and Behavioural Findings

The thematic and behavioural findings supported each other in several ways. The verbal data explained how participants understood trust, while the screen recordings showed where trust-related actions appeared in the pipeline.

The theme "Trust Requires Easy Verification" was evident in behavioural actions such as previewing, validating, fixing, checking output quality, and reviewing annotations. Participants did not only say that outputs needed checking. They also acted in ways that showed checking was built into the workflow.

The theme “Trust Depends on Clear Context and Explanation” was connected to requirement setup, parameter tuning, node configuration, and pipeline linking. These tasks required participants to understand what the system was doing and how each action affected the wider pipeline. When context was unclear, outputs became harder to judge.

The theme “Trust Increases When Engineers Have Control” was connected to behaviours such as adjusting parameters, changing settings, reviewing outputs, and correcting issues. Participants remained actively involved in shaping the workflow. They did not simply accept system outputs.

The theme Trust Is Limited by Downstream Risk connected to dataset preparation, annotation review, and evaluation-related tasks. These stages showed that outputs could not be evaluated only at the moment they were generated. They had to be considered in relation to later use.

Integrated theme	Behavioural evidence	Empirical interpretation
Trust Depends on Task Risk and Engineer Confidence	Participants moved through tasks with different levels of complexity, from setup to configuration, generation, and review.	Trust changed according to the task and the engineer’s ability to judge the output.
Trust Requires Easy Verification	Previewing, validating, fixing, output checking, annotation review.	Verification appeared as both a spoken concern and an observable workflow action.
Trust Depends on Clear Context and Explanation	Requirement setup, parameter tuning, node configuration, connection behaviour.	Outputs were easier to trust when they could be understood in relation to the task context.
Trust Increases When Engineers Have Control	Parameter adjustment, pipeline configuration, review, correction.	Participants stayed involved and preferred to guide or approve changes.
Trust Decreases When LLM Outputs Change Unexpectedly	Validation actions, review after changes, concern about unexpected scope changes.	Unexpected changes increased the need for checking and reduced confidence.
Trust Is Limited by Downstream Risk	Dataset preparation, annotation review, evaluation-related tasks.	Participants considered whether outputs could safely move into later stages.

Table 7: Convergence of behavioural and thematic data

4.5 Chapter Summary

This chapter presented the empirical findings from the interviews, think-aloud verbal data, and screen-recorded behavioural coding. The integrated thematic findings showed that engineers' trust in LLM-based support depended on task risk, verification, context, explanation, control, predictability, and downstream consequences.

The behavioural findings indicated where these trust-related issues occurred within the SCAILAB pipeline. Participants configured requirements, adjusted parameters, linked pipeline components, validated outputs, checked generated results, reviewed annotations, and moved through later workflow stages. These actions showed that trust was connected to practical workflow activity.

Overall, the findings show that LLM support in the SCAILAB pipeline was perceived as useful but not inherently trustworthy. Engineers valued support that helped them begin and structure work, but they still needed to inspect, understand, control, and verify outputs before relying on them. The next chapter discusses these findings in relation to the research questions and the wider literature on trust calibration, LLM-supported work, and human-centred AI design.

Chapter 5 – Discussion

5.1 Introduction

This chapter discusses the empirical findings in relation to the research questions, the literature review, and the SCAILAB synthetic data pipeline context. Chapter 4 presented the findings from the semi-structured interviews, task-based think-aloud verbal data, and behavioural coding of screen-recorded task interaction. The purpose of this chapter is to explain what those findings mean for trust calibration, workflow-based LLM support, and the design of more trustworthy AI assistance in a multi-stage synthetic data pipeline.

The discussion follows the four research questions. The first section discusses how current LLM support shapes engineers' trust. The second section discusses researcher-identified workflow points where additional LLM support could be added or explored in the pipeline. The third section examines the types of mismatches that reduce trust. The fourth section translates the findings into design recommendations for more trustworthy LLM support.

The central argument is that trust in LLM-based support within the SCAILAB pipeline is conditional and workflow-dependent. Engineers did not simply trust or distrust the LLM. Their trust changed with task risk, verification effort, task context, perceived control, output predictability, and the potential downstream effects of weak outputs. This supports the view that trust in automation should be calibrated to the system's actual capabilities and limits, rather than treated as a general attitude toward the tool (Lee and See, 2004; Hoff and Bashir, 2015).

5.2 RQ1: Current LLM Support and Engineers' Trust

The findings show that LLM support currently appears mainly in two parts of the SCAILAB pipeline: the Procedural Editor and the Generate New Module function. Participants did not experience these functions as replacements for engineering judgement. They treated them as forms of assistance for early structure, procedural support, and module generation, but only when outputs could be checked, interpreted, and controlled.

5.2.1 Current LLM Support Is Useful but Conditional

The current LLM-supported functions were useful because they helped participants begin a task, generate an initial structure, or reduce some manual effort. This aligns with studies showing that LLMs can support early drafting, problem exploration, and task acceleration in professional and engineering work (Vaithilingam et al., 2022; Barke et al., 2023; Hou et al., 2024).

However, usefulness did not automatically lead to trust. Participants treated LLM outputs as starting points rather than as final answers. They judged the support not only by whether it produced something plausible, but by whether the output could be inspected, adapted, and safely carried forward. Recent studies on LLM-supported work make a similar point: apparent productivity gains can be reduced by the effort needed to interpret, verify, and repair generated outputs (Mozannar et al., 2024; Tankelevitch et al., 2024).

In the SCAILAB context, the conditional nature of usefulness is important because the pipeline is not a single-step task. A generated module, a parameter setting, or a procedural asset can influence later stages. The value of LLM support, therefore, depends on whether the output fits the wider workflow, not only on whether it looks useful at the point of generation.

5.2.2 Trust Depends on Task Risk and Engineer Confidence

Engineers' trust varied with task risk and their confidence in judging the output. Participants were more willing to use LLM support when the task was familiar, low-risk, or easy to inspect. They became more cautious when the task was complex, when the consequences of error were higher, or when they were less confident in their ability to evaluate the result.

This finding reflects the idea of trust calibration. Lee and See (2004) argue that appropriate reliance depends on whether trust matches the system's capabilities. Hoff and Bashir (2015) also show that trust is shaped by situational and learned factors. In this study, task familiarity, risk, and engineer confidence worked as important situational factors. Participants were not asking whether the LLM was generally trustworthy. They were assessing whether it was sufficiently trustworthy for a specific task at a specific point in the pipeline.

For SCAILAB, this suggests that trust cannot be designed as a single general feature. Engineers may trust the same LLM more in one task and less in another. They may accept LLM support for early problem framing, but become more careful when the output affects pipeline connections, dataset quality, or later evaluation. Trust therefore needs to be supported differently at different stages of the workflow.

5.2.3 Trust Requires Verification Before Use

Verification was one of the strongest trust conditions in the findings. Participants repeatedly indicated that LLM outputs needed to be checked before use. This appeared in the interviews, the think-aloud verbal data, and the behavioural coding. Participants reviewed the outputs, adjusted parameters, validated the steps, and checked the generated results before continuing.

This finding aligns with the trust-in-automation literature. Lee and See (2004) argue that appropriate reliance depends on the user's ability to monitor automation. In LLM-supported work, monitoring is particularly important because generated outputs can appear fluent and plausible even when incomplete, misaligned, or incorrect. Studies of LLM-supported work also show that these systems can shift effort from producing an initial output to checking and correcting it (Vaithilingam et al., 2022; Mozannar et al., 2024; Tankelevitch et al., 2024).

In the SCAILAB pipeline, verification is not only an individual behaviour. It is part of the work process. Engineers need to check whether outputs fit the task, whether parameters are suitable, whether generated data is usable, and whether outputs can be carried into later stages. Trust in current LLM support therefore depends strongly on whether verification is easy, visible, and supported by the system.

5.3 RQ2: Where Additional LLM Support Could Be Added

RQ2 is addressed as an exploratory identification of potential support opportunities. These opportunities were derived mainly from behavioural coding of screen-recorded workflow activity and interpreted in relation to the trust themes identified in the interviews and LLM-supported think-aloud tasks. Since participants were not systematically asked to confirm support needs for every non-LLM task, the findings should be understood as researcher-identified opportunities rather than direct evidence of participant demand.

5.3.1 Early Requirement and Project Configuration Support

One potential area for additional support is the early stage of project setup and requirement configuration. Early requirements shape the rest of the pipeline. If requirements are unclear, incomplete, or weakly structured, later stages may become harder to configure and verify.

LLM support could help engineers clarify missing information, summarise requirements, identify assumptions, or check whether the project setup contains enough information for later stages. This connects to the literature on context and workflow fit, where trust depends on whether system support aligns with the real conditions and constraints of the work (Baxter and Sommerville, 2011; Parasuraman et al., 2000).

At this stage, the LLM should not independently define requirements. Instead, it should assist with clarifying and organizing information. For instance, the system could ask whether the object types, output goals, annotation needs, and evaluation criteria are clearly specified before the engineer moves forward.

5.3.2 Pipeline Construction and Parameter Support

A second area for additional support is pipeline construction, node configuration, parameter tuning, and pipeline linking. Behavioural coding showed that participants engaged in configuration work that required judgement. Parameter changes and node connections are not simple interface actions. They affect how the pipeline behaves and what kind of output is produced.

LLM support could help by explaining parameter meanings, suggesting reasonable ranges, warning about incompatible settings, or identifying possible connection problems. For example, if a node is connected incorrectly, the system could explain why the relationship may not work and suggest what the engineer should inspect. This type of support would address both verification and contextual understanding.

Human-centred AI literature supports this direction. Shneiderman (2020) argues that trustworthy AI should preserve meaningful human control. In the SCAILAB pipeline, useful support would not automatically change the configuration. It would explain, suggest, and allow the engineer to approve or reject changes.

5.3.3 Pipeline Validation, Preview, and Output Quality Support

The strongest opportunities for additional LLM support lie in pipeline validation, preview, and output quality checking. Behavioural coding showed that participants spent time checking

outputs, fixing issues, and assessing whether the results were suitable. This closely aligns with the broader LLM literature, in which generated outputs often require review, correction, and adaptation before use (Mozannar et al., 2024; Tankelevitch et al., 2024).

In this stage, LLM support could explain validation errors, suggest possible fixes, generate verification checklists, compare outputs, or highlight unusual results. Such support would make verification easier and reduce the burden placed on engineers. It would also support trust calibration by providing engineers with better evidence to decide whether to rely on the output.

The system should also make clear what it is suggesting and why. If a fix is suggested without explanation, the support may create another trust problem. Engineers would still need to inspect whether the fix creates hidden changes or affects later stages.

5.3.4 Dataset, Annotation, and Evaluation Support

Additional support may also be useful in later stages such as dataset preparation, annotation review, evaluation, reporting, and monitoring. Synthetic data does not become valuable simply because it is generated. It must also be curated, reviewed, evaluated, and prepared for downstream use (Long et al., 2024; Nadas et al., 2025).

LLM support could help identify annotation inconsistencies, summarise dataset quality issues, interpret evaluations, or generate reports explaining what has been checked. These later-stage opportunities should be discussed carefully, especially where behavioural evidence was less detailed. They should be presented as opportunities for support rather than as fully proven solutions.

The broader implication is that LLM support should not be limited to generation points. It can also support preparation, review, validation, and interpretation. Early-stage support should help with structuring and clarification. Middle-stage support should help with configuration and validation. Later-stage support should help with downstream review, interpretation, and risk reduction.

Pipeline stage	Related workflow task (s)	Observed support need	Possible LLM support	Trust condition required
Requirement/project configuration	T02 – Requirement entry	Clarifying task goals and project constraints	Requirement clarification, checklist prompts, missing-information prompts	Context and explanation
Pipeline construction	T05 -Pipeline node configuration/construction; T07 -Pipeline linking and configuration	Deciding how nodes/components should be structured	Node suggestions, connection explanations, structure previews	Engineer control
Parameter tuning	T06 - Pipeline parameter tuning	Judging whether settings are suitable	Parameter explanations, safe range suggestions, warnings	Easy verification
Pipeline validation/preview/fix loop	T08 - Pipeline validation and preview/fix loop	Understanding errors and deciding corrections	Error explanation, suggested fixes, before/after comparison	Verification and control
Generation/output checking	T09 - Synthetic data generation; T10 - Output quality assessment and verification	Inspecting whether outputs are usable	Output quality checklist, anomaly detection,	Downstream safeguards

			review prompts	
Dataset/annotation/evaluation	T11 -Training dataset curation and split; T12 Annotation verification and dataset preparation; T14 - Evaluation and downstream testing/reporting; partly T15 - Documentation, deployment, and monitoring	Checking consistency before downstream use	Annotation review support, dataset consistency checks, evaluation guidance	Downstream risk awareness

Table 8: Pipeline stages and possible LLM support opportunities

5.4 RQ3: Mismatches That Reduce Trust

The findings show that reduced trust was not caused only by obviously wrong outputs. Trust also decreased when outputs were difficult to verify, poorly contextualised, hard to control, unpredictable, or risky to carry forward. This supports the view that mismatches in LLM-supported work are a coordination problem, not only an output-quality problem (Zamfirescu-Pereira et al., 2023; Tankelevitch et al., 2024).

5.4.1 Context and Prompt Mismatch

One important mismatch was between the engineer's task context and the LLM's understanding of that context. Participants became more cautious when prompts were overly open, when the system lacked sufficient workflow information, or when outputs appeared generic rather than pipeline-specific.

This finding connects to the literature on workflow fit. Baxter and Sommerville (2011) argue that technology must be understood in relation to the wider work system. In this study, an LLM output was not useful simply because it was coherent. It needed to fit the current pipeline task, the configuration state, and the engineer's goal.

Prompt-based systems can be difficult to control when users must translate complex task needs into short instructions (Zamfirescu-Pereira et al., 2023). In SCAILAB, a weak prompt or missing context could result in output that appeared useful but did not match the actual workflow.

5.4.2 Verification Mismatch

A second mismatch occurred when outputs were difficult to verify. Participants were not only concerned with whether the output was correct. They were concerned with whether they could tell that it was correct.

Verification mismatch occurs when the LLM produces something that appears useful but leaves the engineer with too much work to check. In such situations, the support may not reduce effort. It may move effort from production to inspection. This aligns with findings from LLM-supported work, in which users spend time reading, validating, and repairing generated outputs (Vaithilingam et al., 2022; Mozannar et al., 2024).

In a pipeline context, verification mismatches become more serious because errors may not be immediately visible. A parameter setting, a generated module, or a dataset issue may only cause problems later. Trust decreases when verification is difficult, delayed, or unsupported.

5.4.3 Control Mismatch

A third mismatch concerned control. Participants wanted to guide, edit, approve, or reject LLM outputs. Trust decreased when the system appeared to act too broadly, change unexpected parts of the setup, or lack sufficient intermediate checkpoints for review.

This finding aligns with human-centred AI literature. Shneiderman (2020) argues that trustworthy AI should support human control rather than remove it. Dietvorst et al. (2018) also show that users are more willing to use imperfect algorithms when they retain some ability to intervene. In this study, control was not just a preference. It was part of what made LLM support acceptable.

Control mismatch is especially relevant to functions such as Generate New Module. A large amount of generated output may affect multiple subsequent connections. Without intermediate review, the engineer has less opportunity to understand, correct, or stop the change before it enters the workflow.

5.4.4 Predictability and Stability Mismatch

A fourth mismatch involved predictability. Participants lost confidence when LLM outputs changed unexpectedly, behaved inconsistently, or created new problems during repeated prompting. This included hidden changes, circular fixes, and prompt-to-prompt instability.

Predictability is central to trust calibration. Users need to form expectations about what a system will do and when it is likely to fail (Lee and See, 2004; Hoff and Bashir, 2015). If the system behaves differently across similar prompts or changes parts of the setup that the engineer did not intend to modify, the engineer must check more carefully. That increases verification effort and reduces confidence.

LLM variation is not always negative. Variation can be useful for ideation and exploration. The problem appears when variation affects workflow stability, introduces hidden changes, or makes it difficult for engineers to know what has changed.

5.4.5 Downstream Risk Mismatch

The final mismatch concerned downstream risk. Participants became more cautious when weak outputs could affect later stages of the pipeline. This finding reflects the specific context of a synthetic data workflow.

Synthetic data pipelines involve generation, curation, filtering, evaluation, and later use (Long et al., 2024; Nadas et al., 2025). An LLM output therefore cannot be judged only at the moment it is generated. It must also be judged by whether it can be safely carried forward. A weak module, poor parameter settings, or incorrect annotation decisions may lead to later rework or affect dataset quality.

Trust in SCAILAB is therefore stage-dependent. Engineers may accept LLM help for early ideas, but become more careful when the output affects downstream work. This supports the socio-technical view that system value depends on how technology fits within the overall work system, not solely on isolated task performance (Baxter and Sommerville, 2011).

5.5 RQ4: Design Recommendations for Trustworthy LLM Support

The findings suggest that trustworthy LLM support in SCAILAB should not aim for full automation. A more appropriate direction is support that helps engineers verify outputs, understand context, maintain control, anticipate changes, and manage downstream risk. These recommendations are grounded in both the thematic findings and the behavioural coding.

General human-AI interaction guidelines, such as those proposed by Amershi et al. (2019), provide a useful baseline for designing AI-supported systems. These guidelines address broad principles such as making clear what the system can and cannot do, supporting user feedback and correction, and helping users understand the scope of AI actions. The design recommendations developed in this thesis build on this foundation, but they are interpreted in relation to the specific conditions of the SCAILAB synthetic data pipeline. In this setting, three issues are particularly important. First, verification is stage dependent. What engineers need to check depends on where the LLM output will be used in the pipeline and what later stages it may affect. Second, control needs to be scoped. Engineers need to know not only whether they can reject or modify an output, but also what part of the pipeline the LLM has changed and whether that change may affect other nodes, parameters, or connections. Third, trust is shaped by downstream dependency. An accepted LLM output may influence later activities such as dataset preparation, annotation quality, model training, or evaluation. These conditions make the recommendations specific to the SCAILAB workflow rather than only a general application of existing human-AI design guidelines.

5.5.1 Make LLM Support Staged and Reviewable

The first recommendation is to make LLM support staged and reviewable. Participants preferred step-by-step support, especially when an output could affect later workflow stages. The system should not always generate a complete result in one step. It should show intermediate plans, assumptions, or partial outputs before applying larger changes.

Before generating a new module, for example, the system could show what it plans to create, what connections it assumes, and which parts of the pipeline may be affected. The engineer could then approve, edit, or reject the plan. This supports user control and appropriate reliance (Lee and See, 2004; Shneiderman, 2020).

5.5.2 Support Verification Before Reliance

The second recommendation is to support verification before reliance. Since verification was central to trust, LLM support should make checking easier rather than leaving engineers to inspect everything manually.

Possible features include validation checklists, output previews, error explanations, test suggestions, and comparison views between original and changed outputs. These features would help engineers decide whether an output is usable. They would also reduce the hidden verification burden identified in LLM-supported work (Mozannar et al., 2024; Tankelevitch et al., 2024).

5.5.3 Make Context, Assumptions, and Reasoning Visible

The third recommendation is to make context, assumptions, and reasoning visible. Participants wanted to understand why the system produced an output. They were less comfortable when the system made suggestions without explaining the basis for them.

This connects to explanation literature. Liao et al. (2020) show that users ask different types of explanatory questions depending on their task needs. Explanations are useful when they help users evaluate a system’s output. Bansal et al. (2021), however, warn that explanations can increase acceptance without improving judgement. For SCAILAB, explanations should therefore be practical and task-specific.

The system could show which requirement, parameter, or pipeline state was used to generate an output. It could also warn when important context is missing. This would help engineers judge whether the output fits the current task.

5.5.4 Keep Engineers in Control of Changes

The fourth recommendation is to keep engineers in control of changes. Participants were concerned about unexpected scope changes and unintended modifications. The system should clearly show what it intends to change and should not apply broad changes without review.

Possible design features include scoped editing, approve-or-reject controls, undo options, change summaries, and warnings before changes are applied. These features support

meaningful human control, which is central to human-centred AI, and they can also support better trust calibration by helping engineers judge whether reliance is appropriate (Shneiderman, 2020; Zhang et al., 2020). They also reduce the risk of passive acceptance by keeping the engineer actively involved in the decision.

This recommendation is especially relevant when LLM support affects pipeline structure, module generation, or connections between workflow elements.

5.5.5 Add Downstream Risk Safeguards

The fifth recommendation is to add downstream risk safeguards. Participants were concerned not only about immediate output quality, but also about what could happen later if a weak output was carried forward.

The system could warn engineers when an output may affect later pipeline stages, such as dataset preparation, annotation quality, model training, or evaluation. It could also provide "before continuing" checks at important workflow points. Before a generated module is used downstream, the system could ask the engineer to confirm key assumptions, validate connections, and review potential effects.

This recommendation is strongly connected to the pipeline context. In a multi-stage workflow, trustworthy support must help engineers manage consequences over time, not only produce outputs quickly (Long et al., 2024; Baxter and Sommerville, 2011).

Overall, the five design recommendations can serve as guidance for designing LLM-based support for the SCAILAB synthetic data pipeline. They are relevant to the pipeline. However, they are not intended to apply equally to every pipeline function. Some recommendations become more important when the LLM helps define the task context, while others become more important when the LLM generates, modifies, verifies, or connects pipeline components. Therefore, the practical relevance of the recommendations can be understood by looking at how they apply to specific points in the workflow.

During requirements entry and project configuration, engineers may start with incomplete requirements. For example, a bin-picking dataset may be defined without sufficient detail on object types, lighting variations, camera angles, or annotation requirements. If this information is missing, later stages such as asset creation, labelling, generation settings, and evaluation may also be affected. In this situation, the recommendation to make context and assumptions visible becomes particularly important. LLM support could ask follow-up questions or highlight missing information before the requirement is used in later pipeline stages.

Another example is pipeline construction and parameter support. In this part of the workflow, small changes to modules, parameters, or node connections may affect later validation or data generation. Therefore, the recommendations on verification and engineer control become more relevant. If the LLM suggests changing a parameter or connecting two nodes, the system could

show what will change and which later component may be affected. The engineer could then review the suggestion before it is carried forward in the pipeline.

These examples are not intended to cover every possible pipeline function. Instead, they show how SCAILAB could use the recommendations both to improve existing LLM-supported activities and to guide the design of future LLM support within the pipeline.

5.6 Chapter Summary

This chapter discussed the findings in relation to the four research questions. The discussion showed that trust in LLM-based support within the SCAILAB synthetic data pipeline is conditional, situated, and workflow-dependent. Engineers did not simply accept or reject the LLM. They judged its support according to task risk, confidence, verification effort, context, control, predictability, and downstream consequences.

The chapter also discussed where additional LLM support could be useful. These areas include requirement clarification, parameter support, pipeline configuration, validation, output checking, annotation review, and evaluation. However, the support should be designed carefully. The findings suggest that trustworthy LLM support should not aim for full automation. It should help engineers understand outputs, verify them efficiently, remain in control of changes, and recognise downstream risks before outputs are carried forward.

Overall, trustworthy LLM support in SCAILAB should be staged, verifiable, context-aware, controllable, and sensitive to downstream consequences.

Chapter 6 – Conclusion

6.1 Summary of Conclusions

This thesis examined how engineers experience and evaluate LLM-based support within the SCAILAB synthetic data pipeline. The study was motivated by the need to understand how trust is formed, reduced, and recalibrated when LLM support is introduced into a multi-stage engineering workflow. The focus was not on measuring LLM performance in technical terms, but on understanding how engineers judge whether the support is useful, reliable, verifiable, and safe to carry forward in practice.

The main conclusion of the thesis is that trust in LLM-based support is conditional and workflow-dependent. Engineers did not describe the LLM as simply trustworthy or untrustworthy. Their trust varied depending on the task, the risk involved, their confidence in judging the output, the clarity of the context, the effort required for verification, the level of control they retained, and the potential downstream consequences of weak outputs.

The findings showed that current LLM support in the SCAILAB pipeline was useful in selected areas, especially where it helped engineers begin, structure, or explore work. However, participants did not treat LLM outputs as final results. They expected to inspect, verify, modify, or reject outputs before relying on them. This suggests that LLM support becomes valuable when it works with engineering judgement rather than replacing it.

The study also showed that trust problems were not caused only by incorrect outputs. Trust was reduced when outputs were difficult to verify, when the system lacked task context, when changes were unpredictable, when engineers had limited control, or when weak outputs could affect later stages of the pipeline. In a synthetic data workflow, this is especially important because outputs move through connected stages. A decision made during configuration, generation, annotation, or evaluation can create later consequences for dataset quality, model training, or downstream use.

The design recommendations developed from the findings therefore focus on staged, verifiable, controllable, context-aware, and downstream-sensitive LLM support. These recommendations show how LLM support could be improved by making assistance more visible, reviewable, controllable, context-aware, and sensitive to downstream consequences.

Overall, the thesis shows that trust in LLM-based support is not formed only through whether an output appears correct at the moment it is generated. In the SCAILAB synthetic data pipeline, engineers calibrated trust by considering whether the output could be understood, verified, controlled, and safely carried forward into later workflow stages. This means that trust was situated in the workflow rather than attached to the LLM as a general tool. The central conclusion is therefore that trustworthy LLM support in this context should not aim to replace engineering judgement, but should strengthen engineers' ability to make informed, reviewable, and accountable decisions.

6.2 Theoretical Contributions

This thesis contributes to HCI and human-centred AI research by refining the understanding of trust in LLM-based support within multi-stage engineering workflows. Existing trust literature has established that trust depends on appropriate reliance, perceived competence, monitoring, predictability, and the ability to intervene (Mayer et al., 1995; Lee and See, 2004; Hoff and Bashir, 2015). These ideas were also visible in the SCAILAB case. Engineers evaluated LLM support based on the usefulness of the outputs, whether those outputs could be checked, whether the system behaved predictably, and whether they could maintain control over changes.

However, the findings also suggest that these conditions alone are insufficient to explain trust in a synthetic data pipeline. In this setting, engineers did not evaluate LLM outputs only at the point of production. They also considered whether an output could be carried forward safely into later stages of the workflow. A weak asset, module, parameter, or connection could affect later stages such as validation, dataset preparation, annotation quality, model training, or evaluation. This means that trust was shaped not only by immediate output quality, but also by downstream dependency.

The thesis therefore proposes downstream dependency as a workflow-specific condition for trust calibration in LLM-supported engineering work. Downstream dependency refers to the extent to which an LLM-supported output may affect later pipeline stages after it has been accepted or carried forward. This extends existing trust models by showing that, in multi-stage workflows, users need to judge not only whether the system is competent, reliable, or controllable at the moment of interaction, but also whether the consequences of relying on its output remain visible and manageable over time.

This finding also refines the idea of verification burden in LLM-supported work. Previous studies show that LLMs can create additional checking, repair, and interpretation work (Vaithilingam et al., 2022; Mozannar et al., 2024; Tankelevitch et al., 2024). The present study supports this, but adds that verification in pipeline-based work is distributed across stages. Engineers need to verify not only whether an immediate output is correct, but also whether it remains appropriate when connected to later pipeline activities. In this sense, verification becomes consequence-oriented rather than only output-oriented.

A related refinement concerns user control. In existing trust and human-AI literature, control is often discussed in terms of the user's ability to intervene, reject, correct, or override system behaviour. The SCAILAB case supports this, but also shows that control becomes more specific in a pipeline context. Engineers needed to understand what part of the pipeline the LLM was affecting, whether the change was limited to the intended component, and whether it could influence later nodes, parameters, validation steps, or dataset-related work. This suggests that control in LLM-supported pipeline work is not only about accepting or rejecting an output, but also about keeping changes scoped and traceable.

Based on this, the theoretical contribution of the thesis is a case-based refinement of trust calibration for LLM-supported synthetic data pipelines. The study suggests that trust in this context is shaped by three connected workflow conditions: downstream dependency, stage-dependent verification, and scoped control. Downstream dependency refers to the need to recognise how an accepted LLM output may affect later stages of the pipeline. Stage-dependent verification refers to the need to check outputs in relation to the workflow stage in which they will be used. Scoped control refers to the need to understand and limit what part of the pipeline the LLM is changing.

These contributions do not replace existing trust models. Instead, they extend them by showing how trust conditions such as competence, monitoring, control, and appropriate reliance operate differently when LLM support is embedded in a connected engineering workflow. In the SCAILAB case, trust was not only a judgement about whether the LLM output looked useful. It was also a judgement about whether that output could be verified, controlled, and carried forward without creating hidden downstream risks.

6.3 Practical Implications

The findings have practical implications for SCAILAB and for similar engineering environments where LLM support is being introduced into complex workflows. The first implication is that LLM support should be treated as assistive rather than autonomous. Engineers valued support that helped them start, structure, and explore work, but they still needed to inspect and approve outputs before relying on them.

The second implication is that future LLM support should make verification easier. Engineers need to see what has been checked, what evidence supports a warning or suggestion, what assumptions are being made, and what risks remain. Support that only produces an answer without helping the engineer evaluate it may increase the verification burden rather than reduce it.

The third implication is that engineers should retain control over changes. Suggested fixes, generated modules, parameter changes, or pipeline modifications should be reviewable before they are applied. Features such as staged interaction, change previews, review and approval controls, rejection options, undo options, and visible assumptions can help engineers maintain control while still benefiting from LLM assistance.

These practical implications should be understood as design directions rather than fixed technical requirements. Their value lies in showing how LLM support can be integrated into SCAILAB's pipeline in a way that respects how engineers already work. The findings suggest that future support should be integrated into the workflow, not implemented as a separate automation layer. In practice, this means that LLM features should support staged generation, visible assumptions, local verification, change preview, and engineer approval before outputs are carried forward. This would allow SCAILAB to improve LLM support while still keeping engineers responsible for final judgement and downstream quality.

6.4 Limitations

The study has several limitations. First, it was conducted as a qualitative case study within a single organisation and a single synthetic data pipeline. This allowed the research to examine trust in depth within the SCAILAB context, but it also limits how far the findings can be generalised to other organisations, tools, or engineering domains.

The participant group was limited, partly because SCAILAB is a small startup based in Västerås, Sweden, with only a few internal engineers working directly on the synthetic data pipeline and its LLM-supported functions. In this exploratory qualitative case study, this group provided valuable, experience-based insights into the pipeline. However, involving a larger or more diverse group could have brought additional perspectives. Engineers with varying experience levels, external users, or client-side stakeholders might have raised different trust concerns or identified other support needs.

The verbal data from think-aloud sessions reflect what participants chose to express during task performance, meaning that unexpressed hesitations or implicit trust judgements may not be fully represented in the findings. Future studies could combine think-aloud sessions with eye-tracking. This would make it possible to examine where engineers look, which interface elements attract attention, and how visual attention relates to verification, hesitation, and trust-related decisions.

Behavioural coding of the screen recordings also involved an interpretive limitation. Observing that a participant repeatedly returned to an earlier configuration step, spent extended time on validation, or corrected an output does not, by itself, confirm that the participant wanted, expected, or would have benefited from LLM support at that point. These patterns required researcher interpretation when they were considered trust-related or support-relevant. For this reason, the behavioural findings are framed as researcher-identified opportunities rather than participant-confirmed needs, and RQ2 is treated as exploratory. Future research could address this limitation by asking participants to reflect on selected moments from their screen recordings after the session, or by conducting follow-up interviews focused specifically on the workflow points identified through behavioural coding.

The study also focused on trust rather than technical benchmarking. It did not measure the objective accuracy of LLM outputs or compare model performance across systems. This focus was consistent with the research aim, but it means that the findings should be read as an analysis of human experience and workflow support rather than as a technical evaluation of the LLM.

The transferability of the findings should therefore be understood carefully. The findings are most transferable to professional engineering settings where LLM support is embedded in a multi-stage workflow, where outputs affect later tasks, and where users remain responsible for checking and accepting system suggestions. The findings may be less transferable to casual LLM use, single-step tasks, or domains where errors are easy to identify and reverse. In this sense, the study does not claim that all engineers will experience LLM support in the same

way. Instead, it provides a situated account of how trust is shaped when LLM support is used inside a connected synthetic data workflow.

6.5 Future Research

Future research could extend this study by examining LLM-supported synthetic data pipelines across multiple organisations or project contexts. Since this thesis focused on a single organisation and pipeline, further studies could explore whether similar trust-related conditions occur in other engineering environments. This would help assess the transferability of the findings and identify which trust issues are specific to SCAILAB and which are more general to LLM-supported engineering workflows.

A second direction for future research would be to study trust over a longer period of use. This thesis captured engineers' experiences at a specific point in time, but trust in LLM support is likely to develop through repeated interaction. A longitudinal study could examine how engineers' trust changes over several weeks or months as they use improved LLM support. This would make it possible to understand how successful outputs, repeated verification, unexpected changes, and visible failures shape long-term reliance.

Future work could also implement and evaluate the design recommendations developed in this thesis. For example, prototypes could be created for staged generation, visible assumptions, change previews, verification checkpoints, and downstream risk warnings. These prototypes could then be tested with engineers to examine whether they reduce verification effort, improve workflow fit, and support more appropriate trust.

Another useful direction would be to combine qualitative methods with quantitative measures. Future studies could measure time spent verifying LLM outputs, number of corrections made, acceptance and rejection rates, number of prompt iterations, and downstream errors detected after LLM-supported steps. This would help connect engineers' subjective trust experiences with observable workflow performance.

Future research could also examine how different types of LLM support affect trust at different pipeline stages. For example, support for requirement clarification may require strong context awareness, while support for module generation may require stronger change previews, verification checkpoints, and rollback options. Studying these differences could help design LLM support that is more stage-specific rather than applying the same type of assistance across the whole pipeline.

Finally, future studies could include additional stakeholders beyond internal engineers, such as project managers, client-side users, or people responsible for evaluating the generated synthetic data. This would help examine how trust in LLM-supported pipeline work extends beyond the individual engineer and affects broader organisational confidence in synthetic data outputs.

6.6 Usage of Tools

Microsoft Excel was used to organise the coding spreadsheets, compare evidence across participants, and maintain a traceable link between data excerpts, codes, themes, and analytical interpretations. Draw.io was used to create the methodological and analytical diagrams included in the thesis, including the methodological framework and analytical framework diagrams (draw.io, n.d.). Grammarly and OpenAI ChatGPT were used to support grammar checking, spelling correction, language refinement, and structural clarity (Grammarly, n.d.; OpenAI, n.d.). NotebookLM was used as a note-organisation and reading-support tool during the writing process (Google, n.d.). These tools were not used to generate participant data, replace coding decisions, conduct the thematic analysis, or determine the study's findings.

References

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S.T., Bennett, P.N., Inkpen, K., Teevan, J., Kikin-Gil, R. and Horvitz, E. (2019). Guidelines for human-AI interaction. In: *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. New York: Association for Computing Machinery, pp. 1–13. doi: 10.1145/3290605.3300233.

Bansal, G., Wu, T., Zhou, J., Fok, R., Nushi, B., Kamar, E., Ribeiro, M.T. and Weld, D.S. (2021). Does the whole exceed its parts? The effect of AI explanations on complementary team performance. In: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. New York: Association for Computing Machinery, Article 81, pp. 1–16. doi: 10.1145/3411764.3445717.

Barke, S., James, M.B. and Polikarpova, N. (2023). Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), pp. 85–111. doi: 10.1145/3586030.

Baxter, G. and Sommerville, I. (2011). Socio-technical systems: From design methods to systems engineering. *Interacting with Computers*, 23(1), pp. 4–17. doi: 10.1016/j.intcom.2010.07.003.

Braun, V. and Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), pp. 77–101. doi: 10.1191/1478088706qp063oa.

Cai, C.J., Winter, S., Steiner, D., Wilcox, L. and Terry, M. (2019). Hello AI: Uncovering the onboarding needs of medical practitioners for human-AI collaborative decision-making. *Proceedings of the ACM on Human-Computer Interaction*, 3(CSCW), Article 104, pp. 1–24. doi: 10.1145/3359206.

Chen, K., Cao, R., James, S., Li, Y., Liu, Y.-H., Abbeel, P. and Dou, Q. (2022). Sim-to-real 6D object pose estimation via iterative self-training for robotic bin picking. In: *Computer Vision – ECCV 2022*, Lecture Notes in Computer Science, vol. 13699. Cham: Springer, pp. 533–550. doi: 10.1007/978-3-031-19842-7_31.

Creswell, J.W. and Creswell, J.D. (2018). *Research design: Qualitative, quantitative, and mixed methods approaches*. 5th ed. Thousand Oaks, CA: SAGE Publications.

Denzin, N.K. (1978). *The research act: A theoretical introduction to sociological methods*. 2nd ed. New York: McGraw-Hill.

Dietvorst, B.J., Simmons, J.P. and Massey, C. (2015). Algorithm aversion: People erroneously avoid algorithms after seeing them err. *Journal of Experimental Psychology: General*, 144(1), pp. 114–126. doi: 10.1037/xge0000033.

Dietvorst, B.J., Simmons, J.P. and Massey, C. (2018). Overcoming algorithm aversion: People will use imperfect algorithms if they can (even slightly) modify them. *Management Science*, 64(3), pp. 1155–1170. doi: 10.1287/mnsc.2016.2643.

draw.io (n.d.). *Flowchart maker & online diagram software*. Available at: https://www.drawio.com/ [Accessed 24 May 2026].

Ericsson, K.A. and Simon, H.A. (1993). *Protocol analysis: Verbal reports as data*. Revised ed. Cambridge, MA: MIT Press.

Glikson, E. and Woolley, A.W. (2020). Human trust in artificial intelligence: Review of empirical research. *Academy of Management Annals*, 14(2), pp. 627–660. doi: 10.5465/annals.2018.0057.

Google (n.d.). *Google NotebookLM: AI research tool and thinking partner*. Available at: https://www.notebooklm.google/ [Accessed 24 May 2026].

Grammarly (n.d.). *Grammarly: Free AI writing assistance*. Available at: https://www.grammarly.com/ [Accessed 24 May 2026].

Hoff, K.A. and Bashir, M. (2015). Trust in automation: Integrating empirical evidence on factors that influence trust. *Human Factors*, 57(3), pp. 407–434. doi: 10.1177/0018720814547570.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J. and Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), Article 220, pp. 1–79. doi: 10.1145/3695988.

International Federation of Robotics (2024). *World Robotics 2024 Report by International Federation of Robotics released*. Available at: https://ifr.org/ifr-press-releases/news/record-of-4-million-robots-working-in-factories-worldwide [Accessed 24 May 2026].

International Federation of Robotics (2025). *World Robotics 2025 report – Industrial Robots*. Available at: https://ifr.org/ifr-press-releases/news/global-robot-demand-in-factories-doubles-over-10-years [Accessed 24 May 2026].

Ivarsson, J., & Lindwall, O. (2023). Suspicious minds: The problem of trust and conversational agents. *Computer Supported Cooperative Work (CSCW), 32*, 545–571. https://doi.org/10.1007/s10606-023-09465-8

James, S., Wohlhart, P., Kalakrishnan, M., Kalashnikov, D., Irpan, A., Ibarz, J., Levine, S., Hadsell, R. and Bousmalis, K. (2019). 'Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks'. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 12627–12637.

Kallio, H., Pietilä, A.-M., Johnson, M. and Kangasniemi, M. (2016). Systematic methodological review: developing a framework for a qualitative semi-structured interview guide. *Journal of Advanced Nursing*, 72(12), pp. 2954–2965. doi: 10.1111/jan.13031.

Kelly, M., Kumar, A., Smyth, P. and Steyvers, M. (2023). 'Capturing humans' mental models of AI: An item response theory approach'. In: *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency (FAccT '23)*, Chicago, IL, USA,

12–15 June. New York: Association for Computing Machinery, 18 pages. doi: 10.1145/3593013.3594111.

Klingbeil, A., Grützner, C. and Schreck, P. (2024). Trust and reliance on AI – An experimental study on the extent and costs of overreliance on AI. *Computers in Human Behavior*, 160, Article 108352. doi: 10.1016/j.chb.2024.108352.

Kocielnik, R., Amershi, S. and Bennett, P.N. (2019). Will you accept an imperfect AI? Exploring designs for adjusting end-user expectations of AI systems. In: *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. New York: Association for Computing Machinery, Article 411, pp. 1–14. doi: 10.1145/3290605.3300641.

Lee, J.D. and See, K.A. (2004). Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1), pp. 50–80. doi: 10.1518/hfes.46.1.50_30392.

Liang, J.T., Yang, C. and Myers, B.A. (2024). A large-scale survey on the usability of AI programming assistants: Successes and challenges. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. New York: Association for Computing Machinery, pp. 616–628. doi: 10.1145/3597503.3608128.

Liao, Q.V., Gruen, D. and Miller, S. (2020). Questioning the AI: Informing design practices for explainable AI user experiences. In: *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. New York: Association for Computing Machinery, pp. 1–15. doi: 10.1145/3313831.3376590.

Lincoln, Y.S. and Guba, E.G. (1985). *Naturalistic inquiry*. Beverly Hills, CA: SAGE Publications.

Long, L., Wang, R., Xiao, R., Zhao, J., Ding, X., Chen, G. and Wang, H. (2024). On LLMs-Driven Synthetic Data Generation, Curation, and Evaluation: A Survey. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.658.

Mayer, R.C., Davis, J.H. and Schoorman, F.D. (1995). An integrative model of organizational trust. *Academy of Management Review*, 20(3), pp. 709–734. doi: 10.5465/amr.1995.9508080335.

Mehrotra, S., Degachi, C., Vereschak, O., Jonker, C.M. and Tielman, M.L. (2024). A systematic review on fostering appropriate trust in human-AI interaction: Trends, opportunities and challenges. *ACM Journal on Responsible Computing*, 1(4), Article 21, pp. 1–28. doi: 10.1145/3696449.

Merriam, S.B. and Tisdell, E.J. (2016). *Qualitative research: A guide to design and implementation*. 4th ed. San Francisco, CA: Jossey-Bass.

Mozannar, H., Bansal, G., Fourney, A. and Horvitz, E. (2024). Reading between the lines: Modeling user behavior and costs in AI-assisted programming. In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*. New York: Association for Computing Machinery, Article 155, pp. 1–16. doi: 10.1145/3613904.3641936.

Nadas, M., Diosan, L. and Tomescu, A. (2025). 'Synthetic Data Generation Using Large Language Models: Advances in Text and Code'. *IEEE Access*, 13, pp. 134615–134633. doi: 10.1109/ACCESS.2025.3589503.

Nourani, M., King, J.T. and Ragan, E.D. (2020). 'The role of domain expertise in user trust and the impact of first impressions with intelligent systems'. *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing*, 8(1), pp. 112–121. doi: 10.1609/hcomp.v8i1.7469.

Nowell, L.S., Norris, J.M., White, D.E. and Moules, N.J. (2017). Thematic analysis: Striving to meet the trustworthiness criteria. *International Journal of Qualitative Methods*, 16(1), pp. 1–13. doi: 10.1177/1609406917733847.

OpenAI (n.d.) ChatGPT. Available at: https://openai.com/chatgpt/ [Accessed: 25 May 2026].

Parasuraman, R., Sheridan, T.B. and Wickens, C.D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 30(3), pp. 286–297. doi: 10.1109/3468.844354.

Patton, M.Q. (2015). *Qualitative research & evaluation methods: Integrating theory and practice*. 4th ed. Thousand Oaks, CA: SAGE Publications.

Sarkar, A., Gordon, A.D., Negreanu, C., Poelitz, C., Ragavan, S.S. and Zorn, B. (2022). 'What is it like to program with artificial intelligence?' In: *Proceedings of the 33rd Annual Conference of the Psychology of Programming Interest Group (PPIG 2022)*. Available at: https://ppig.org/papers/2022-ppig-33rd-sarkar/ [Accessed: 09 March 2026].

SCAILAB (2026). *SCAILAB*. Available at: https://www.scailab.se [Accessed: 25 May 2026].

Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6), pp. 495–504. doi: 10.1080/10447318.2020.1741118.

Singh, R., Liu, J., Van Wyk, K., Chao, Y.-W., Lafleche, J.-F., Shkurti, F., Ratliff, N. and Handa, A. (2024). 'Synthetica: large scale synthetic data for robot perception'. arXiv preprint arXiv:2410.21153. Available at: https://arxiv.org/abs/2410.21153 (Accessed: 24 May 2026). doi: 10.48550/arXiv.2410.21153.

Tang, J.C., Liu, S.B., Muller, M.J., Lin, J. and Drews, C. (2006). Unobtrusive but invasive: Using screen recording to collect field data on computer-mediated interaction. In: *Proceedings of the 2006 20th Anniversary Conference on Computer Supported Cooperative Work*. New York: Association for Computing Machinery, pp. 479–482. doi: 10.1145/1180875.1180948.

Tankelevitch, L., Kewenig, V., Simkute, A., Scott, A.E., Sarkar, A., Sellen, A. and Rintel, S. (2024). The metacognitive demands and opportunities of generative AI. In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*. New York: Association for Computing Machinery, pp. 1–24. doi: 10.1145/3613904.3642902.

Tan, J., Zhang, T., Coumans, E., Iscen, A., Bai, Y., Hafner, D., Bohez, S. and Vanhoucke, V. (2018). Sim-to-real: Learning agile locomotion for quadruped robots. In: *Robotics: Science and Systems XIV*. doi: 10.15607/RSS.2018.XIV.010.

Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W. and Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30. doi: 10.1109/IROS.2017.8202133.

Vaithilingam, P., Zhang, T. and Glassman, E.L. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In: *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, pp. 1–7. doi: 10.1145/3491101.3519665.

Wang, R., Cheng, R., Ford, D. and Zimmermann, T. (2024). Investigating and designing for trust in AI-powered code generation tools. In: *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*. New York: Association for Computing Machinery, pp. 1475–1493. doi: 10.1145/3630106.3658984.

Xie, Z., Liang, X. and Roberto, C. (2023). Learning-based robotic grasping: A review. *Frontiers in Robotics and AI*, 10, Article 1038658. doi: 10.3389/frobt.2023.1038658.

Yin, M., Vaughan, J.W. and Wallach, H. (2019). Understanding the effect of accuracy on trust in machine learning models. In: *Conference on Human Factors in Computing Systems - Proceedings*. Association for Computing Machinery. doi: 10.1145/3290605.3300509.

Yin, R.K. (2018). *Case study research and applications: Design and methods*. 6th ed. Thousand Oaks, CA: SAGE Publications.

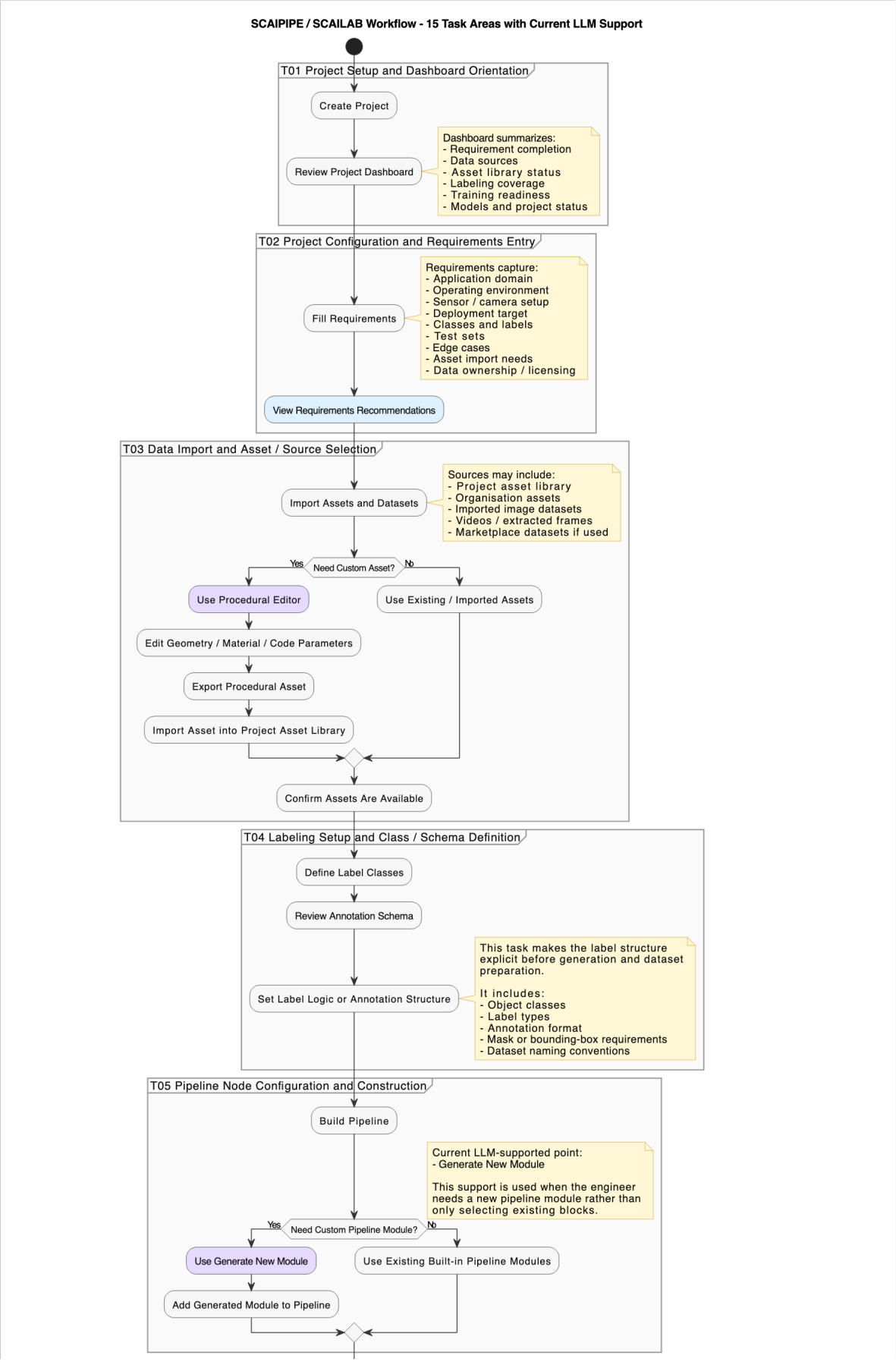
Yu, W., Kumar, V.C., Turk, G. and Liu, C.K. (2019). Sim-to-real transfer for biped locomotion. In: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3503–3510. doi: 10.1109/IROS40897.2019.8968053.

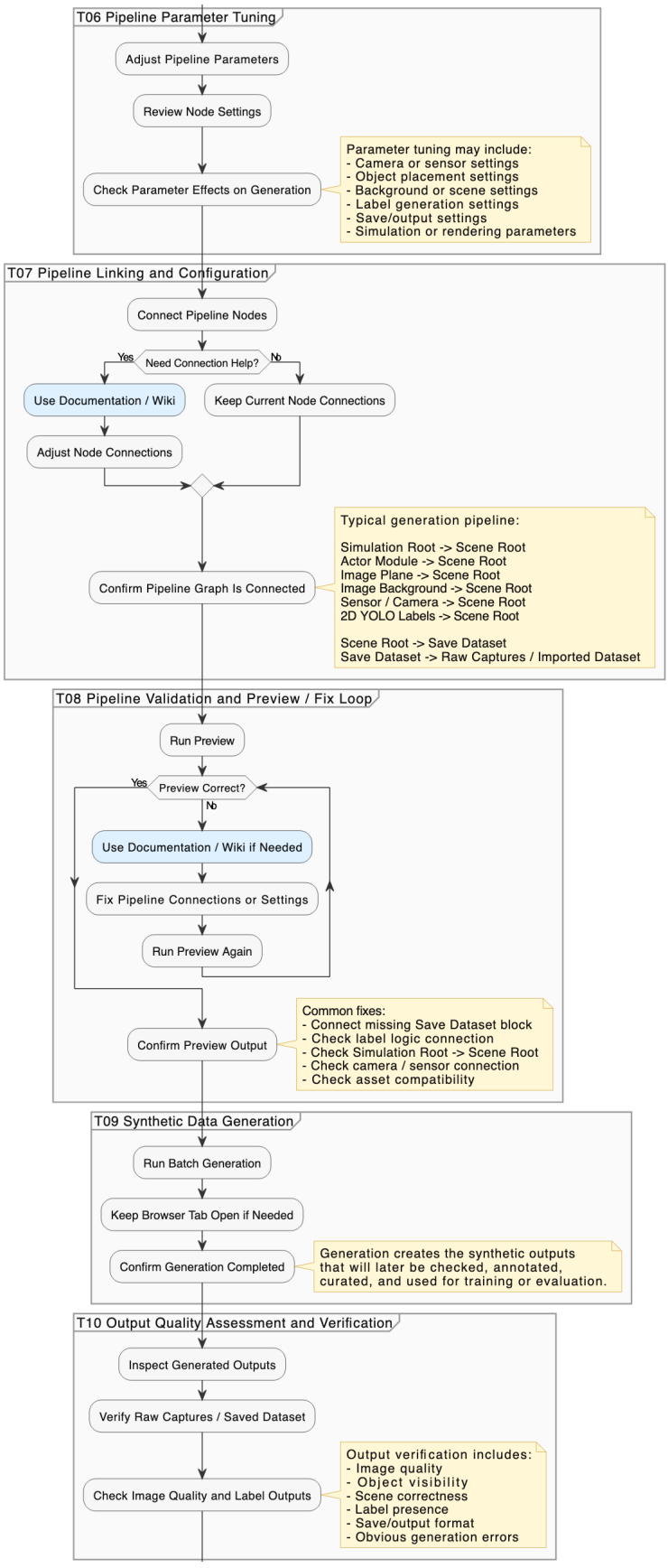
Zamfirescu-Pereira, J.D., Wong, R.Y., Hartmann, B. and Yang, Q. (2023). Why Johnny can't prompt: How non-AI experts try (and fail) to design LLM prompts. In: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. New York: Association for Computing Machinery, Article 437, pp. 1–21. doi: 10.1145/3544548.3581388.

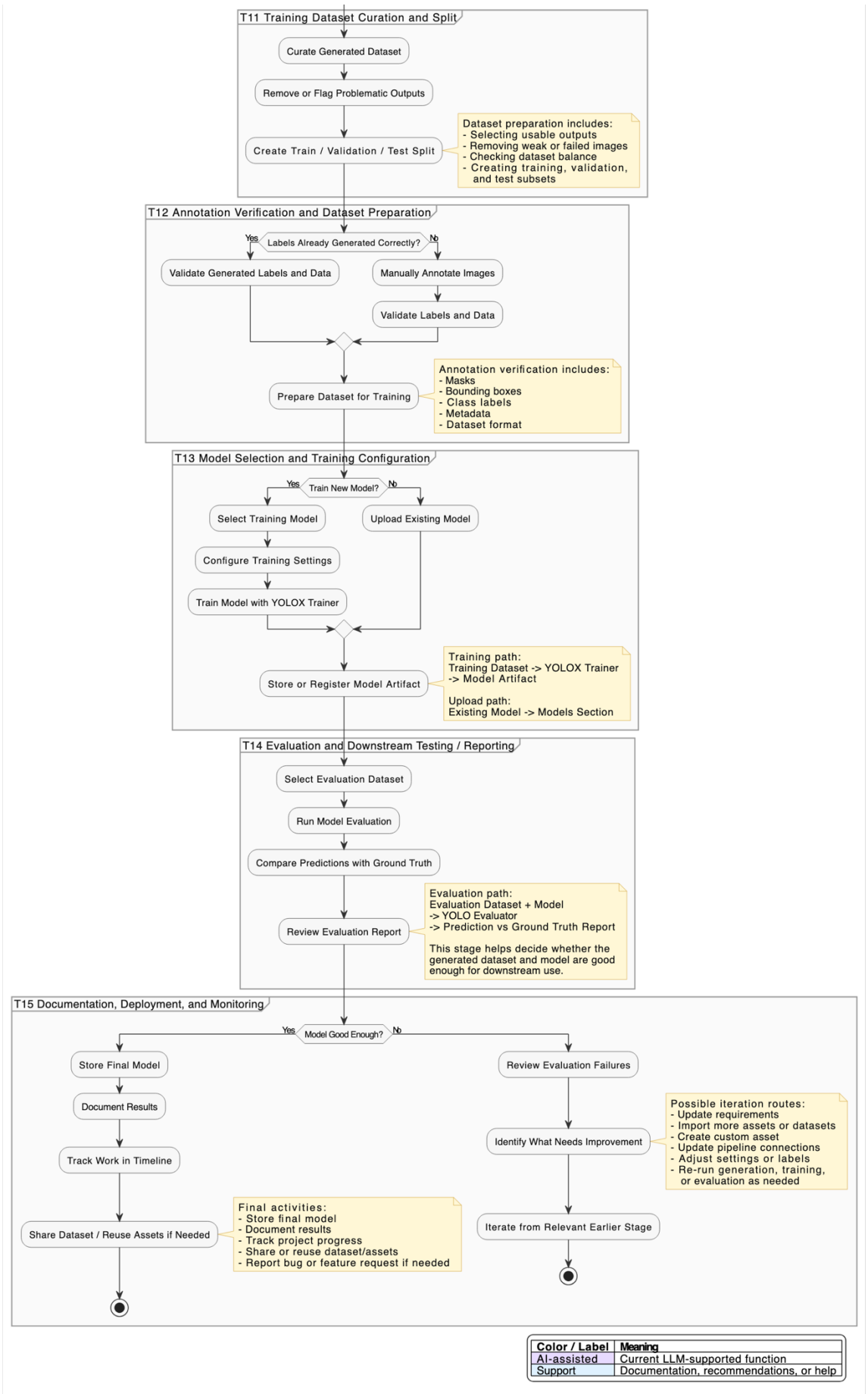
Zhang, Y., Liao, Q.V. and Bellamy, R.K.E. (2020). Effect of confidence and explanation on accuracy and trust calibration in AI-assisted decision making. In: *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*. New York: Association for Computing Machinery, pp. 295–305. doi: 10.1145/3351095.3372852.

Zhao, D., Xing, Z., Xia, X., Ye, D., Xu, X. and Zhu, L. (2023). SeeHow: Workflow extraction from programming screencasts through action-aware video analytics. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1946–1957. doi: 10.1109/ICSE48619.2023.00165.

Annex 1: SCAILAB Synthetic Data Pipeline full workflow




T06 Pipeline Parameter Tuning
Adjust Pipeline Parameters
Review Node Settings
Check Parameter Effects on Generation
Parameter tuning may include:
- Camera or sensor settings
- Object placement settings
- Background or scene settings
- Label generation settings
- Save/output settings
- Simulation or rendering parameters
T07 Pipeline Linking and Configuration
Connect Pipeline Nodes
Need Connection Help?
Yes
No
Use Documentation / Wiki
Keep Current Node Connections
Adjust Node Connections
Confirm Pipeline Graph Is Connected
Typical generation pipeline:
Simulation Root -> Scene Root
Actor Module -> Scene Root
Image Plane -> Scene Root
Image Background -> Scene Root
Sensor / Camera -> Scene Root
2D YOLO Labels -> Scene Root
Scene Root -> Save Dataset
Save Dataset -> Raw Captures / Imported Dataset
T08 Pipeline Validation and Preview / Fix Loop
Run Preview
Preview Correct?
Yes
No
Use Documentation / Wiki if Needed
Fix Pipeline Connections or Settings
Run Preview Again
Confirm Preview Output
Common fixes:
- Connect missing Save Dataset block
- Check label logic connection
- Check Simulation Root -> Scene Root
- Check camera / sensor connection
- Check asset compatibility
T09 Synthetic Data Generation
Run Batch Generation
Keep Browser Tab Open if Needed
Confirm Generation Completed
Generation creates the synthetic outputs
that will later be checked, annotated,
curated, and used for training or evaluation.
T10 Output Quality Assessment and Verification
Inspect Generated Outputs
Verify Raw Captures / Saved Dataset
Check Image Quality and Label Outputs
Output verification includes:
- Image quality
- Object visibility
- Scene correctness
- Label presence
- Save/output format
- Obvious generation errors


T11 Training Dataset Curation and Split
Curate Generated Dataset
Remove or Flag Problematic Outputs
Create Train / Validation / Test Split
Dataset preparation includes:
- Selecting usable outputs
- Removing weak or failed images
- Checking dataset balance
- Creating training, validation, and test subsets
T12 Annotation Verification and Dataset Preparation
Labels Already Generated Correctly?
Yes
No
Validate Generated Labels and Data
Manually Annotate Images
Validate Labels and Data
Prepare Dataset for Training
Annotation verification includes:
- Masks
- Bounding boxes
- Class labels
- Metadata
- Dataset format
T13 Model Selection and Training Configuration
Train New Model?
Yes
No
Select Training Model
Upload Existing Model
Configure Training Settings
Train Model with YOLOX Trainer
Store or Register Model Artifact
Training path:
Training Dataset -> YOLOX Trainer
-> Model Artifact
Upload path:
Existing Model -> Models Section
T14 Evaluation and Downstream Testing / Reporting
Select Evaluation Dataset
Run Model Evaluation
Compare Predictions with Ground Truth
Review Evaluation Report
Evaluation path:
Evaluation Dataset + Model
-> YOLO Evaluator
-> Prediction vs Ground Truth Report
This stage helps decide whether the generated dataset and model are good enough for downstream use.
T15 Documentation, Deployment, and Monitoring
Model Good Enough?
Yes
No
Store Final Model
Document Results
Track Work in Timeline
Share Dataset / Reuse Assets if Needed
Final activities:
- Store final model
- Document results
- Track project progress
- Share or reuse dataset/assets
- Report bug or feature request if needed
Review Evaluation Failures
Identify What Needs Improvement
Possible iteration routes:
- Update requirements
- Import more assets or datasets
- Create custom asset
- Update pipeline connections
- Adjust settings or labels
- Re-run generation, training, or evaluation as needed
Iterate from Relevant Earlier Stage
Color / Label
Meaning
AI-assisted
Current LLM-supported function
Support
Documentation, recommendations, or help

Annex 2: Interview analysis overview

Codebook

Codebook (Deductive + Inductive)						
Parent_Code	Definition	Include	Exclude / boundary	Guide prompt area	Linked working theme	Code / cluster label used in thesis
General trust	How participants describe trusting or not trusting LLM support overall.	Broad comments on relying on outputs, baseline confidence, trust language.	Specific comments only about verification burden; code those under Verifiability too if needed.	A3	Trust Depends on Task Risk and Engineer Confidence	task-complexity calibration
Accuracy expectations	How expectations of performance changed after experience with the system.	Expectation vs reality, surprise, disappointment, acceptable performance.	Purely present-day trust statements without comparison to expectations.	B2	Trust Depends on Task Risk and Engineer Confidence	task-complexity calibration
Predictability	Whether outputs feel stable, consistent, or dependable across time or users.	Consistency, time pressure, repeatability, confidence under deadline.	Checking output after the fact without discussing consistency.	C2 / C3	Trust Decreases When LLM Outputs Change Unexpectedly	counter-prompt loop / hidden change
Verifiability	How easy or difficult it is to evaluate whether an output is correct enough to use.	Need to double-check, auditability, validation effort.	General complaints about context or control unless directly tied to checking.	D1 / D2	Trust Requires Easy Verification	verification-related concern
Transparency / explanation	Need for explanations, reasoning visibility, local or global understanding.	Requests for why/how the model suggested something, inspectability.	Comments about autonomy without explanation.	E3	Trust Depends on Clear Context and Explanation	explanation-related concern
Control / user agency	Degree to which users want to steer, revise, or override the system.	Prompt steering, retries, edits, wanting final say.	Pure verification statements without agency dimension.	F1	Trust Increases When Engineers Have Control	user control
Automation vs control	Attitudes toward full automation compared with assistive support.	Caution about autonomous behavior, preference for support not replacement.	Minor workflow comments without automation implications.	F3	Trust Increases When Engineers Have Control	control-related concern
Perceived competence over time	Whether trust changes as users observe the system over repeated use.	Growing trust, declining trust, calibration over time.	Static trust comments without time dimension.	G3	Trust Decreases When LLM Outputs Change Unexpectedly	counter-prompt loop
Known limitations	Explicit boundaries where LLM support is seen as weak or unsuitable.	Failure cases, weaknesses, non-ideal scenarios.	General negative sentiment without a stated limitation.	H2	Trust Depends on Task Risk and Engineer Confidence	task-complexity calibration
Context understanding	Whether the system appears to grasp the full task, environment, or engineering context.	Missing context, local vs global understanding, domain mismatch.	Generic explanation requests with no context dimension.	I4	Trust Depends on Clear Context and Explanation	prompt dependency
Problem structuring / prompt clarity	How much output quality depends on clear task framing, decomposition, and prompt quality.	Comments about breaking down the problem, structuring the request, ambiguity in wording, or the need to articulate tasks clearly.	Pure workflow comments that are only about where to use LLMs in the pipeline.	C2 / J3 / B2	Trust Depends on Clear Context and Explanation	prompt-structure concern
Workflow fit / redesign	How LLM support fits into pipeline stages and how workflows may need redesign.	Best stage to use LLMs, redesign suggestions, task decomposition.	Only trust comments with no workflow dimension.	J3	Trust Is Limited by Downstream Risk	early framing value
Past experience / advice	Advice to colleagues or lessons learned from prior use.	Recommendations, rules of thumb, onboarding guidance.	General opinion without advisory framing.	N4	Trust Requires Easy Verification	verification-related concern
Downstream effect	How poor or good outputs influence later pipeline steps or data quality.	Error propagation, trash data, later-stage cleanup.	Local one-off mistakes with no downstream impact.	M3	Trust Is Limited by Downstream Risk	downstream concern
Iterative instability / hidden changes	How repeated prompting or local fixes create drift, loops, or unintended changes elsewhere.	Changing one thing breaks another, moving problems around, cycling, unintended side effects.	Simple verification comments without cross-step instability.	A3 / C3 / F1 / G3	Trust Decreases When LLM Outputs Change Unexpectedly	counter-prompt loop / hidden change
Controlled LLM inclusion / core-data boundary	How participants define boundaries around where LLMs should be included so core data/model outcomes remain human- or simulation-governed.	LLM proposes settings, templates, or support, but the user decides and core data generation remains controlled.	General verification comments without discussion of where LLMs should or should not be placed in the pipeline.	A3 / F3 / J3 / M3	Trust Increases When Engineers Have Control	control-related concern

Theme summary

Theme Summary				
Theme	Interpretive definition	Participant breadth	Question breadth	Illustrative evidence
Trust Depends on Task Risk and Engineer Confidence	Trust is not stable across uses. Participants calibrate trust based on task complexity, stakes, domain risk, and how well they understand the problem. P04 strengthens this by stressing that LLMs should stay outside core data-generation decisions.	P01, P02, P03, P04	A2, A3, B2, C3, H2	"the LLM proposes something and then the user decides"; "depends on the complexity of the task"
Trust Requires Easy Verification	LLM output is rarely accepted at face value. Practical use depends on whether the result can be tested, inspected, or checked by a knowledgeable user.	P01, P02, P03, P04	A3, C3, D1, D2, N4	"you must check whether what the LLM gives you makes sense"; "double check everything"
Trust Depends on Clear Context and Explanation	Output quality depends heavily on how the task is framed, how much context is supplied, and whether the user can ask for explanation or restructuring.	P01, P02, P03, P04	C2, E3, H2, I4	"it is not deterministic"; "if you do not have a structured way of defining"
Trust Increases When Engineers Have Control	Participants prefer LLMs as steerable assistants rather than autonomous agents. They want to review, redirect, and approve outputs before use.	P01, P02, P03, P04	D3, F1, F3	"still very much on human control"; "I can kind of debug that"
Trust Decreases When LLM Outputs Change Unexpectedly	Repeated prompting can introduce drift or unexpected behaviour: fixing one issue may break another, create loops, or produce something different from what was intended.	P01, P02, P03, P04	A3, B2, C3, F1, G3	"not at all what I meant"; "we fix something, a broken other thing"
Trust Is Limited by Downstream Risk	The clearest workflow value is early support, templates, problem framing, and decomposition. At the same time, weak outputs can propagate into datasets, labels, training data, or user-facing pipeline stages.	P01, P02, P03, P04	A2, E3, J3, M3	"better templates or more templates for the pipeline"; "compromises everything later in the pipeline"

Annex 3: Think-aloud analysis overview

Codebook

Code name	Short definition	When this code was used	When this code was not used / exclusion criteria	Related RQs	Chapter 4 theme link
Parameter verification before trust	LLM output is not accepted until parameters or results can be checked.	Used when participants described checking, testing, previewing, validating, or double-checking before relying on output.	Do not use for general positive or negative comments about the LLM unless the participant specifically checks, verifies, previews, validates, or double-checks an output before relying on it.	RQ1, RQ3, RQ4	Trust Requires Easy Verification
Useful starting point requiring inspection	LLM output is useful as a draft or scaffold, but still requires review before use.	Used when participants treated output as helpful for starting, structuring, or orienting work, but not as final.	Do not use when participants fully accept or reject an output without describing it as a draft, scaffold, starting point, or material requiring further inspection.	RQ1, RQ3, RQ4	Trust Requires Easy Verification; Trust Is Limited by Downstream Risk
Staged and reviewable generation	Engineers prefer stepwise, editable, and reviewable LLM support.	Used when participants wanted intermediate steps, outlines, checkpoints, or confirmation before continuing.	Do not use for any editable output unless the participant specifically refers to stepwise review, checkpoints, intermediate confirmation, or the need to control the generation process.	RQ3, RQ4	Trust Increases When Engineers Have Control
Explanation before acceptance	Engineers need reasons, assumptions, or dependencies before accepting output.	Used when participants asked why a suggestion was made or needed an explanation before acceptance.	Do not use for general requests for better output quality unless the participant asks for, refers to, or needs reasons, assumptions, dependencies, or explanations before accepting the output.	RQ3, RQ4	Trust Depends on Clear Context and Explanation; Trust Increases When Engineers Have Control
Task-dependent usefulness of LLM support	Trust and usefulness depend on task complexity, risk, familiarity, and engineer expertise.	Used when participants judged LLM usefulness differently across simple, familiar, complex, or risky tasks.	Do not use for task difficulty alone unless the participant connects usefulness, trust, or reliance to task complexity, risk, familiarity, or their own expertise.	RQ1, RQ3	Trust Depends on Task Risk and Engineer Confidence
Prompt-context dependency	Output quality depends on clear task, module, assumption, and workflow context.	Used when participants linked output reliability to prompt clarity, added details, or structured task context.	Do not use for poor output quality alone unless the participant links the quality of the output to missing, unclear, insufficient, or poorly structured prompt/task/workflow context.	RQ1, RQ3, RQ4	Trust Depends on Clear Context and Explanation
Prompt-to-prompt instability	Repeated prompting or repair loops produce shifting, inconsistent, or circular outputs.	Used when participants described inconsistent results, circular fixes, or different outputs across prompts.	Do not use for a single incorrect output unless the participant describes shifting outputs, repeated prompting, circular fixes, inconsistent results, or instability across prompts.	RQ3	Trust Decreases When LLM Outputs Change Unexpectedly
Workflow ambiguity	Output is hard to judge or use because it does not clearly fit workflow or module constraints.	Used when participants found output generic, unclear, unsuitable, or insufficiently connected to the pipeline context.	Do not use for any general workflow issue unless the participant describes difficulty judging, applying, or fitting the LLM output within the pipeline, module, or task constraints.	RQ2, RQ3	Trust Depends on Clear Context and Explanation
Unexpected scope change	The LLM changes or appears to affect parts of the setup outside the requested area.	Used when participants noticed or worried about unintended changes beyond the intended scope.	Do not use for ordinary edits within the expected task scope unless the participant notices, suspects, or worries that the LLM has changed something beyond the requested area.	RQ3	Trust Decreases When LLM Outputs Change Unexpectedly
Downstream pipeline risk	Weak LLM output may create later problems in the pipeline.	Used when participants connected weak output to rework, wrong data, model-training issues, or later-stage consequences.	Do not use for immediate checking only unless the participant links weak output to later pipeline consequences, rework, wrong data, training issues, or downstream effects.	RQ3, RQ4	Trust Is Limited by Downstream Risk
Early verification of downstream risk	Engineers verify early to stop small errors from propagating into later stages.	Used when participants linked early checking to prevention of later pipeline problems.	Do not use for general verification unless the participant explicitly connects early checking with preventing later-stage pipeline problems or downstream risk.	RQ3, RQ4	Trust Requires Easy Verification; Trust Is Limited by Downstream Risk

Theme development

Think-Aloud Interaction Themes				
Theme ID	Interaction-focused theme	Meaning in the think-aloud data	Participants	Final integrated theme
TA1	Participants used their own expertise to set the boundary of AI use	Participants judged whether AI support was suitable by comparing output with task complexity and their own expertise.	P02, P03	Trust Depends on Task Risk and Engineer Confidence
TA2	LLM output was used as a starting point, not a final deliverable	Participants valued AI output for structuring and orientation, but treated it as requiring inspection, review, or editing.	P01, P02, P03, P04	Trust Requires Easy Verification
TA3	Verification became the gate before accepting or progressing with AI output	Participants paused at parameter checks, explanations, fit checks, and validation points before carrying output forward.	P01, P02, P03, P04	Trust Requires Easy Verification
TA4	Prompting became a context-setting and assumption-checking activity	Participants used prompts to supply missing context, structure the task, or request assumptions before larger generation.	P03, P04	Trust Depends on Clear Context and Explanation
TA5	Participants preferred staged, editable interaction over one-shot generation	Participants wanted outlines, checkpoints, and editable suggestions before allowing the AI to continue or make broader changes.	P01, P02, P04	Trust Increases When Engineers Have Control
TA6	Unstable changes and repair loops made participants protect the downstream pipeline	Participants became cautious when AI changed unintended parts, produced circular fixes, or created outputs that could affect later stages.	P01, P02, P04	Trust Decreases When LLM Outputs Change Unexpectedly; Trust Is Limited by Downstream Risk

Annex 4: Behavioural coding overview

Codebook

Linked_Task	Behaviour_Category	Behaviour_Code	Definition	Include_When	Exclude_When	Final_Theme_Link
T01	Navigation behaviour	NAVIGATES_PROJECT	Participant opens project areas, switches views, or orients to dashboard/workspace	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Depends on Clear Context and Explanation
T02	Configuration behaviour	CONFIGURES_REQUIREMENTS	Participant enters, reviews, or adjusts project configuration and requirements	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Depends on Clear Context and Explanation
T03	Import behaviour	IMPORTS_OR_SELECTS_DATA	Participant imports, selects, or locates assets/datasets/source data	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Depends on Task Risk and Engineer Confidence / Trust Depends on Clear Context and Explanation
T04	Annotation setup behaviour	SETS_LABEL_SCHEMA	Participant defines label type, class schema, inclusion criteria, or annotation setup	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Requires Easy Verification / Trust Is Limited by Downstream Risk
T05	Pipeline construction behaviour	CONFIGURES_NODES	Participant adds/selects pipeline modules/nodes or constructs the pipeline graph	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Depends on Clear Context and Explanation / Trust Increases When Engineers Have Control
T06	Parameter interaction behaviour	ADJUSTS_PARAMETERS	Participant changes parameter values, ranges, or settings	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Requires Easy Verification / Trust Increases When Engineers Have Control
T07	Connection behaviour	CONNECTS_OR_CONFIGURES_PIPELINE	Participant links nodes/modules or configures pipeline connections	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Depends on Clear Context and Explanation / Trust Decreases When LLM Outputs Change Unexpectedly
T08	Verification/correction behaviour	PREVIEWS_VALIDATES_AND_FIXES	Participant previews, validates, identifies issues, corrects, or re-checks	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Requires Easy Verification / Trust Decreases When LLM Outputs Change Unexpectedly
T09	Generation behaviour	RUNS_GENERATION	Participant starts or monitors generation/batch output process	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Is Limited by Downstream Risk
T10	Verification behaviour	CHECKS_OUTPUT_QUALITY	Participant inspects generated output, quality, save state, or sample results	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Requires Easy Verification / Trust Is Limited by Downstream Risk
T11	Dataset preparation behaviour	CURATES_TRAINING_DATASET	Participant creates/curates dataset, chooses images, or sets train/validation split	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Is Limited by Downstream Risk
T12	Annotation verification behaviour	REVIEWS_ANNOTATIONS	Participant reviews labels/annotations and prepares dataset for downstream use	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Requires Easy Verification / Trust Is Limited by Downstream Risk
T13	Model workflow behaviour	CONFIGURES_MODEL_TRAINING	Participant selects, uploads, or configures model/training workflow	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Is Limited by Downstream Risk
T14	Evaluation behaviour	REVIEWS_EVALUATION_OUTPUT	Participant checks evaluation, reports, downstream testing, or quality/privacy results	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Is Limited by Downstream Risk
T15	Reporting/deployment behaviour	DOCUMENTS_DEPLOYS_OR_MONITORS	Participant documents, deploys, shares, or monitors the pipeline/workflow	Code when the participant performs or completes this workflow action in the relevant task context.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Trust Is Limited by Downstream Risk
	Task completion marker	TASK_COMPLETION_CONFIRMED	Used when the user confirmed all participants completed the task but the source workbook does not separate a detailed non-teaching behavioural event	Use only when task completion is confirmed but a detailed non-teaching behavioural event is not available.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Task-level completion marker; not a final theme
	Excluded material	EXCLUDED_TEACHING_COACHING	Teaching/coaching/screen-sharing instruction content excluded from behavioural coding	Use only in the exclusion log to document material removed from behavioural coding.	Do not code host/session management notes, or another participant's action as this participant's behaviour.	Excluded material; not a final theme

LLM opportunity justification

Thesis LLM support function	Related task IDs / behaviour codes	Main friction evidence from master log	Friction indicators supporting the function	Evidence strength	Justification for adding or exploring LLM support	Methodological caution	Trust-informed design safeguard
Early requirement and project configuration support	T01 / NAVIGATES_PROJECT; T02 / CONFIGURES_REQUIREMENTS; T03 / IMPORTS_OR_SELECTS_DATA	This is strong area. The evidence is mainly from setup, requirement review, and asset/source handling. P02 spent time navigating the project and viewing the Requirements tab, while P04 described setting up project requirements and definitions. For T03, P04 downloaded/exported assets and the log explicitly notes struggle during asset/source handling. However, T01 and T02 include limited evidence of direct task friction compared with later pipeline stages.	Orientation effort; requirement-review time; asset/source handling effort; export/download activity; reported struggle in asset handling	Strong	This function can be justified as an exploratory support opportunity because early project setup and asset/source selection shape later pipeline work. LLM support could help clarify requirements, summarise project setup choices, or guide asset/source selection. However, the behavioural evidence is weaker than for construction, parameter, validation, and generation tasks.	Do not claim this was strongly proven by behavioural coding. Present it as a cautious, early-stage opportunity supported mainly by setup and asset-handling evidence.	Support should make assumptions visible, allow engineers to edit requirements, and avoid automatically changing project definitions.
Pipeline construction and parameter support	T05 / CONFIGURES_NODES; T06 / ADJUSTS_PARAMETERS; T07 / CONNECTS_OR_CONFIGURES_PIPELINE	This is the strongest opportunity area. P02 spent 600s working in the node graph and 420s adjusting Random Number nodes and parameter values. P03 and P04 both expressed uncertainty about whether a value was in degrees or radians. P03 and P04 also both encountered a major linking/configuration error where the subject was connected to itself. These events show long duration, uncertainty, technical judgement, and clear configuration error.	Long duration; repeated configuration effort; unit confusion; parameter uncertainty; manual judgement burden; connection error; correction/backtracking risk	Strong	The behavioural evidence strongly justifies LLM support for node construction, parameter explanation, unit clarification, and pipeline-connection checking. The support could reduce configuration burden by explaining node roles, warning about invalid links, and suggesting parameter ranges while keeping engineers in control.	The evidence supports exploring assistance, not full automation. The LLM should not silently construct or change nodes without review.	Support should be staged, reviewable, editable, and reversible. It should explain parameter assumptions and highlight risky pipeline connections before changes are applied.
Validation, preview, and output quality support	T08 / PREVIEWS_VALIDATES_AND_FIXES; T09 / RUNS_GENERATION; T10 / CHECKS_OUTPUT_QUALITY	This area is also strongly supported. P03 said they were lost and encountered an error during validation/preview. P04 expected an asset to appear but said it did not. During generation, P03 and P04 both complained that generation was 'very black' or lacked visibility and asked how to wait or understand what was happening. P02 later inspected the generated dataset and save state. These events indicate validation friction, unmet expectations, lack of visibility, monitoring burden, and manual	Confusion; error; unmet expectation; missing/unclear output; lack of visibility; monitoring burden; manual quality checking	Strong	The evidence justifies LLM support for explaining validation errors, suggesting fixes, improving generation-progress visibility, and assisting output quality review. These functions would address points where participants needed to understand what had gone wrong, whether generation was progressing, and whether outputs were suitable.	Output quality checking is supported by fewer detailed events than validation and generation visibility. Present quality support as connected to validation/generation rather than as equally strong standalone evidence.	Support should explain errors and generation status clearly, provide fix suggestions without applying them automatically, and enable engineers to verify outputs before reliance.
Dataset, annotation, and evaluation support	T04 / SETS_LABEL_SCHEMA; T11 / CURATES_TRAINING_DATASET; T12 / REVIEWS_ANNOTATIONS; T13-T15 completion-only context	This area is strongly supported, mainly by label/schema and dataset-preparation friction. P03 was confused about why an item appeared twice during labelling/schema setup. P04 was confused about how labelling should be done and whether a stack should be labelled as one stack. In T11, P04 asked whether it was 'just train' and learned the correct step order of selecting images first and then continuing. P02 reviewed labelled captures/images in T12. T13-T15 are completion-only records and do not provide	Label/schema confusion; annotation decision burden; dataset step-order uncertainty; manual annotation review; downstream dependency; completion-only evidence for later stages	Strong	This function can be justified where annotation and dataset preparation affect downstream quality. LLM support could help explain label/schema choices, guide dataset curation steps, and flag annotation inconsistencies. Evaluation and documentation support should be framed more cautiously because the updated log does not contain detailed friction evidence for T13-T15.	Do not overclaim strong evidence for model training, evaluation, documentation, deployment, or monitoring. The strongest support in this category comes from T04 and T11, with T12 as limited supporting evidence.	Support should flag issues and explain consequences rather than replacing engineering review. It should make downstream risks visible and keep final dataset/annotation decisions under engineer control.

Annex 5: Analytical Traceability Across Thematic Analyses

The purpose of this annex is to improve the transparency and traceability of the analytical process used in this study.

The trust-related components developed from the literature were used as sensitising concepts during the deductive stage of hybrid thematic coding. The analysis also remained open to inductive patterns that emerged from the interview and think-aloud data. As a result, some components were retained, some were merged with related components, and others were repositioned or treated as background concepts depending on the empirical evidence. Therefore, not all trust-related components were used equally or retained as separate final themes.

Status terms used in Tables A and B:
Retained = clearly visible in the final theme.
Merged = combined with related components.
Repositioned = interpreted under a broader theme.
Background/weakly present = considered during analysis but not strong enough to become a separate final theme.

Table A: Interview thematic analysis traceability

This table shows how the interview analysis combined framework-informed sensitising concepts with inductive interview patterns.

Trust-related framework component / sensitising concept used in interview analysis	Role in interview coding	Inductive interview pattern or code	Status in final analysis	Final integrated theme
Accuracy and reliability	Guided attention to whether participants expected LLM outputs to be correct, reliable, and suitable for use.	Accuracy expectations; expectation–experience comparison; task-complexity calibration.	Merged / repositioned	Trust Depends on Task Risk and Engineer Confidence
Predictability	Guided attention to whether outputs appeared stable and repeatable across use situations.	Predictability; iterative instability; hidden changes; counter-prompt loops.	Retained	Trust Decreases When LLM Outputs Change Unexpectedly
Verifiability	Guided attention to whether engineers could inspect, test, or check outputs before relying on them.	Verification-related concern; need to double-check; output accepted only after inspection.	Retained / merged with verification cost	Trust Requires Easy Verification
Transparency and reasoning	Guided attention to whether participants needed explanations, reasoning, or local/global understanding.	Explanation-related concern; requests for why or how the model suggested something.	Retained / merged with context understanding	Trust Depends on Clear Context and Explanation
Control and user agency	Guided attention to whether engineers wanted to steer, edit, approve, or override LLM support.	User control; controlled LLM inclusion; core-data boundary; assistive rather than autonomous use.	Retained	Trust Increases When Engineers Have Control
Perceived competence	Guided attention to whether users perceived	Perceived competence over time; early errors; task complexity as a boundary of competence.	Merged / repositioned	Trust Depends on Task Risk and Engineer Confidence; Trust

	the system as capable after repeated experience.			Decreases When LLM Outputs Change Unexpectedly
Known limitations	Guided attention to recognised boundaries where LLM support was considered weak or unsuitable.	High-stakes domain limits; cybersecurity/compliance concerns; text-recognition and social limitations.	Repositioned / merged	Trust Depends on Task Risk and Engineer Confidence
Context understanding	Guided attention to whether the LLM appeared to understand the local task, domain, and pipeline context.	Prompt dependency; prompt-structure concern; need to define the task clearly.	Retained	Trust Depends on Clear Context and Explanation
Workflow fit	Guided attention to where LLM support did or did not fit within the pipeline workflow.	Workflow fit/redesign; early framing value; templates and problem decomposition.	Repositioned / merged	Trust Depends on Clear Context and Explanation; Trust Is Limited by Downstream Risk
Uncertainty indication	Guided attention to whether the system made uncertainty, limits, or confidence visible to users.	Weakly present through explanation and checking concerns rather than as a separate code cluster.	Background / weakly present	Trust Depends on Clear Context and Explanation; Trust Requires Easy Verification
Verification cost	Guided attention to the effort required to check outputs before use.	Verification effort; difficult-to-validate outputs; compliance and UI checks as higher-cost cases.	Merged with verifiability	Trust Requires Easy Verification
Risk and consequences of errors	Guided attention to whether weak outputs could create later consequences in the pipeline.	Downstream concern; weak data, labels, or settings compromising later stages.	Retained	Trust Is Limited by Downstream Risk
Experience and trust evolution	Guided attention to how trust changed through	Reduced or selective use over time; past experience/advice; calibration through use.	Repositioned / merged	Trust Depends on Task Risk and Engineer Confidence; Trust

	repeated use and advice to others.			Decreases When LLM Outputs Change Unexpectedly

Table B: Think-aloud thematic analysis traceability

This table shows how the think-aloud verbal analysis used hybrid coding. The trust-related concepts guided attention to trust-relevant interaction moments, while the final interaction themes were shaped by inductive task-based patterns.

Trust-related framework component / sensitising concept used in think-aloud analysis	Role in think-aloud coding	Inductive think-aloud pattern or code	Status in final analysis	Final integrated theme
Accuracy and reliability	Used to interpret whether participants treated LLM output as usable, dependable, or requiring checking before use	Useful starting point requiring inspection; Parameter verification before trust	Merged	Trust requires easy verification; Trust depends on task risk and engineer confidence
Predictability	Used to code moments where participants reacted to inconsistent, unstable, or unexpected LLM behaviour	Prompt-to-prompt instability; Unexpected scope change	Retained through theme	Trust decreases when LLM outputs change unexpectedly
Verifiability	Used to code moments where participants checked, tested, previewed, or validated LLM output before relying on it	Parameter verification before trust; Early verification of downstream risk	Retained and merged	Trust requires easy verification
Transparency and reasoning	Used to code moments where participants needed reasons, assumptions, or explanations before accepting output	Explanation before acceptance	Merged	Trust depends on clear context and explanation

Control and user agency	Used to code moments where participants wanted stepwise, editable, or reviewable LLM support	Staged and reviewable generation	Retained as direct theme	Trust increases when engineers have control
Perceived competence	Used to interpret how participants judged whether LLM support was suitable for the task	Task-dependent usefulness of LLM support	Repositioned	Trust depends on task risk and engineer confidence
Known limitations	Used to interpret moments where participants treated LLM output cautiously or recognised limits in what could be accepted directly	Useful starting point requiring inspection; Downstream pipeline risk; Early verification of downstream risk	Merged	Trust requires easy verification; Trust is limited by downstream risk
Context understanding	Used to code moments where participants connected output quality to prompt clarity, task context, assumptions, or module context	Prompt-context dependency; Explanation before acceptance	Merged	Trust depends on clear context and explanation
Workflow fit	Used to code moments where participants found output difficult to judge or apply because it did not clearly fit the pipeline or module constraints	Workflow ambiguity; Prompt-context dependency	Merged	Trust depends on clear context and explanation
Uncertainty indication	Used to interpret moments where uncertainty was not clearly communicated and participants needed checking, explanation, or staged review	Explanation before acceptance; Parameter verification before trust; Staged and reviewable generation	Repositioned	Trust requires easy verification; Trust increases when engineers have control
Verification cost	Used to interpret the effort required to inspect, test, or confirm LLM output before use	Parameter verification before trust; Useful starting point requiring inspection	Merged with verifiability	Trust requires easy verification
Risk and consequences of errors	Used to code moments where weak LLM output could create later	Downstream pipeline risk; Early verification of downstream risk	Retained as direct theme	Trust is limited by downstream risk

	pipeline problems, rework, wrong data, or training issues			
Experience and trust evolution	Used to interpret how participants' trust developed across interaction with the two LLM-supported tasks	Cross-task trust pattern; cautious and verification-oriented trust	Cross-cutting / repositioned	Applies across the final integrated themes